NEA Project

OCR Computer Science H446

Title: Hunted Horizon

Contents

Analysis

The problem

One problem many people who play games face is not having the time to play certain games, this may be due to work/school or their daily responsibilities, and the current games being played today tend to require quite a bit of time dedicated towards it. Therefore, many people just do not play games altogether which may hinder their own well-being since they do not have as much time for themselves to have fun through playing games.

My game hopes to help those who do not have the time to play games by providing them with levels that are short but challenging and, overall, aims to allow the players to have fun and relax.

The computational methods used to solve the problem will be abstraction, decomposition, concurrence, selection, iteration, visualisation and heuristics.

Abstraction is required to make the game efficient because it will break down the data required for the problem and may help make the game more efficient since it will reduce clutter within the code and the game's visuals. Abstraction will also be used when using Object-Oriented Programming or procedural programming since once methods within classes or functions are created and tested, you wouldn't need to know how exactly the methods/functions work – you only need to know the inputs and outputs rather than the process.

Decomposition is required to break down the different parts of the game and allow for modularisation as certain parts of the game may be reused using Object-Oriented Programming (e.g., class methods for the player or enemies). This will also help the development of the game to be easier as the development will be split into tasks that will be easier to understand and create.

Concurrence is required since there are many processes that will be running simultaneously in the background of the game such as the music loop, enemy movement, player movement, the timer, etc. This may also require data to be fetched beforehand while other processes of the specific task are being processed in the background of the processing of the game as well as the tasks processing to switch between each other to make the tasks seem simultaneous for the user.

Selection is required when the player makes certain choices like attacking an enemy or getting attacked by an enemy which would deplete either the enemy or player's health. Many of the selections within the game may change variables and give an output to the user so they can see the change.

Iteration is required because it may be used the background music or player/enemy animation such as walking. This is because the processes will have to be repeated and instead of repeating code, iteration can be coded into the program.

Visualisation is especially required since it is a video game that requires a GUI, level maps, player character, enemy characters and music. All these outputs may change due to the player inputs, and this may result in graphical changes or sound changes (e.g., if a player presses the attack button, there should be the corresponding animation and sound played). Text-based output will also be needed to teach the user how to play the game and to update the player's progress within the game (e.g., when the player completes a level, the level should appear as 'Completed').

Heuristics is required for certain features of the game such as the movement and physics because instead of the correct and actual physics being applied, the game would use processes that look

and seem close to the physics. This means that the game would be able to run faster and would still have close enough solutions to the actual solution.

Stakeholders

Client

My main client is ___ who is a university student currently studying for a master's degree and is aged 22. This means that he often struggles to find time for playing video games to relax when he must do his work to meet deadlines.

Interview:

"How often do you play video games in a week?"

"I play video games sometimes – usually around 2 times a week."

"What video games do you play?"

"I usually play Fortnite and Apex Legends, but I want to play something new.

"Have you played a platformer before?"

"In the past, I've played Sackboy: A Big Adventure and Death's Door and I found these games quite entertaining and challenging and I'd like to experience that with another game."

"What emotions do you feel while gaming?"

"It makes me happy when I complete difficult levels of a game. Sometimes I get angry when the game is too difficult though."

"What genre of video games do you prefer?"

"I like story driven single-player games with a bit of multiplayer but, I don't have a direct preference, and I like to play a wide variety of games."

"What makes a good game?"

"It needs to be fun and engaging otherwise I'll get bored."

"What do you want from the solution?"

"I want a fun and engaging platformer game with an interesting spin of what's currently out there."

Using this interview, I know that my game should be a platformer game that's entertaining and has a story, levels and possibly a multiplayer feature. The game should be challenging but still fun and not overly difficult that the player can't progress.

Workers/Students

My other clients are A-level students who are around 16-17 and some also have part-time work. This means that they also may struggle to find time to play games and relax due to their responsibilities. However, there is an age difference between this larger set of stakeholders and my

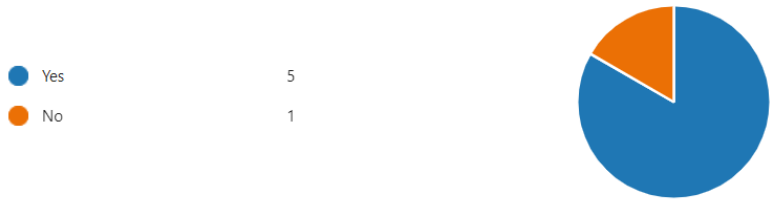
main client so this may show different tastes, and the group may have a better insight on current game trends with younger students.

Questionnaire:

I created an online questionnaire using Microsoft Forms to gain an insight to all the students' opinions and views. The questions were mainly qualitative rather than quantitative because then the data is easier to understand since it gives an overall view of the sample group.

1. Do you play video games?

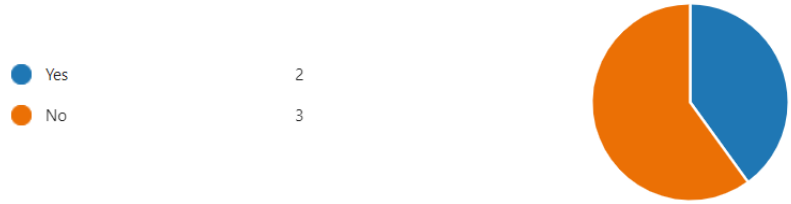
More Details



This first question ensures that all the participants in the survey play video games to get accurate information for the rest of the questionnaire. If the participant selects 'No' then the questionnaire ends. Most of the participants selected 'Yes' so, they continue to the next question.

2. Do you have a job?

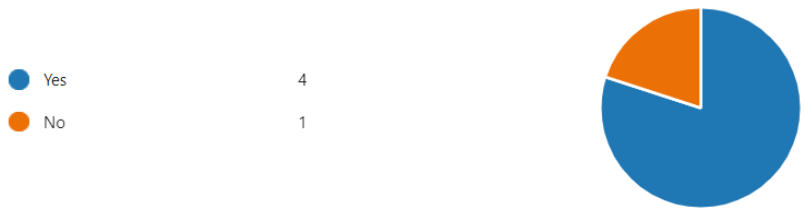
More Details



This question was to understand how many participants have less free time compared to the normal A-level student because of their job. This is because it would impact whether they get the time to relax or play video games. Most participants selected 'No' so it's more likely that they have enough free time.

3. Do you feel like you get enough time to relax and have fun?

More Details



Most participants answered 'Yes' on this question which means that my original may have been incorrect, at least with these participants. However, I can still use the participants' opinions to help my game appeal to more audiences as the participants are younger than my client.

4. Do you usually play single-player or multi-player games?

More Details

Single-player	1
Multi-player	0
Both	4

This question helped me understand whether my game should generally be single-player or multi-player according to the participants' tastes.

5. Do you prefer playing longer or shorter games?

More Details

Longer	1
Shorter	1
No preference	3

The participants seem to have either no preference or a mixed preference. This means that my game can be any length without much difference in enjoyment from players.

6. What are some of the games you currently play/like?

5 Responses

ID ↑	Name	Responses
1		Fall guys
2		Minecraft(bedwars, and my worlds), Roblox, (Mexico border simulator, Barbie, cheese escape, dress to impress, skateboard obby) F1 2023,
3		Roblox(Barbie, dress to impress, tycoons, obbies, survival)
4		call of duty
5		roblox speed draw + roblox obbies

This question helps me to understand the genres of video games the participants play. Most the games listed are multi-player games except Minecraft but, there's a range of types of multi-player games such as FPS games, fashion and art games, platformers, racing games, and more. There are also a few team games but most of them have a focus on a player winning alone so this may influence how I might make a multiplayer game.

7. Do you like platformer games (e.g. Roblox obbies)

More Details

Yes	5
No	0

This helps me understand whether platformer games appeal to younger audiences and all the participants do.

8. What platformer games do you like?

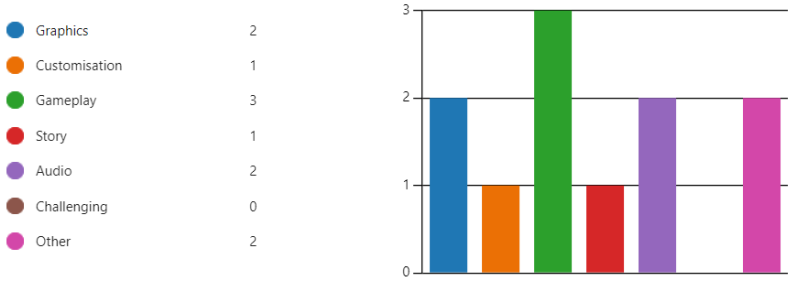
5 Responses

ID ↑	Name	Responses
1		roblox
2		Roblox, as mentioned above (the mimic, doors)
3		I'm so good at them- Roblox obbies
4		minecraft
5		roblox obby

Most of the participants stated Roblox so I will try to implement ideas from Roblox obbies into my game so that it will appeal to everyone. One participant said 'minecraft' which is like Roblox but has extra materials that are used to make the platformer difficult such as slime that the player can bounce on and cobwebs that slow down the player. Therefore, I will also take into consideration the use of special materials that affect the player's ability to move.

9. What did you like about these platformer games?

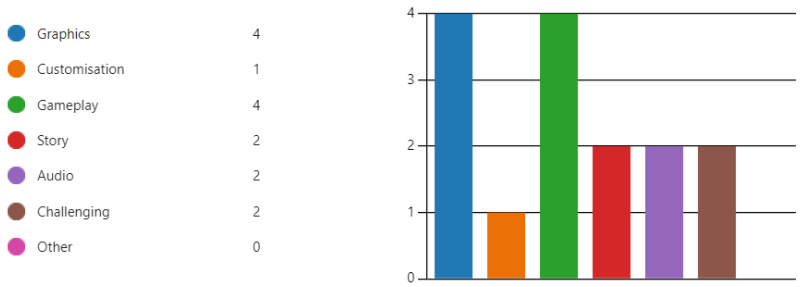
More Details



This question helps me understand what aspects of the game I should focus on to make sure that it appeals to everyone. The participant can select up to 3 of the options listed and the main aspect selected was the gameplay. The graphics and audio were also important to the participants which means I should ensure that they are selected or created with the influence of the participants in future questionnaires. Two participants selected Other and said, "Challenging but not too challenging" and "Playing with friends". This means that the game shouldn't be too difficult that the player can't complete it (like what the main client stated) and that I have to consider multiplayer aspects.

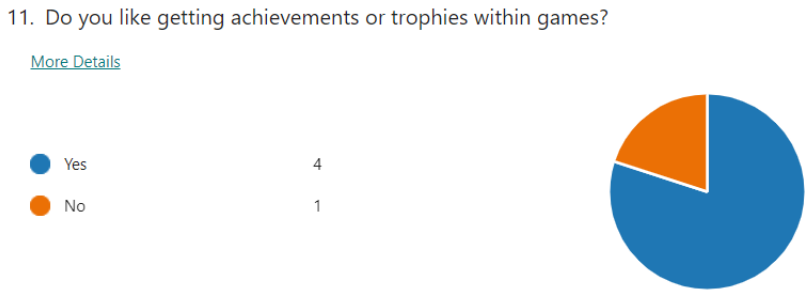
10. What three aspects do you value most in a video game?

More Details

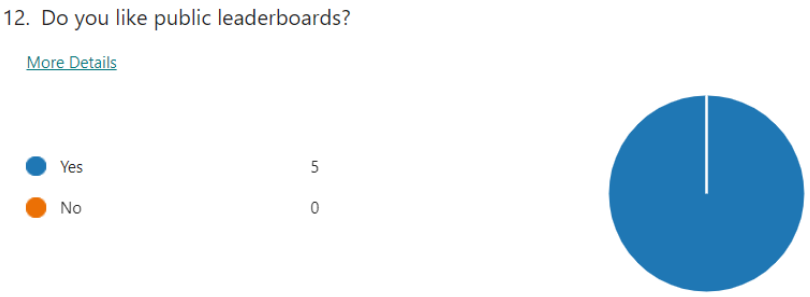


This question helps me understand what the participants prefer overall in video games and not just in platformer games so I can understand the differences between the needs of a

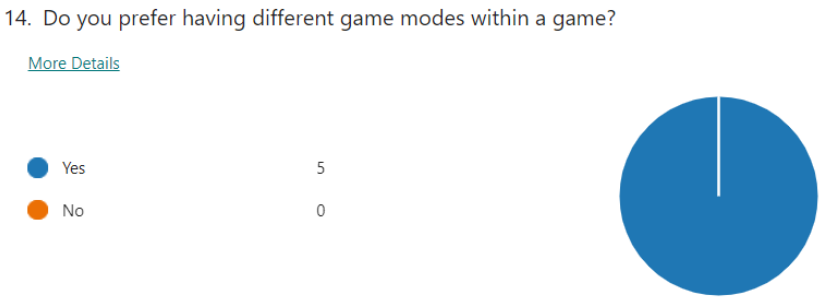
platformer game and any other game. This question made the participant choose three of the options and the main two options that were chosen were graphics and gameplay. Gameplay is a main aspect in both questions which shows that the gameplay is a vital aspect in all games and that I should ensure that the gameplay of my game is entertaining and varied enough.



This question helped me whether having a trophy system within the game affects the player's enjoyment of the game or not. Most participants answered 'Yes' which means that it does affect the player and that having a trophy system is beneficial for the player.



Every participant selected 'Yes' which means that it may also be beneficial to have a public leaderboard in the game – it would also be a multiplayer element. If a participant selected 'No' then the next question would be why they don't like public leaderboards but since no one selected 'No', there's no data from that question.

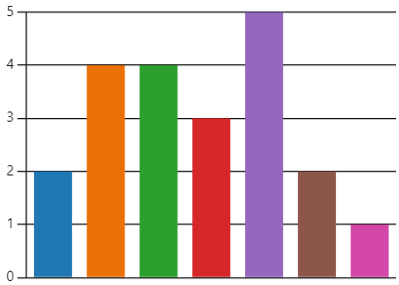


Every participant selected 'Yes' for this question as well which means that the participants prefer variety than sticking to one thing. This means that I may include different game modes within my game.

15. What sorts of game modes do you like?

More Details

Story mode	2
Survival/Arcade mode	4
Level mode	4
Difficulty modes (Easy, Normal, ...	3
Multiplayer mode	5
Creative mode	2
Other	1



This question allowed the participant to select all the game modes they liked, and the most popular game mode chosen was multiplayer mode with both survival/arcade mode and level mode the second most popular. The participant that chose Other said a “Fighting” mode which could also come under a survival/arcade mode. Therefore, I should try including all the three modes within my game – starting with the level and survival/arcade mode since a multiplayer mode would be easier to create afterwards.

16. Do you like boss battles within games?

More Details

Yes	4
No	1

This question helps me to understand whether adding a boss to the game would be beneficial and many of the participants liked boss battles so I may think about adding one within my game.

17. What do you think makes a good boss battle?

More Details

Challenging	4
Soundtrack	1
Boss Design	1
Other	0

If I add a boss battle, this question will help me to understand what aspects I should be focusing on. This question also had no limit on the number of choices chosen but, the main aspect chosen was that the boss should be challenging. Therefore, I should ensure that any boss I add into the game is challenging and fun for the player.

18. Do you prefer having a story in a game?

More Details

Yes	5
No	0

This question helped me to see whether players may care about having a story to follow in the game or not. All the participants selected 'Yes' which means that I should ensure that my game has an entertaining story to go along with the rest of it.

Existing Solutions

Sackboy: A Big Adventure[1]

This is a **3D platform game** that was developed by Sumo Digital and is both **single-player and multi-player**. The game has a **story** where Sackboy (the player) must save Craftworld by completing levels and collecting Dreamer Orbs to progress and eventually fight the antagonist Vex.

Levels:

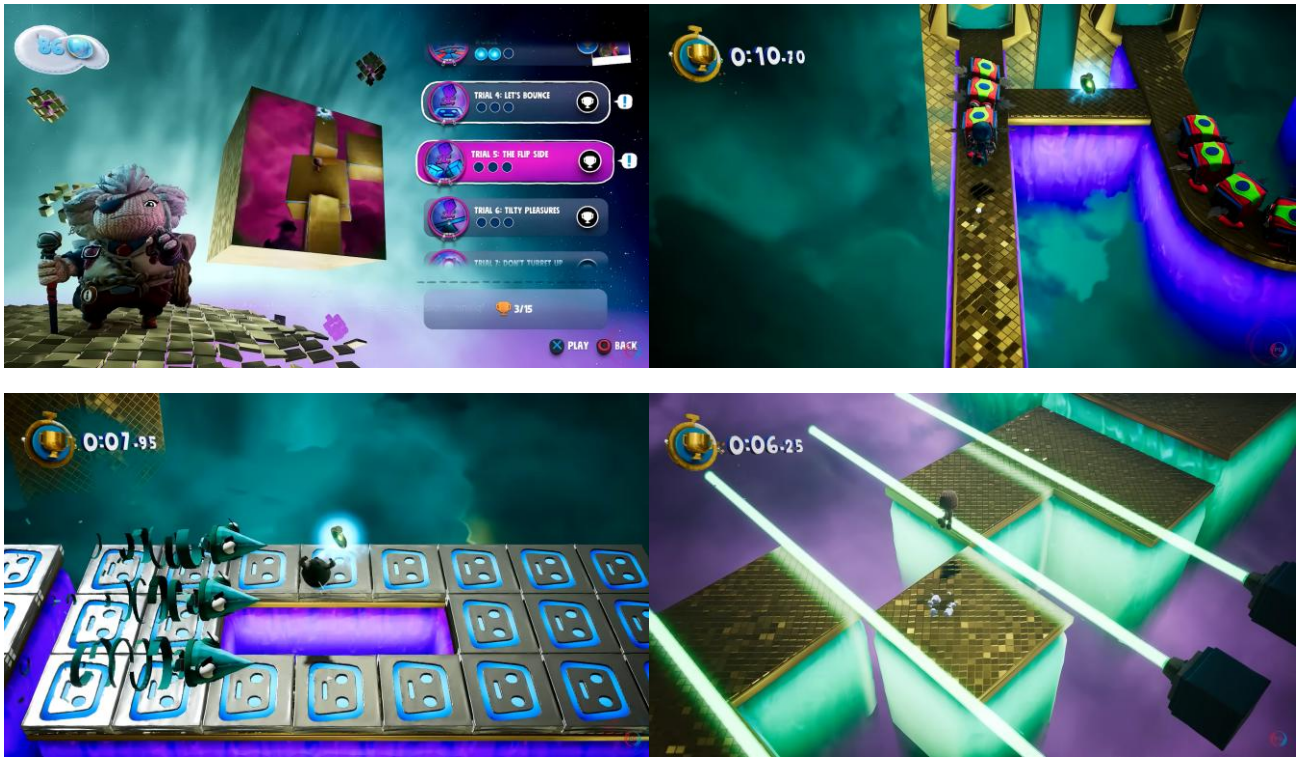
Most of the levels are single player but there are also levels made to be played with another player which can be **local** or **online** with a limit of **4 players**. Each level challenges the player with different layouts, **items that can be equipped**, different enemies, **hidden areas**, attack methods and more. Each of the levels also have a **unique design and soundtrack** that plays and may change as the player makes their way through the level. The levels are grouped in their own planet and each planet is themed differently – that means that all the levels within the world follow that theme. For example, The Soaring Summit has a winter and spring theme, The Colossal Canopy has a rainforest and jungle theme, The Kingdom of Crablantis has an ocean theme, The Interstellar Junction has a futuristic theme, and The Centre of Craftworld has a generally distorted theme.

(Screenshots of some of the levels, sourced from PLAYGAMES YouTube[2])

Collectables + Mini game:

The game also has another collectable called **Knightly Energy** that **unlocks a level in another game mode** called the **Knitted Knight Trials** where players can challenge themselves to complete the level within strict timing to get more **Dreamer Orbs according to your time taken**. There are **gold, silver and bronze times** with gold getting you all three Dreamer Orbs from that Knitted Knight Trial.

The different Knitted Knight Trials can be unlocked when finding the Knightly Energy in normal levels.



(Screenshots of the Knitted Knight Trial Menu and some of the Trial levels, sourced from PLAYGAMES YouTube)

Main map:

Sackboy: A Big Adventure also has a **main map where the player can navigate** from different worlds and choose levels. **Some levels can only be unlocked after gathering enough Dreamer Orbs** which can be obtained in levels and the Knitted Knight Trials. **Some levels also require you to play with another player** either through LAN or online.

(Screenshot of main map, sourced from PLAYGAMES YouTube)

Boss battles:

At the end of each world's levels, there is a boss battle level which has a **part of a normal level and then goes into a boss battle**.

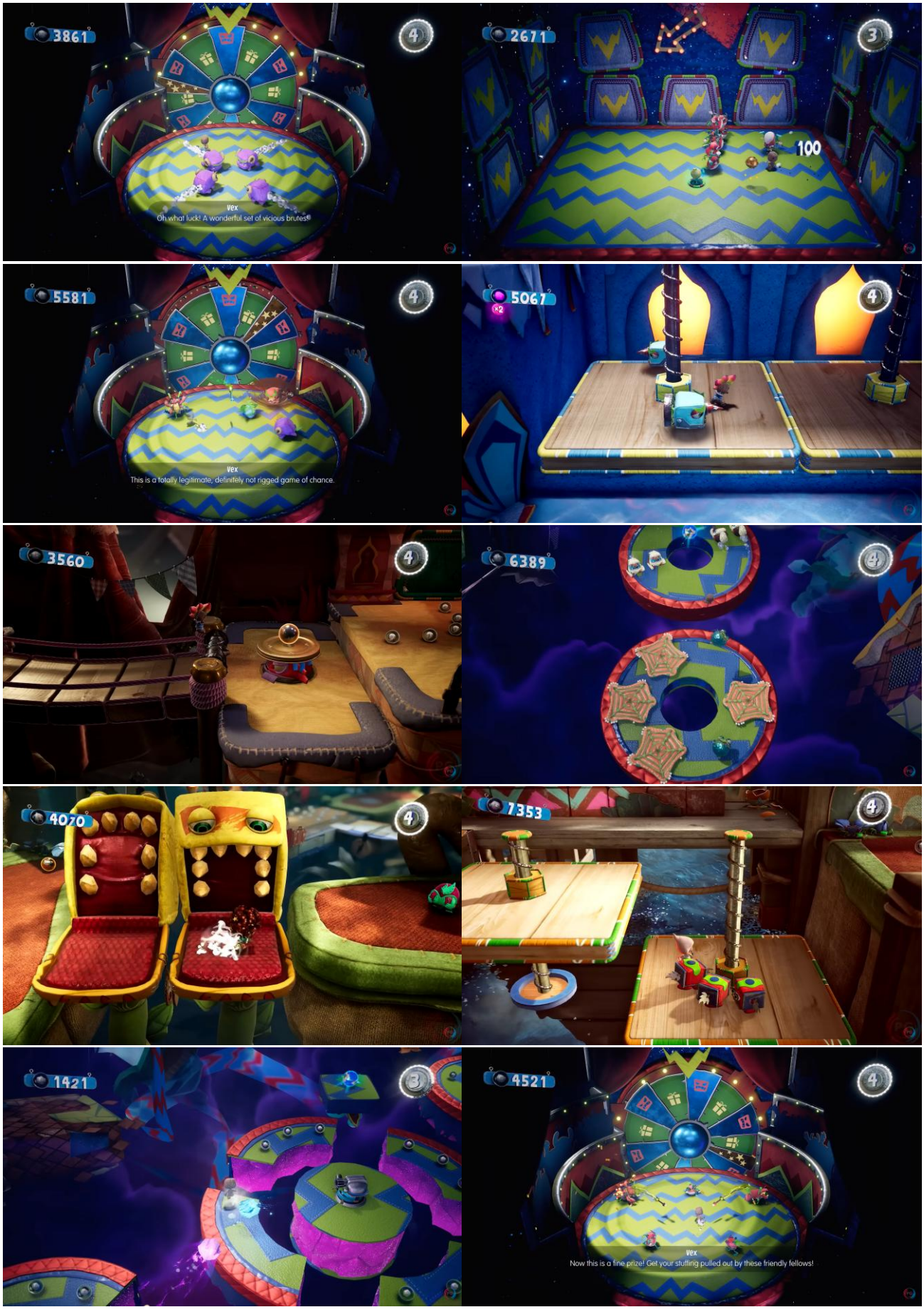
(Screenshots of some boss battles, sourced from PLAYGAMES YouTube)

Movement:

Unlike the previous games in the LittleBigPlanet series, Sackboy has a larger range of movement. Along with the normal **walking or running NESW** and **jumping**, Sackboy can also **attack enemies by slapping** them, **nosediving** on them, and leading them off the platform as well as other controls like **rolling** around, **picking up objects**, **flutter jumping** and more.

Enemies:

The game also has many different styles of enemies that **attack and can be defeated differently**. One of the enemies is a bull-like enemy that is purple and square which **charges towards you**, another enemy is a small red creature that slaps you and they can **stack on top of each other and then flip onto the player**. There's also a larger red enemy that **attacks the player by spinning** into them but also has a cooldown to its spin where the player can then attack the enemy. Another enemy **floats** and only moves in a specific way but while moving, it has a sharp head that can attack the player. One enemy **shoots out obstacles** from a hole and can **cover itself once the player comes near**, but the player can stun the enemy by nosediving and then attack them. One enemy doesn't attack the player but hinders them by **trapping them** inside of the enemy if the player stands on top of the enemy while its mouth is open. Like the previous enemy, there's a giant enemy that does kill the player once they have been eaten and the enemy snaps down on the player when they are in the enemy's mouth. There are also enemies that walk together in a line and **can only be attacked by the top of its body** otherwise it attacks the player from the sides. There are also many other enemies like a scorpion-like enemy that **shoots arrows** at the player, an enemy that flips onto the player, a jellyfish-like enemy that also walks in a line but can't be attacked by usual methods, an enemy that **shoots out plasma orbs** and many more enemies.



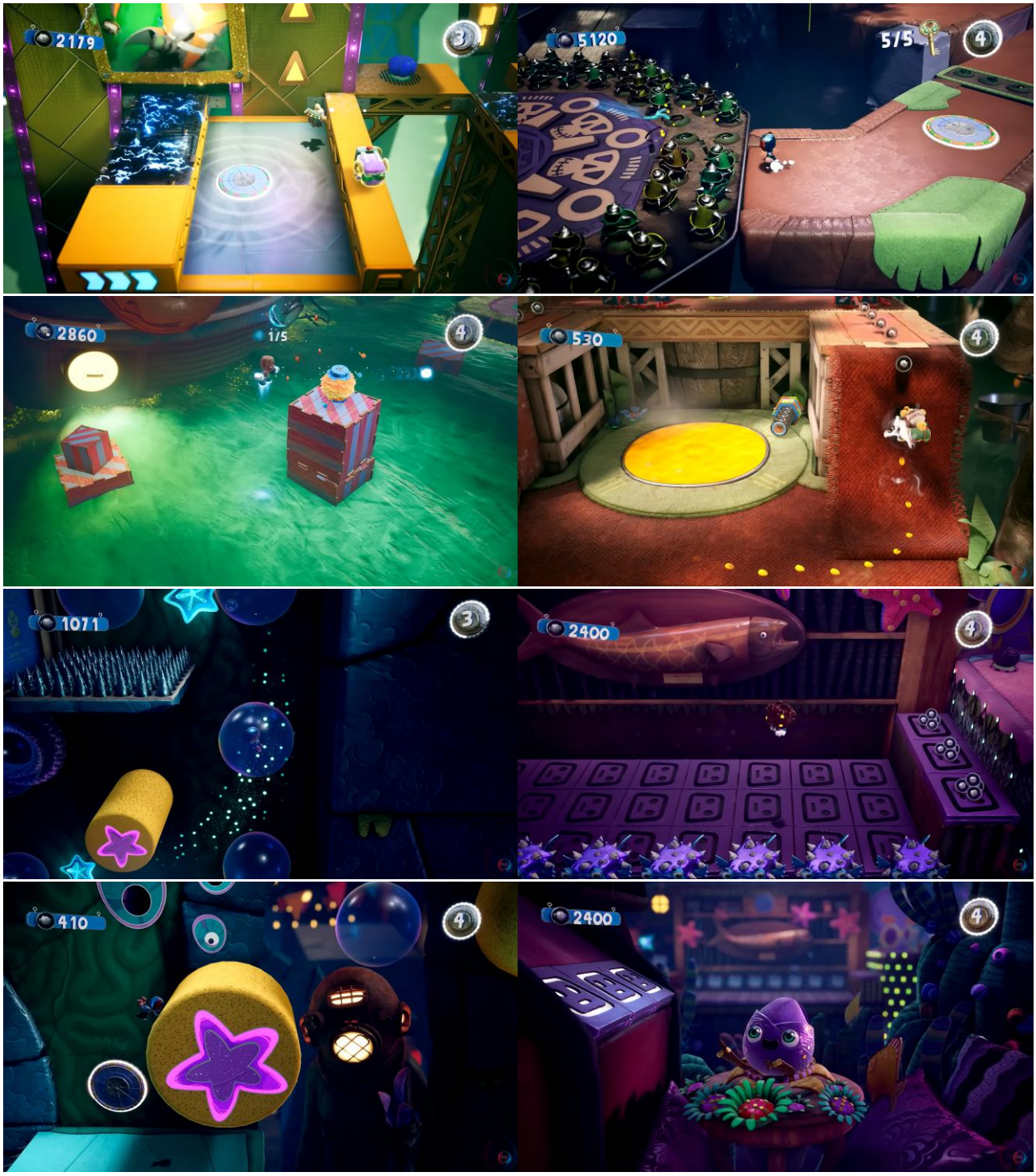
(Screenshots of different game enemies, sourced from PLAYGAMES YouTube)

Obstacles:

Within levels, the player can die from **enemies**, **falling off the map**, and by certain **terrain** or **objects** (e.g., there are surfaces that can **electrocute** Sackboy, there are cacti that can **prick** Sackboy, there is slime that can **drown** Sackboy, and more) like the previous LittleBigPlanet games.

Travelling:

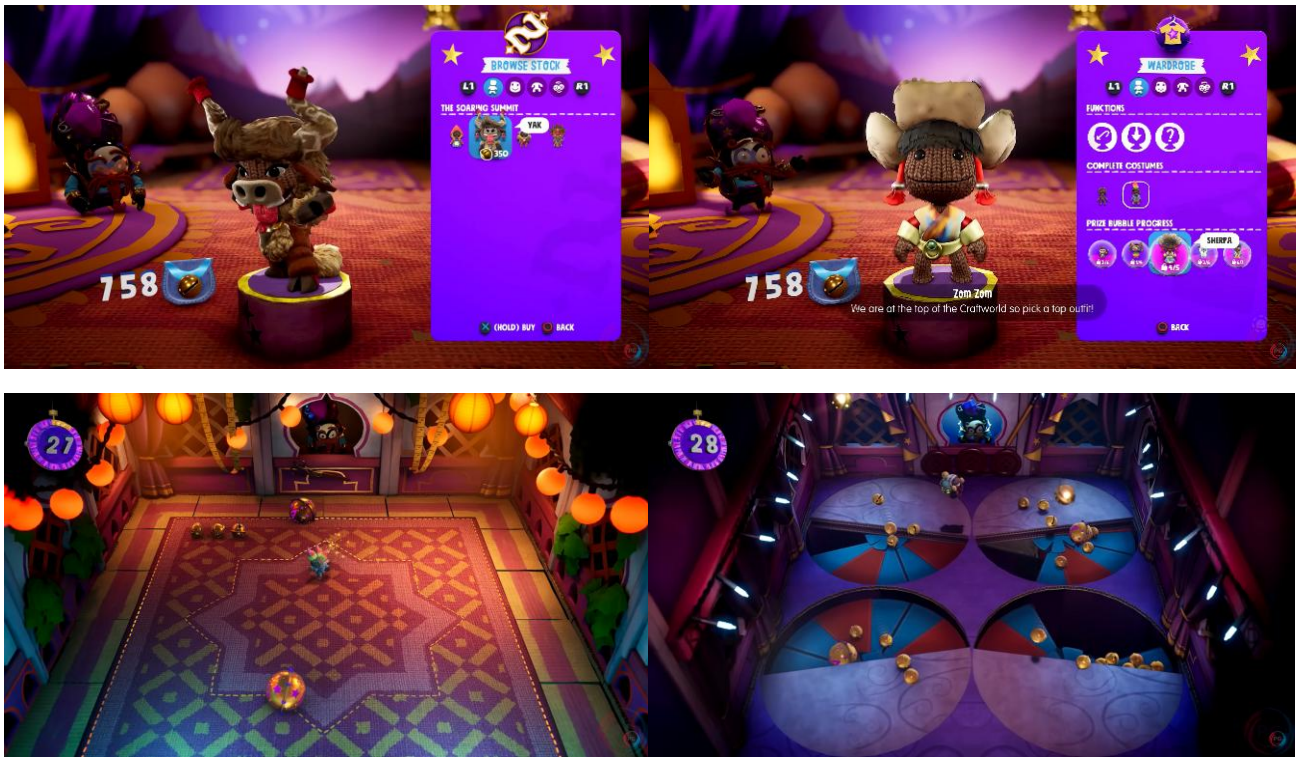
There's also a **yellow goo** that the player can use to **walk on walls**, **bubbles** that the player can use to travel, **jump pads**, materials that the **player can grab and hold onto** and a **flower** that **launches the player** to other platforms.



(Screenshots of some of the level gameplay features, sourced from PLAYGAMES YouTube)

Customisation + Mini games:

The player can also get other items called **Collectibells** in levels, the main maps, and in Zom-Zom's shop which can be used to **buy items** at the shop to **customise their Sackboy**. This is in addition to the **prize bubbles** that are found in levels that can also be used for customisation. Zom-Zom's shop also has a **timed mini-game** every time you go to a new one of their shops that allows the player to gather more Collectibells and, if in multi-player, play against the other player.



(Screenshots of Zom-Zom's shop and mini-game, sourced from PLAYGAMES YouTube)

Trophies:

The game is also playable on PlayStation so it has **PlayStation trophies** you can gain from playing the game by completing the story, completing the Knitted Knight Trials, completing specific tasks, and a trophy for getting all the other trophies.

This existing solution doesn't fix my initial problem because Sackboy: A Big Adventure **requires you to spend quite a lot of time** to complete levels. Although the Knitted Knight Trials are quicker than the main levels, they **require you to have found the Knightly Energy** within the main levels to play – meaning they still require time. Plus, players may not always find where the collectable is and would either search for someone else's help or never find the collectable and not play the Knitted Knight Trial. **Some levels are also only for two or more players** which means that players may also not play those levels if they can't find anyone to play with them. In addition, the boss levels are usually quite long. These factors aren't ideal for players that simply wish to play a quick game without too much hassle.

For my solution, I'm going to implement some of the features of Sackboy: A Big Adventure such as the **main map** where all the levels will be displayed and can be accessed after completing the previous level. The main map will also contain a side part that leads to the **arcade mode**. Like how

the Knitted Knight Trials are timed, the **main levels will be timed**, and **trophies** can be rewarded to players for completing the level in a specific amount of time. Although the level designs for many of Sackboy: A Big Adventure's levels are complex, I will be using some of the simpler parts of levels to reference when creating my game's levels (such as the **positioning of enemies and obstacles**) to ensure the game is challenging. I will also take inspiration from the last boss battle in Sackboy: A Big Adventure to help create my **mini boss** since it is less complex compared to its other boss battles however, I won't have the mini boss battle contain a level prior to it for the sake of simplicity. I may also add **hidden items** that the player can find to get trophies like the Dreamer Orbs or Knightly Energy although these **collectables won't bar the player from playing levels or the arcade mode** for my game. In addition, I will use Sackboy: A Big Adventure's enemies to help me create unique **enemies** that attack the player and can be attacked in different methods. I may also take inspiration for possible **obstacles** I can add into my game to increase the difficulty for the player.

Death's Door[3]

Death's Door is a **3D isometric action-adventure game** that was developed by Acid Nerve that is **single-player**. The game has a **story** where the player is a small crow reaper that collects souls for the Reaping Commission Headquarters (RCH) and while a reaper has an assigned soul to collect, the reaper is mortal. The first soul assigned to the player is stolen by the Grey Crow and then used by the Grey Crow to enter Death's Door, but it fails which means that the player remains mortal for the rest of the game. The Grey Crow explains that the player needs to gather three Giant Souls to enter Death's Door but as you play the game, the player finds more information about the world and then must fight the Lord of Doors (the leader of the RCH) to return the world back to its normal state after the Lord of Doors abused their power for immortality. After this, you can continue to play the game for its **true ending**.

Movement:

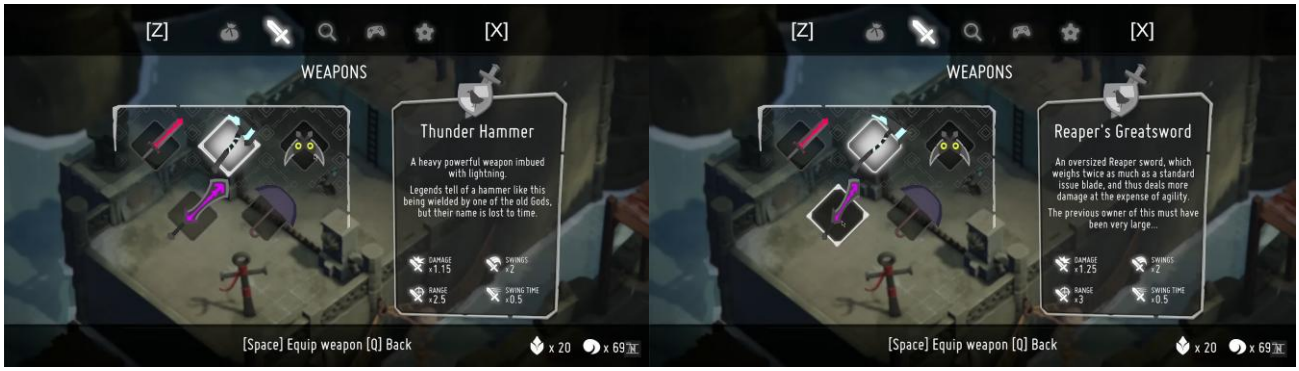
The player can **move in every direction** and **dodge** but **can't jump**, the player can also **attack using a sword or magic abilities** that are long ranged. The player first starts out with only their **sword** and **bow** but as they progress through the game they can get **Fire**, **Bomb** and **Hookshot** abilities.

Magic abilities:

All magic abilities can also be upgraded by defeating a mini-boss, the Fire ability upgrade allows the fire to continue to hurt enemies when they've been attacked with it, the Bomb ability upgrade stops the player from getting injured from using the bomb, the Hookshot ability upgrade allows you to perform a powerful attack on enemies when you attack them right after grappling onto them, and the arrow ability upgrade allows you to charge your arrow for longer so it does more damage.

Weapons:

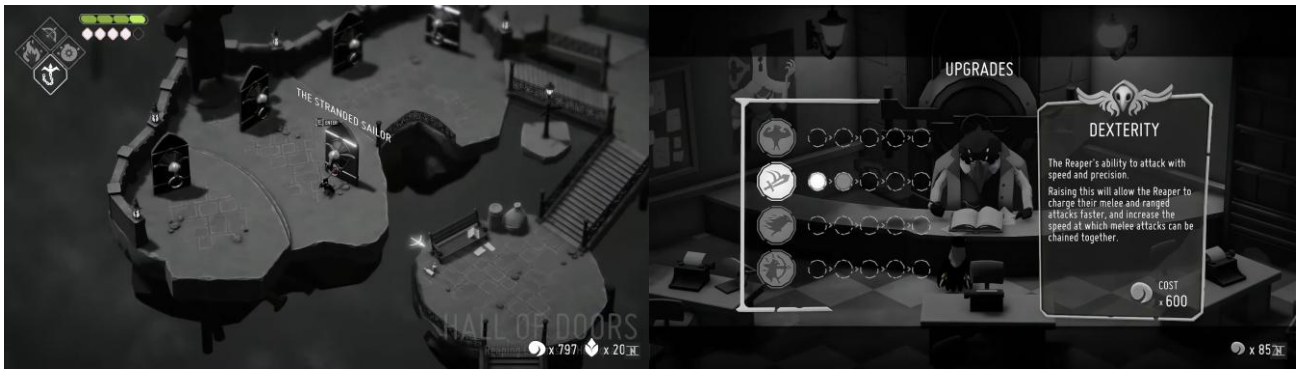
There are also **five different weapons** that the player can use: four of them must be found by the player around the different areas and one is already with the player (Reaper's Sword). The **weapons all have different stats** (damage, swings, range, swing time) and some also have **extra abilities** such as the Thunder Hammer doing **Area-Of-Effect damage** (damages multiple enemies in an area at the same time). When the player uses one of their magic attacks, the **player can regain their magic by using their melee weapon against enemies or specific objects** such as urns or mushrooms.



(Screenshots of the weapons menu, sourced from NeatPick YouTube[4])

Main map + Upgrading player:

Like Sackboy: A Big Adventure, this game also has a **main map where the player can travel to the different areas** if they have been there before and have placed the door down. Within this main map, the player can also access the **Soul Vault** where they can **upgrade their character using souls they have gained**. The player can upgrade their **Strength, Dexterity, Haste and Magic by five levels**: Strength increases the damage done with melee weapons and their reach, Dexterity allows the player to charge their attacks faster and increase the speed of chained melee attacks, Haste increases the player's base speed, their dodge speed and reduces cooldown time between dodges, Magic increases the damage of ranged abilities.

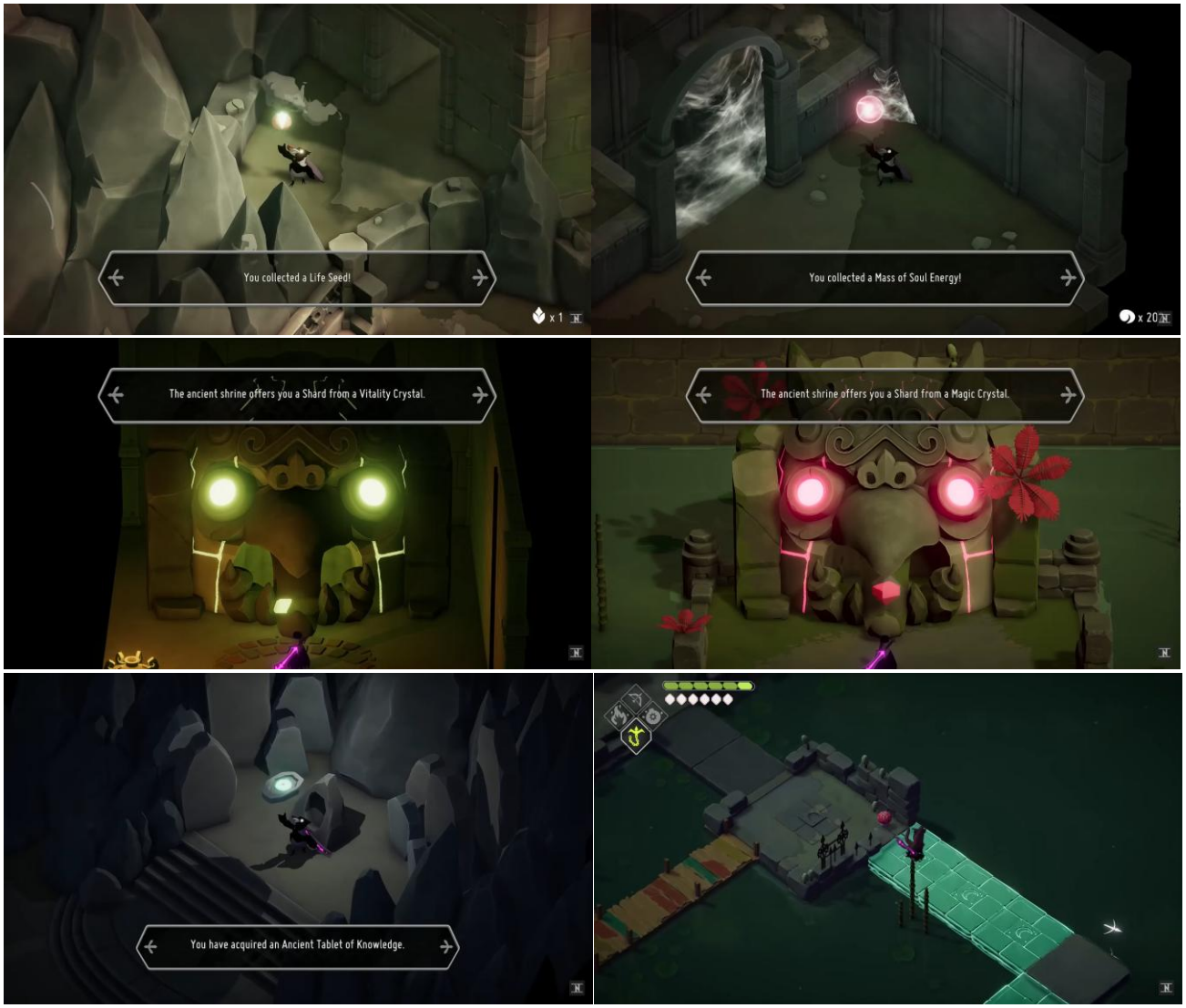


(Screenshots of the main map's doors and the Soul Vault menu, sourced from NeatPick YouTube)

Collectables:

Death's Door also has a lot of **secrets** that can be **accessed using abilities** since areas may be blocked off (requires Bomb ability) or on separate platforms (requires Hookshot ability) or require igniting braziers for instance (requires Fire ability). Around all the areas you can go to, there are collectibles called **Shiny Things, Soul Orbs, Life Seeds, Magic and Vitality Shards**. Soul Orbs usually give you 100 more souls that can be used at the Soul Vault. **Life Seeds are a method for the player to heal themselves completely**: the player can collect the seeds and then **plant them inside pots** that are scattered around the maps. However, the player can also heal themselves completely by returning to the RCH. **Magic Shards can be collected to increase the number of magic attacks the player can use**. **Vitality Shards can be collected to increase the amount of lives the player has**. Crystal Shards only improve the player's stats if they have gained 4 of them. The Shards can be found within Shrines that were left by the Camp of Free Crows to aid the player. After the Lord of Doors is defeated, you can also acquire 7 Ancient Tablets of Knowledge to get the True Ending of the game. You also gain a key after defeating the Lord of Doors which gives you access to a bell tower that allows you to change the time to night since there are **secret areas that are only**

unlocked at night. Death's Door also uses keys and Lost Souls that you must find to get past doors and gates.



(Screenshots of collectables and secret night path, sourced from NeatPick YouTube)

Wave battle:

There are specific doors that give you abilities and to open them, you must find the four Lost Souls for the door. The door connects back to the RCH and there is a chest that triggers **four waves of enemies that you must defeat to gain the ability**. If the player dies during this challenge, a message will appear to allow the player to retry the challenge.

(Screenshots of the ability challenge and completion, sourced from NeatPick YouTube)

Player help:

To help the player find all the secrets within the game, when the player reaches the Stranded Sailor there is a shop where a **character called Jefferson offers you a bowl of soup and information** about the whereabouts of the secrets while you eat the soup. When the player also unlocks the secret area within the Estate of the Urn Witch by using the back of the Old Photograph, you can find a Rusty Garden Trowel which you show to Pothead (the character placing the pots everywhere) and he will go to the secret Estate area and **will tell you where the pots that don't have any Life Seeds planted in them are**.

(Screenshots of Jefferson's and Pothead's dialogue, sourced from NeatPick YouTube)

Maps:

There are multiple areas within Death's Door that have **themes based on the bosses** (the Giant Souls) within the area. The Witch of Urns has a mansion estate filled with urns and enemies and a basement that's a furnace. The King of the Swamp is a frog that has seized a jungle-like forest from the Forest Mother and Forest Children and has a dungeon filled with mushrooms as well as a lair called the Flooded Fortress. Betty is a yeti that lives above Castle Lockstone in the mountains near the Old Watchtowers. The Demonic Forest Spirit lives within a forest. Within the Flooded Fortress, there are **statues that you can hit that reveal paths** and within the Overgrown Ruins there are **vines that the player can hit which takes the player up to the area above the vine**. There are also **sewers which the player can access to get to secret locations**. The main map area (RCH) is based like a normal office and street area, but the areas are floating and joined together by stairways.

(Screenshots of the different areas, sourced from NeatPick YouTube)

Enemies:

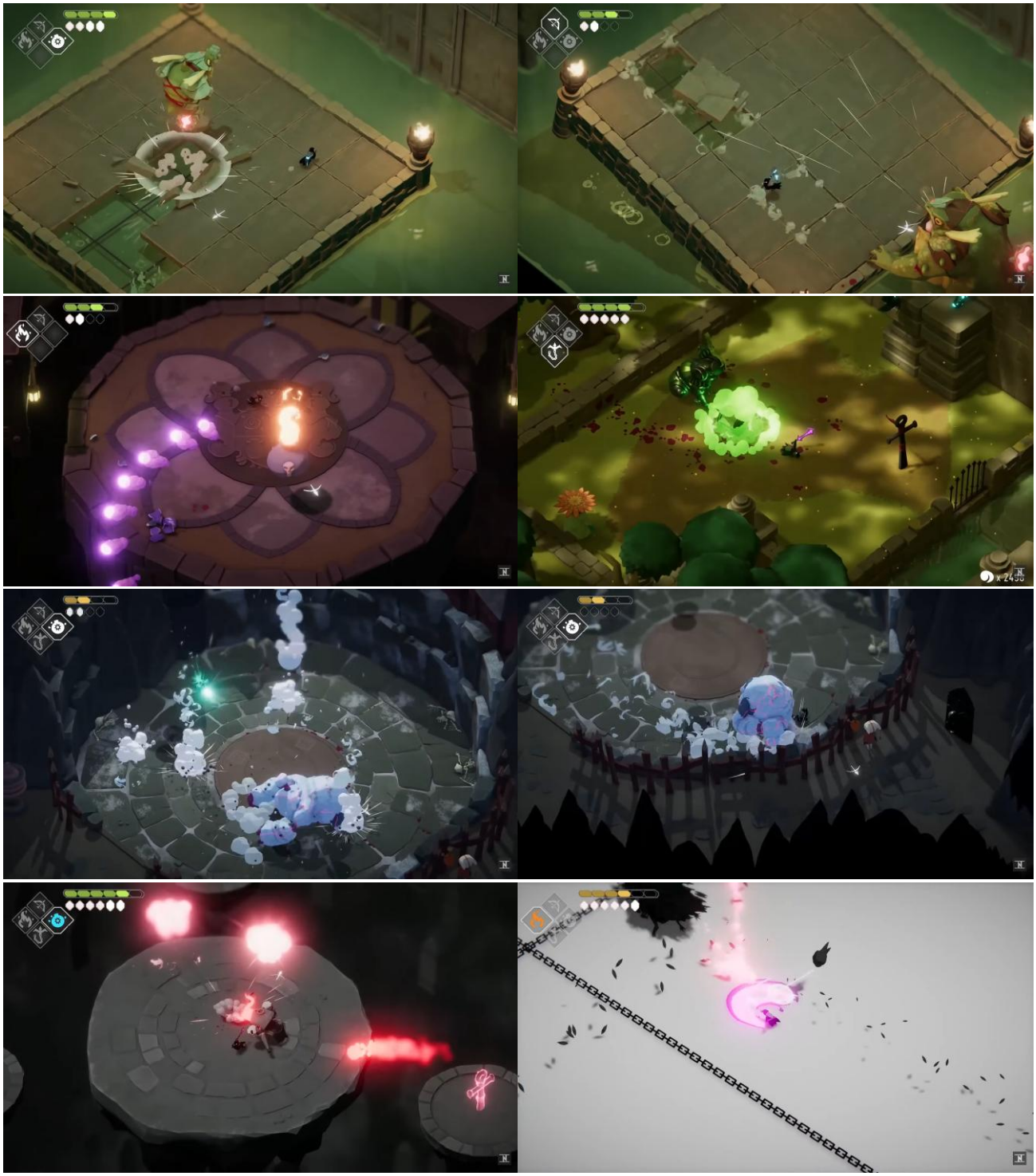
Death's Door's **smaller enemies** tend to be either **long-range or medium-range and short-range**. There is one enemy that's in most of the areas that **runs after the player** and **slashes them** as well as **jumping onto the player**. Another enemy **rolls into the player to attack** and has a hard shell for a back. There are two enemies that **shoot arrows**, one is slower and has lower health than the other one. There are also **static enemies** like the carnivorous plants that either **shoot out fire balls** or can **only attack the player when the player stands near** it. There are different types of the enemies that have slightly different abilities such as the **mage-like enemy that shoots orbs** at the player, the green mage enemy that **shoots gas bombs** and the red mage enemy that **shoots out a line of fire**. One enemy holds a **boomerang** and can attack the player using the boomerang or by jumping onto the player. There's an enemy that wears **armour** and attacks the player using **small bombs and slashes from near** and it has two versions: a silver and gold version.

(Screenshots of different enemies, sourced from NeatPick YouTube)

Boss battles:

There are multiple boss battles including mini boss battles, but the mini boss battles are usually similar enemies. The main bosses use both **short-range and long-range attacks** such as **lasers**, a **swatting motion** around them, **orbs that are shot** out, **falling snowballs**, and more. The King of the Swamp (Frog King) battle is slightly different because the Frog King **destroys the platforms that the player stands on** and the **player can shoot the Frog King's staff to recover the missing platforms**. He also attacks the player by **grabbing the entire platform and sucking the air to eat the player**

while some of the platforms fall off. Both the Frog King and the Urn Witch battles are **weak to the abilities that the player gains for the fight**: when the Urn Witch shoots out orbs from within the urn, the player can shoot fire to **stun** her and when the Frog King tries to eat the player, the player can shoot a bomb into his mouth to stun him. Within the battle against Betty, she **rolls towards the player** and then eventually sends down snowballs that randomly fall on the platform as well as **throwing snowballs** at the player. The **mini bosses for certain areas have the same attacks** where they charge at the player and strike down, but one mini boss **emits green gas that harms the player when they strike down**. The **mini bosses for ability upgrades are based on the ability** and have different attacks but the **same base attacks**: charging the player and then striking down, a large slash around themselves, jumping onto the player and a **magic attack on the player that slows the player and requires the player to spam any button to free themselves**. Some bosses also use **chains** to attack the player like the player's Hookshot ability such as the Grey Crow.



(Screenshots of mini boss and boss battles, sourced from NeatPick YouTube)

The battle against the Lord of Doors takes place after you pass through a giant door within the RCH. The battle begins where the Lord of Doors resides (in a room overlooking all the Giant Souls' realms and Death's Door) and then the Lord of Doors sets **obstacles for the player to pass**, such as **bulls with door heads that charge into the player**, and then **moves to a location within each of the Giant Souls' areas** and repeats this until they've fought in all the locations. Then a checkpoint is set, and the main fight takes place within the middle of the RCH. The Lord of Doors **attacks the player using basic attacks** such as jumping onto the player, swatting the player away and rolling into the player as well as more complex attacks like **jumping onto the player** and **shooting out lasers simultaneously**, **shooting red orbs** at the player from external doors, going to the top of the map and shooting a laser at the player while door-head bulls run across the area.

(Screenshots of the battle against the Lord of Doors, sourced from NeatPick YouTube)

Trophies:

Death's Door is another PlayStation game, so it also has **PlayStation trophies** for completing tasks such as **finding all Shiny Things**, **completing the game only using the Umbrella as a melee weapon**, **planting all Life Seeds** in all the pots, and more.

This existing solution doesn't fix my initial problem because Death's Door has more of an open-world and story driven progression which means it also **requires time to play and progress** – making it harder for the player to quit the game at points. This is especially the case since some **checkpoints within the game were also quite far away from each other**, making it take time to get back to where you were if you stopped playing between checkpoints. Death's Door also has lots of secret areas within the game that require the player to return to areas and it **takes time to find these secrets**. In addition, the boss fights take quite long. These factors aren't ideal for players that simply wish to play a quick game without too much hassle and is again not a suitable solution for my problem.

For my solution I will implement some of the battling aspects such as how the player can fight as well as the way enemies fight the player to create a **range of different enemies**. I will also take inspiration from the **puzzles** within Death's Door as well as the **boss battles** to make the game more challenging for the player, although the boss battles are slightly complex so I should take some simpler aspects to create the game in the time given. The **wave fights also give me inspiration for how I might create a survival mode for my game**. I might also take inspiration from the game's **story** because it's easy to understand and not too lengthy as well as the way characters can speak to each other through speech bubbles. However, **I won't adapt the world style** where there aren't established levels because I want my game levels to be quick to play and that also means that **returning to the main map without completing the level will require the player to start the level from the beginning again** like how it is in Sackboy: A Big Adventure. I will take into consideration the **design of the areas** of Death's Door because they are simpler than Sackboy: A Big Adventure's designs but still aesthetically pleasing and challenging.

Features

GUI/Graphics

The game will be a single-player top-down 2D platformer game and the Graphical User Interface I'll be using to create it is Pygame's GUI. This is because it's easy to use and understand which will reduce the time taken to create the game since I already understand Python and Pygame. To focus more on the coding, I will also be using a free-to-use asset pack called Ninja Adventure[5] by pixel-boy which will affect the graphics and aesthetics of the game. The player will be able to see a Heads-Up Display (HUD) that will show the user important information about their character such as their health, number of collectibles, and more. This means that the player can understand what is going on and will benefit their gameplay which is an aspect that all the stakeholders said was very important.

Audio

The game's audio will also be sourced from the free-to-use asset pack. There will be audio tracks in each level to make the game more interesting as well as audio played when the player makes certain choices like attacking, attacking an enemy, gaining a collectible, killing an enemy, walking, blocking, opening the menu, and more.

Levels/Story mode

This will have 2 normal levels that will take around 5 minutes to complete and a mini boss level which may take around 7 minutes to complete – this is because although there was no length preference from the participants in the questionnaire, the main client doesn't have much time to play video games so it would be beneficial to have slightly shorter levels. Each of the normal levels will be designed for the player to make their way through enemies and obstacles towards the end of the level. To design the levels according to the stakeholder's preferences, I will use inspiration from Sackboy: A Big Adventure, Death's Door, Roblox and Minecraft. As the player progresses through the levels, there will also be a story for the player to follow alike Death's Door. This may also require having NPCs (Non-Playable Characters) to talk to the player.

Arcade mode

This is a special level that continues until the player dies from an enemy. Enemies will come to attack the player in waves and each wave gets increasingly difficult through three different ways: spawning different types of enemies, increasing the number of enemies, the platform the player is on changing. Each wave ends when the player kills all the enemies that are meant to have spawned during that wave and then the next wave begins after 10 seconds of rest for the player. This mode will be like Death's Door's ability challenge however the waves may be infinite or to a larger number than the four waves in Death's Door.

Main map

For the game to play more continuously rather than being disconnected, there will be a main map where the player can access all the game modes, and their trophies. This would also allow the game to be more story based since there can be NPCs in the main map that the player may be able to interact with. When the player is on the main map, their health doesn't matter because they can't lose lives.

Menu

The game will have a menu where the player can go back to the main map, change the audio settings, see how many collectibles they have, close the game and check the controls. Going to the menu screen will also pause the game for the player.

Collectibles

There will be a collectible that the player can earn from any of the game modes for either completing a level or passing a specific number of waves on the arcade mode. These collectibles can also be used later for a leaderboard between players that ranks the players based on how many collectibles they have.

Trophy tracker

The game will have certain trophies and allow the user to view them like the PlayStation trophies used for PlayStation games. The trophies will be obtained by completing challenges such as completing a level under a specific amount of time. This would then require the user's time on each

level to be tracked. Other trophies can be gained by defeating a boss, gaining a specific number of collectibles, and by other means. The player will be able to view the trophies collected and see how many trophies they haven't collected.

Enemies

The game will have 3 main enemies that will attack the player and will be able to be killed by the player. Each enemy will have their own method of attack to create a variety of enemies: one will attack when they are close to the player, another will attack from long distances, the other will attack from a short and medium range (like Death's Door's enemies). This means that the player would have to find different ways to attack each enemy which may make the game more challenging but fun and varied which is what all the stakeholders wanted. Within the levels, the enemies will be placed in specific areas to make it more difficult for the player to get to the end like in Sackboy: A Big Adventure and Death's Door. The enemies may also have different amounts of health which means that the player may have to hit the enemy more times if it has a larger amount of health.

Player

The player will have normal WASD style movement (they can move up, down, left, right) but will also be able to attack, jump and block. While the player is jumping, the player will be able to move mid-air. The player will also have a certain amount of health that's indicated by five hearts which means they can only get attacked five times until it's game over.

Mini Boss

The mini boss level will require for the player to defeat the boss and kill enemies while not dying. The mini boss will summon enemies to the platform and there will be instances where the boss can be attacked – like the final battle against Vex in Sackboy: A Big Adventure. I need to ensure that the mini boss isn't too difficult to defeat as the stakeholders have mentioned so having a mini boss battle like this would make it easier to alter the difficulty by changing the number of enemies summoned. This mini boss would also be easier for me to create within the time frame.

Limitations

In development:

Time

Time will limit how developed the game will be because I might not be able to develop every aspect of the game as initially planned. However, I will ensure that the main features of the game such as the levels and arcade mode are developed so that my client's problem can be solved, and all my stakeholders are happy.

Pygame/Python

Because I'm using Python and the Pygame library, this means that my game will have the same limitations that they have. One of the limitations of using a Pygame GUI is that you must create everything by scratch, but this limitation can be overcome by using assets that are pre-made which saves development time.

Asset Pack

Using an asset pack will speed up the development of the game however, it also means that I'm limited to what is included within the asset pack unless I decide to create my own assets. In addition, the style and aesthetics of the game will be determined by the asset pack which limits the creativity of the game slightly.

In use:

Storage + Transfer Speed

The game's assets will be sourced from a free to use asset pack which means that the size of assets will be varied. If the game assets are too big, they may need to be compressed as a ZIP file which means that would require me to use another Python module called zipfile[6] or use gzip[7] or bz2[8]. However, this may be tedious to implement since the assets would have to be compressed and then decompressed every time I need to use the asset which may affect transfer speed and slow down the game slightly. Although, compressing the files would make the download speed for the game faster and it would allow the user to save storage space on their computer. Furthermore, if the transfer speed of the assets is too slow then it may create glitches within the game which may make players and stakeholders unhappy.

Requirements

Developer:

The game will be coded with Python and Pygame which means that I will need to have an adequate knowledge and understanding of Python and the Pygame library. I will also need a good understanding of OOP because the game will use OOP like many games that exist since it makes it easier to create games.

User:

The game will be created on Python using Pygame therefore the Pygame library requires downloading to run the game through Python.

There wouldn't be any specific hardware requirements other than having a usable computer (including a monitor, keyboard and audio output such as a speaker or headphones) that has the storage for the game's content to be downloaded so the game can access it. Although your computer needs to have around 8GB of memory and 8GB of RAM to play the game.

Success Criteria

Requirement	Justification
At least three playable levels	This is so there can be a variety in the level designs and the situations within the levels which makes the game more interesting and is what the stakeholders want.
One mini boss fight	This adds difficulty to the game as well as variety in gameplay which is what the stakeholders want.

Arcade mode	This allows players to try a different game mode and provides variety to the gameplay. Variety is what the stakeholders all wanted as they voted that they prefer different game modes, and the client wanted to play something new.
Main map	This makes the game flow more continuously rather than being more disconnected since it makes the game feel like it has its own world rather than the game modes being separate. This also allows for world-building story which makes the game more story-driven and is what the client prefers in games. The questionnaire participants also preferred having a story.
At least three working enemies	Having at least three enemies makes the gameplay more varied since they can attack the player in different ways, and it would require the player to react differently to each of the enemies. This would improve gameplay which is an aspect that the stakeholders found very important.
Player WASD movement and attack/block	This is to ensure that the player movement is working correctly so that the player can play the game without any worries. This also affects the gameplay a lot so it is a vital requirement since otherwise the player can't play the game properly.
Player health	This is to create difficulty for the player otherwise if the player had infinite health, there wouldn't be much of a challenge to complete levels. Most of the stakeholders expressed that the game should be challenging if it's a platformer game.
Collectibles	This is to make the game more fun for the player as they have incentive to play the game more to gain more collectibles. This also ties into having trophies in the game which is something that many stakeholders preferred.
NPCs and NPC dialogue	This adds a story to the game through dialogue which is something that all stakeholders preferred to have.
Trophy tracking	Having trophies allows players to challenge themselves to obtain trophies but it only challenges players that want to obtain the trophies. It also gives the players a sense of achievement which may make the game feel more fun and is supported by the questionnaire.
Working menu	This is to allow the player to pause the game, understand the controls, change settings as well as leave what they are currently playing to go to the main map. A working menu is usually in every game and is something that players would most likely expect.
Save mechanism	This is so players can keep track of their progress of the game and their trophies. This means that the player can come back to the game at any time if they save before they quit which is in line with the original audience who aren't able to play games enough.
Title screen	The screen would show the name of the game and tell the player to click a button to start the game. This is so the

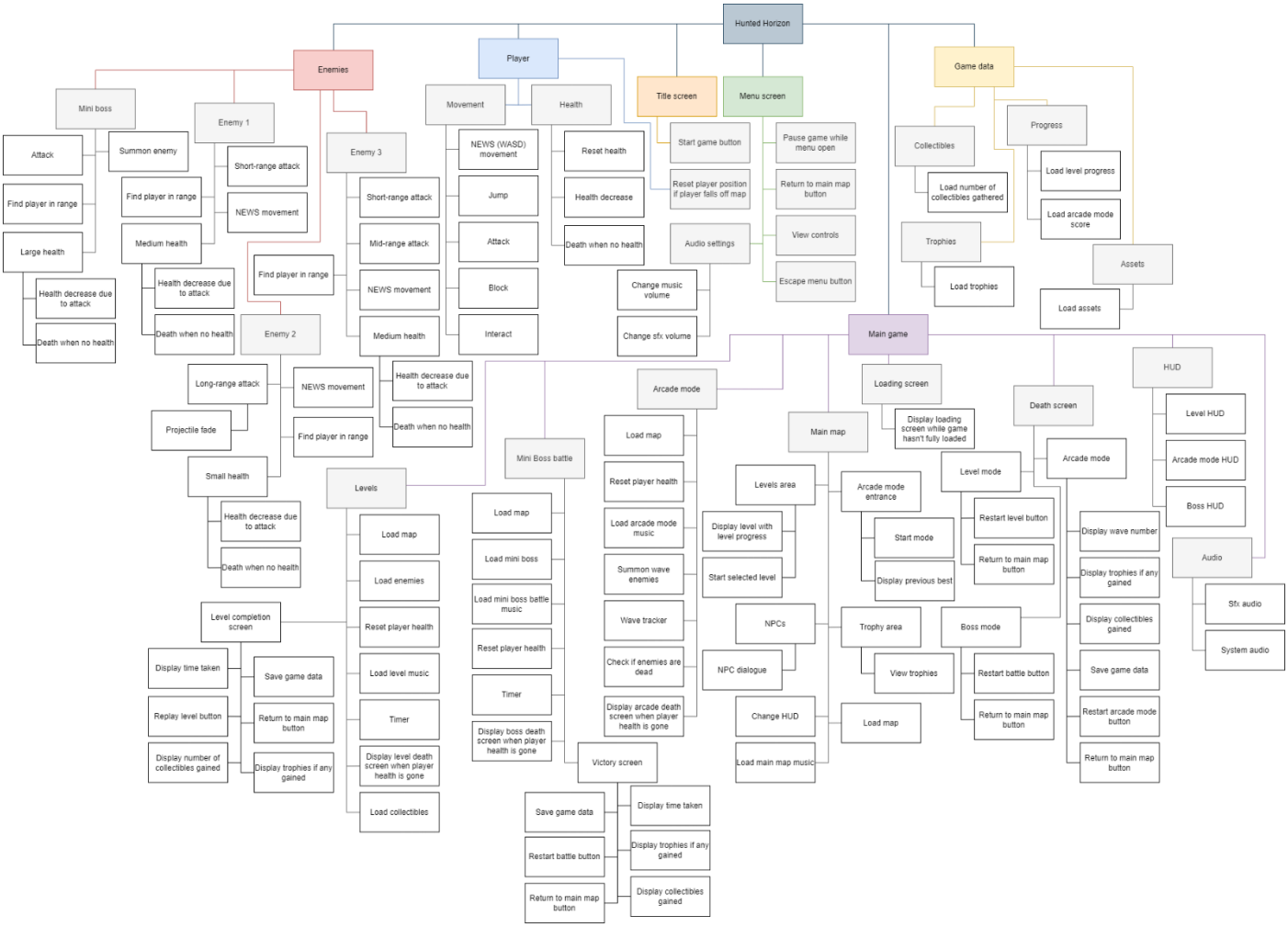
	player can have a small menu before they play the game to improve the gameplay by allowing the player to settle into the game easier and because many games have this feature.
Death screen	This is so that the player knows they have used all their lives and will exit them from what they were doing (a level or arcade mode) to the main map. The stakeholders wished for a challenging game so this will also add challenge and stop the player from continuing where they left off.
Loading screens	This helps the player understand that the game is loading and that they've properly interacted with whatever they wanted to interact with (e.g. starting a level, starting the arcade mode). Therefore, this avoids the player from clicking too much and provides a clearer understanding of what is going on in the game, also improving the gameplay since players are less confused.

Design

After a consultation with some of my stakeholders, the game will be named ‘Hunted Horizon’ and will be referred to as such for the rest of the documentation.

Systems Diagram

This is a top-down approach of a system diagram that helps me to decompose the problem into modules and discard of any unnecessary information of the creation of the game. Creating this helps me understand what I need to do and how I will create my game by breaking each of the features down.



Enemies

Enemy 1

This is a top-down analysis of the short-range attacking enemy with a medium health (meaning the player can kill the enemy with four hits). The NEWS movement refers to the fact that the enemy can move upwards, right, left and downwards. When the player hits the enemy four times, the enemy dies and the sprite is deleted to reduce the data being processed. Otherwise, the game might crash if multiple sprites are still running in the background even though they are dead. The enemy also must be able to find the player in a specific range so it can direct its attacks to the player. The range is there to stop the enemy from attacking the player from anywhere within the map but only when the player is near the enemy.

Enemy 2

This is a top-down analysis of the long-range attacking enemy with a small health (meaning the player can kill the enemy with two hits). Everything is the same as the first enemy except the health and attack. This enemy will attack with a projectile that will be aimed at the player however the projectile has to have a limit to how far it can reach. Once it reaches that limit, the projectile should fade which is what the 'Projectile fade' is referring to.

Enemy 3

This is a top-down analysis of the short and medium range attacking enemy with a medium health. Everything is the same as Enemy 1 except this enemy has an extra mid-range attack.

Mini Boss

This is a top-down analysis of the Mini Boss that will have the largest health out of all the enemies. It has a similar analysis to the rest of the enemies, but it can summon enemies and has a general attack that affects the entire platform and must be dodged by the player by jumping.

Player

Movement

This analysis shows the different movements the player can do: walking up, down, left, right, jumping, blocking, attacking, and interacting.

Health

This shows the different parts of the player's health that must be considered such as resetting the health, decreasing the health and killing the player once its health has been depleted.

Reset player position if player falls off map

This is so the player can continue playing after losing a life and so they don't eternally die in the same area until they lose all their lives.

Title Screen

Start game button

This is the only thing that will appear on the title screen along with the name of the game and a background.

Menu Screen

Pause game while menu open

This is to allow the player to pause the game while they're playing it.

Return to main map button

This allows the player to stop what they are doing and just return to the main map.

View controls

This will display an image showing the user how to move their character and play the game using the keyboard controls.

Escape menu button

This is so the player can escape the menu and go back to the game. This will most likely be using the same keyboard button the player used to open the menu.

Audio settings

This just allows the player to alter the audio if it's too loud for them.

Game data

Collectibles

This shows how the collectibles earned by the player will be stored on their device if they have saved the game.

Trophies

This shows how the trophies earned by the player will be stored on their device if they have saved the game.

Progress

This shows how the progress made by the player will be stored on their device if they have saved the game.

Assets

This shows how all the game assets will be stored on the player's device. The assets consist of the game's maps, FX (special effects), sprites, audio, fonts, items and more.

Main Game

Levels

This breaks down the different aspects that each level will have and that will need to be coded such as extra screens, the map, a timer and collectibles.

Mini Boss battle

This breaks down the different aspects that the mini boss battle requires and is similar to the breakdown of the levels but has a different map, has the mini boss and has a victory screen rather than a completion screen.

Arcade mode

This shows the different aspects of the arcade mode that I need to code. It has the same main aspects such as loading the map and music, but it also requires coding the enemies to come in waves, keeping track of the wave number and checking if the enemies within the current wave are dead so the player can progress to the next wave. The arcade mode also only ends once the player dies so there isn't a need for an extra end screen, but the death screen is specific to the arcade mode and acts as the arcade game over screen.

Main map

This breaks down the different features and areas of the main map such as the levels area, arcade mode area, trophy area and features like NPCs. This is so the player can access all the different game modes and features through this main map.

Loading screen

This just shows how I will have to create a loading screen that loops while the part of the game the player is trying to access is loading.

Death screen

This breaks down how I need to have separate death screens based on where the player dies since the arcade death screen also acts as a game over screen however the boss and level death screen are the same.

HUD

This breaks down the different HUDs the player will need to have the correct information for the mode of the game that they're playing. This is because the player needs to see what wave number they're on while they play the arcade mode in addition to their health which would be shown in the level HUD. The boss HUD will be alike the level HUD but with information on the boss included.

Audio

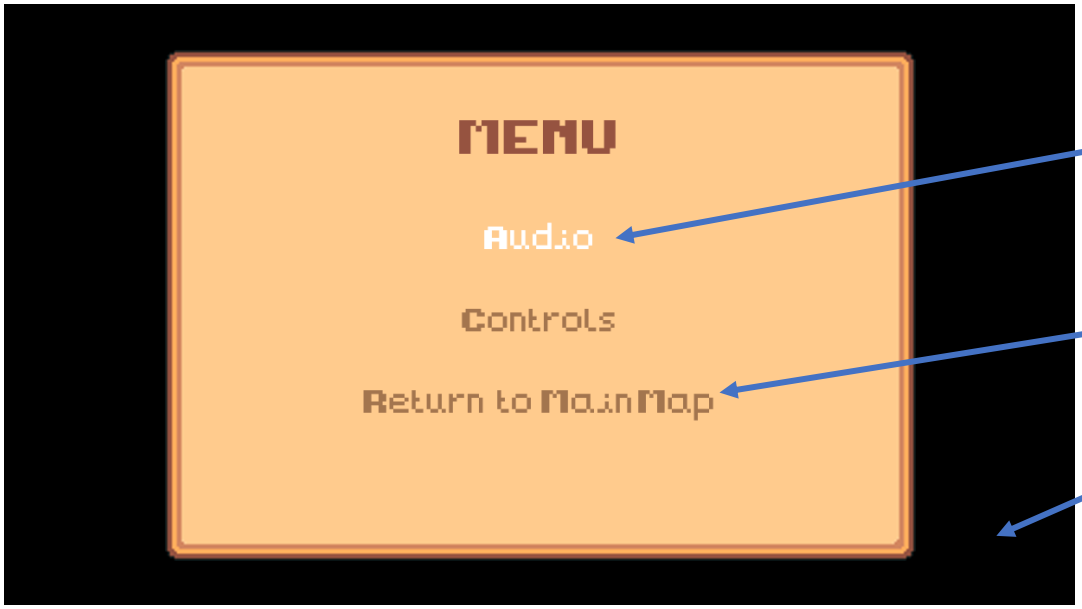
This shows the audio needed other than the music like the SFX (special effects) audio and system audio. The SFX audio for attacks and the system audio will be used when the player interacts with items or the menu.

Usability Features

To show the usability features of the game, I have created how the features may look within **PowerPoint**. This is because I am using an asset pack, and it would help me better understand how I want the usability features to look like. The actual game may have the same style or will look different depending on how it will be coded.

Menu

One of the main usability features of Hunted Horizon is the menu where you'll be able to view the controls and change audio settings while the rest of the game is paused. This is what the menu may look like in the game:



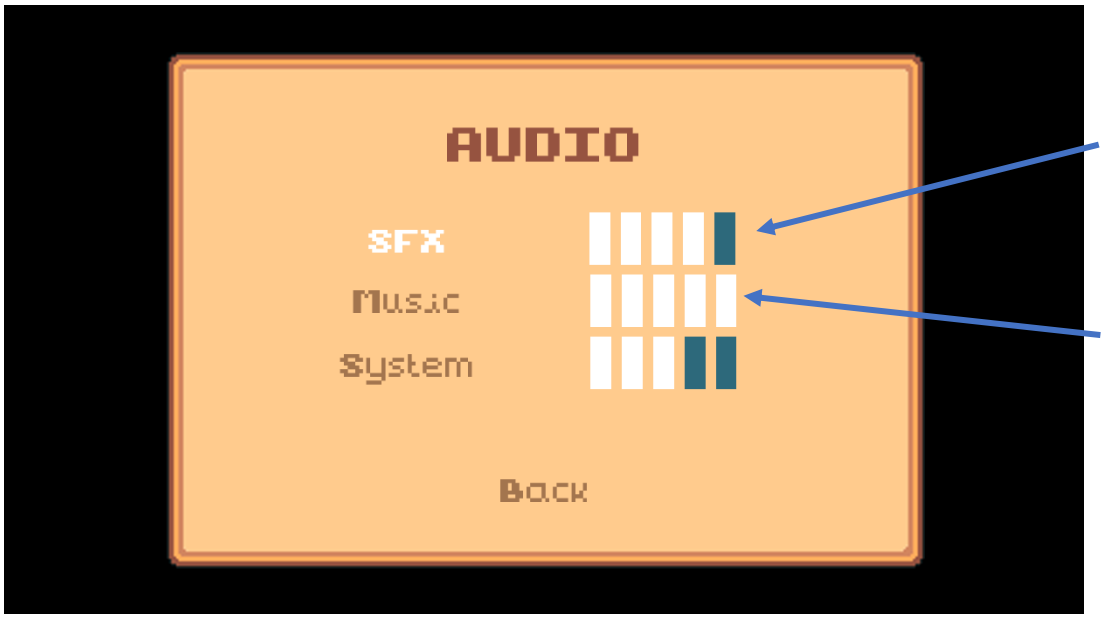
Clear indication of what is being selected

Clear text that is large enough for player to read

Player will be able to see the background and know that the game is paused

The player will be able to access the menu by pressing 'M' on their keyboard. The menu will appear above the last frame of the game as it is paused but this just shows a black background instead. The player will be able to use their WASD controls to navigate the menu (W=up, S=down, A=left, D=right) and what the player is selected on will be a white colour as shown by the image. The player can exit the menu by pressing 'M' on their keyboard again. The player can select the option by using 'L' on their keyboard which will take them to the other part of the menu.

The audio settings from the menu will look something like this:

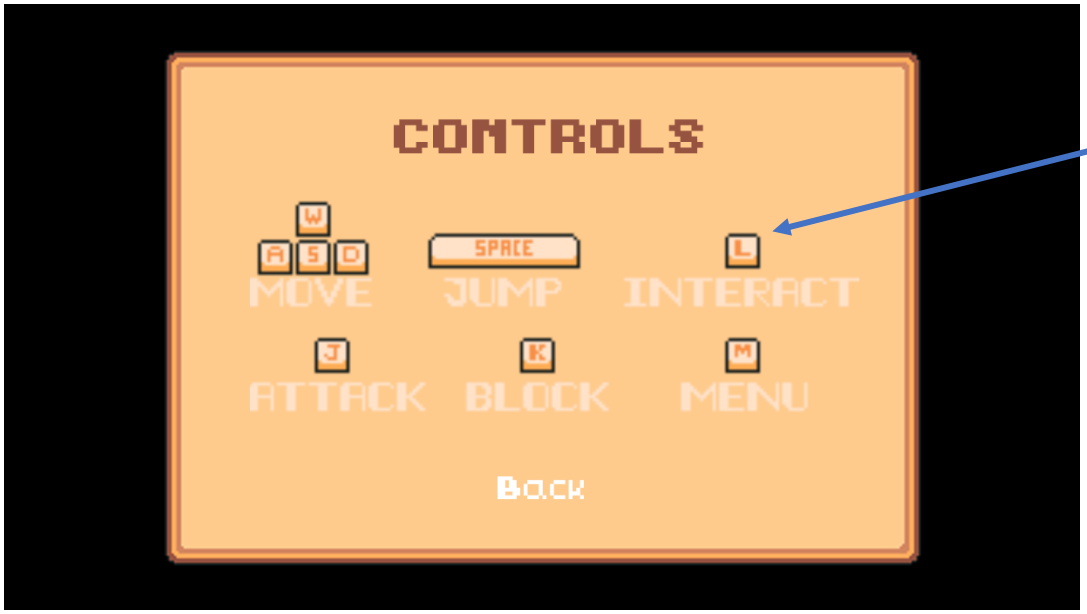


Universally used sound bars to indicate the sound level to the user easily

More white bars indicate louder sound and player can understand this by changing the audio and seeing what happens

The player can once again use their WASD controls and interact button ('L') to navigate the settings. The player can select the audio type and then use the keys 'A' and 'D' to increase or decrease the audio. This would then change the audio settings within the code, so they are saved. The player can also select back which will take them back to the main part of the menu. The reason for allowing the player to change the audio may be because the user may find the game too loud or certain audios of the game too loud therefore allowing the player to change the audio settings improves the user experience.

Viewing the controls will look like this:



Symbols can clearly communicate to the user that these are keyboard keys and what the keys can be used for

This just shows a diagram of the controls so the player can familiarise themselves with them. The player can only select the option 'Back' to go back to the main part of the menu. This is important to have otherwise the player can't play the game properly and will have to guess the controls which worsens the user experience. The controls have also been placed in a way that will be comfortable for the player on any keyboard since they can use one hand for moving and jumping and the other hand for attacking, blocking and interacting. WASD controls are commonly used in many games, so I kept the controls for movement something that is already familiar since then the player can understand the controls faster. Space for jumping and 'M' for the menu are also both commonly used in games.

Completion screens

When the player completes a level, there will be a level completion screen that will appear so that the player knows that they've completed it. The screen may look something like this:

Clearly indicates if player has achieved anything using a bright font telling the player what they achieved and a trophy symbol

Shows number of collectibles in a different colour to indicate that they are valuable

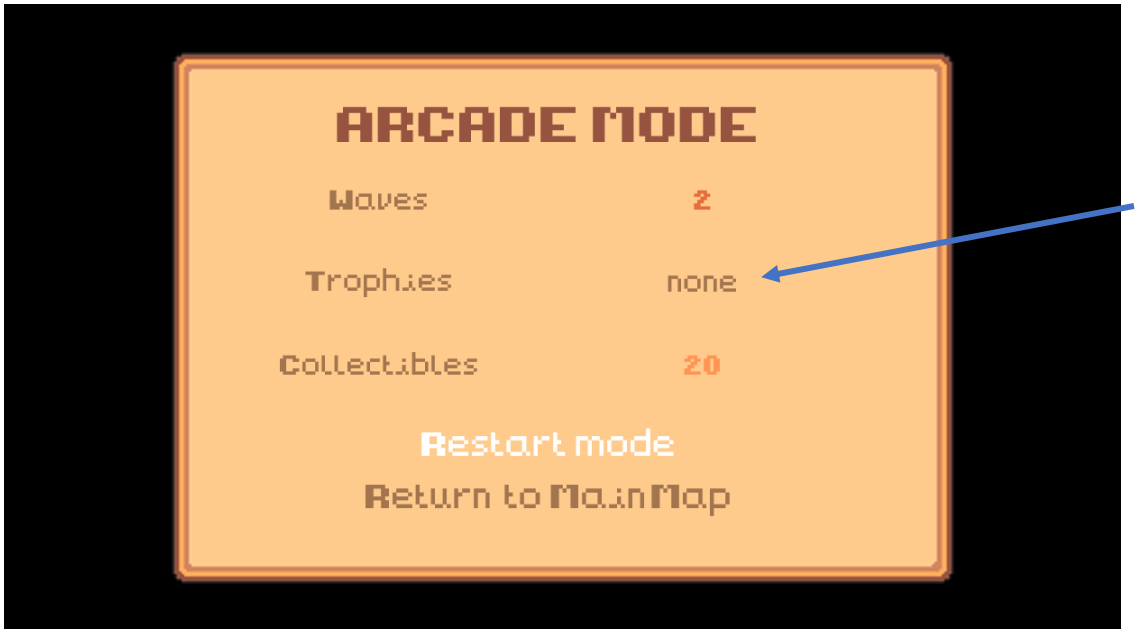
This will again appear over the last frame of the level. This informs the player of how long they took to complete the level, how many collectibles they earned in the level and if they gained any trophies. The player can then replay the level or return to the main map. This information may

influence the player's decision to replay the level – especially if they were trying to get a trophy and weren't able to. If the player doesn't achieve any trophies, it will just say none.

When the player dies in the arcade mode, rather than a death screen, a completion screen will appear to indicate the end of the arcade mode. This screen will look like the level completion screen but with different information and selection options:

Waves number is also in another colour to indicate its importance

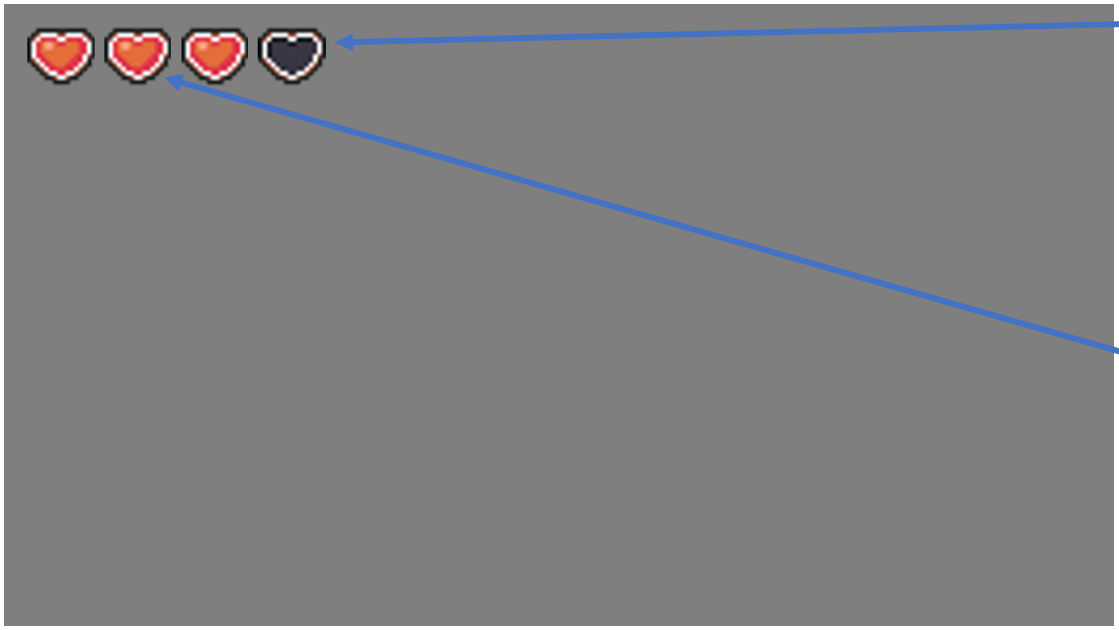
This completion screen shows how many waves the player had completed in addition to the trophies gained and collectibles gained from completing waves. The player can restart the arcade mode or return to the main map from this screen. If the player doesn't gain any trophies, the screen will appear like this:



'none' is in the same colour as the rest of the text which makes the red text and trophy symbol appearing when the player does gain a trophy more eye-catching

HUD

The HUD is important to the player since it quickly gives the player information about their game and character while they play. The HUD when playing levels may look like this:



Empty heart indicates a lost life which is used in many games and is easy to understand

Full hearts indicate lives that the player still has

The grey background isn't a part of the HUD but indicates whereabouts the HUD information will be displayed while the player is playing the game. The player's health is indicated clearly at the top of the game, so the player knows this information as they continue the game. This is because if the player loses all their lives, it will be game over and the player must either restart the level or go back to the main map. The HUD is the similar during boss battles but includes the health bar of the boss at the bottom of the game like this:

This is so that the player knows how much more they must damage the boss to defeat it which then ends the battle. This is alike many other video games so players will be able to understand that it's a health bar. If not, then the player will realise that it is the boss' health when they attack the boss.

This is what the HUD may look like while playing the arcade mode. It's similar to the level HUD but has information on what wave the player is on which may be helpful if the player is trying to complete a specific number of waves.

The main map doesn't have a HUD because the player doesn't need any immediate information while on the main map. This is since there are no enemies on the main map.

Trophy screen

The player will be able to view their trophies on the main map in a specific area. This is what it may look like:

The player can only select to go back and stop viewing the trophies. All the trophies are shown with their names although currently there's only placeholders for their names. The trophies that the player has gained are in colour but the ones they haven't gained are in greyscale. This clearly indicates this information to the player because all of the trophies will be greyscale in the beginning and will only appear in colour once they've gained a trophy. In addition, this style of viewing trophies is used in many other games so it's easy to communicate to the player.

Death screens

There are only 3 death screens, one being the arcade death screen which is also a completion style screen and the other two being for boss battles and levels. They may look like this:

They are both very similar and the only difference is the option to either restart the battle or restart the level. The player can also give up on what they were doing temporarily and return to the main map so they can come back later and play the level or battle again. The text has a lot of contrast which makes this screen very clear however, this screen design may be later altered.

Loading Screen

This is to show to the player that the game is loading and stop them from spamming inputs if they think the interaction button isn't working properly. It uses the background of the title screen, but this may change later in the development.

Title Screen

This is used to improve the user experience since the user isn't immediately thrown into the game. The player can press space which will then load the game in the main map. The design for this title screen may change later in the development also.

NPC and Object Interaction

This is used to indicate to the player that they can interact with the NPC or object. If the object can only be interacted with once, then the bubble above the object will disappear and the player will be able to see a small animation of the object being interacted with.

Client feedback on usability features:

The client was pleased with the menu and said it was consistent with his expectations. He particularly liked the addition of an audio menu since he said that he likes changing the audio settings if they're too loud or quiet and it gives choice to the user.

He also liked the controls section since if there's no tutorial for the game, the player can still figure out the controls. However, he said that **informing the user of how to open the menu on the HUD** was probably a good idea, so they know how to view the rest of the controls. He also had an additional suggestion of adding **custom binding** to the keys if I had time.

The client liked the completion screens but thought that the **level completion screen could include how many times the player died**. He also asked about **what it would look like if various trophies were earned** on one level since my mock-up versions only show one. He also said to **replace collectibles** with the **time taken and add how many enemies were defeated on the arcade mode** end screen because that is what users usually want to know.

He also suggested **being able to view the trophy info by selecting them** on the trophy screen.

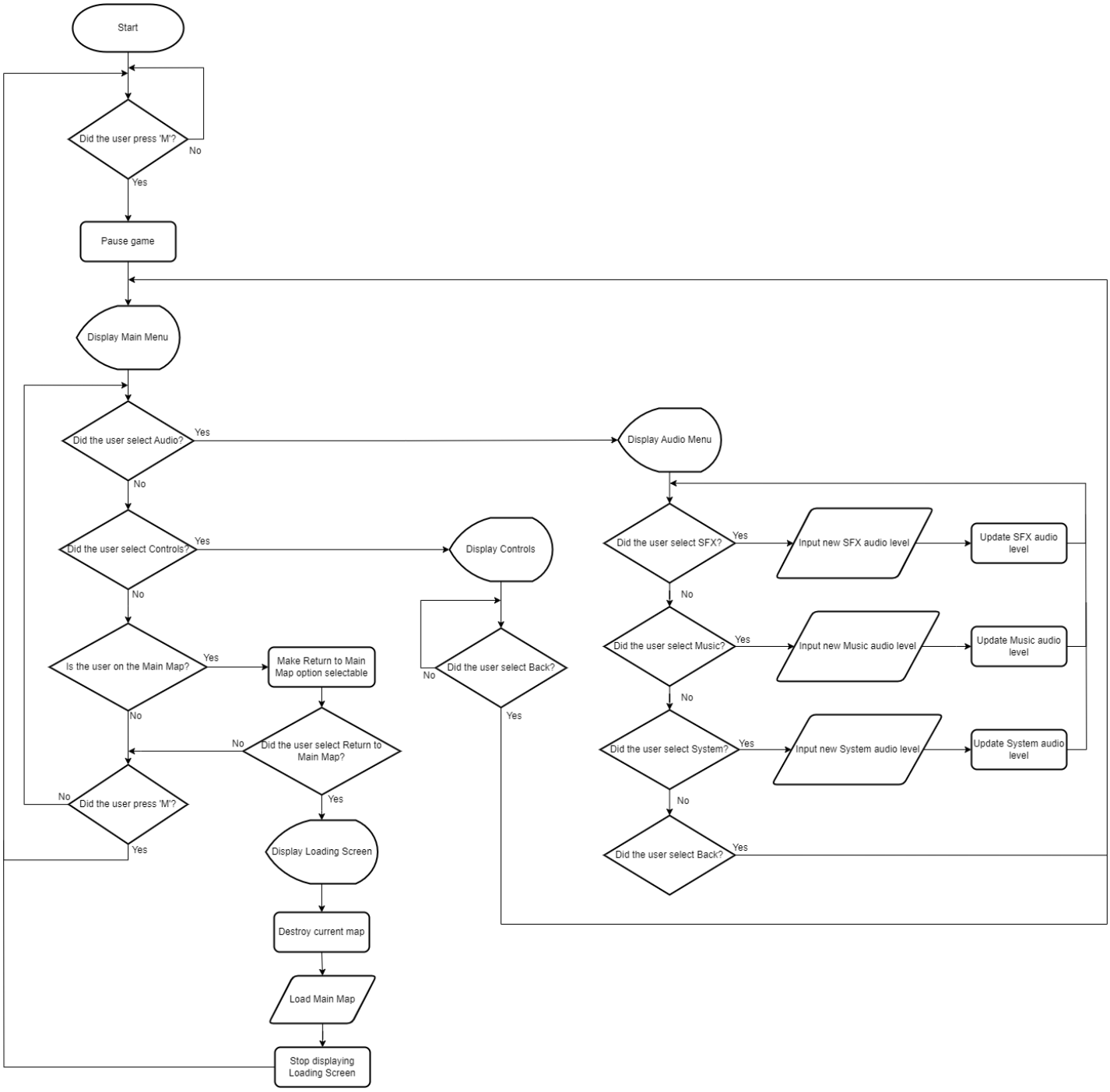
He liked that the colour palettes of the loading, death and title screens complemented each other. He also requested if there could be the **player character walking on the loading screen** – a feature that many games have. He also said that the **title screen lacked imagery and didn't indicate what the game was about**.

When I develop the game, I will keep these ideas in mind and check with my client again to see if it fits his criteria. Any additional features such as custom key binding may be features I add at the end of my development.

Algorithms

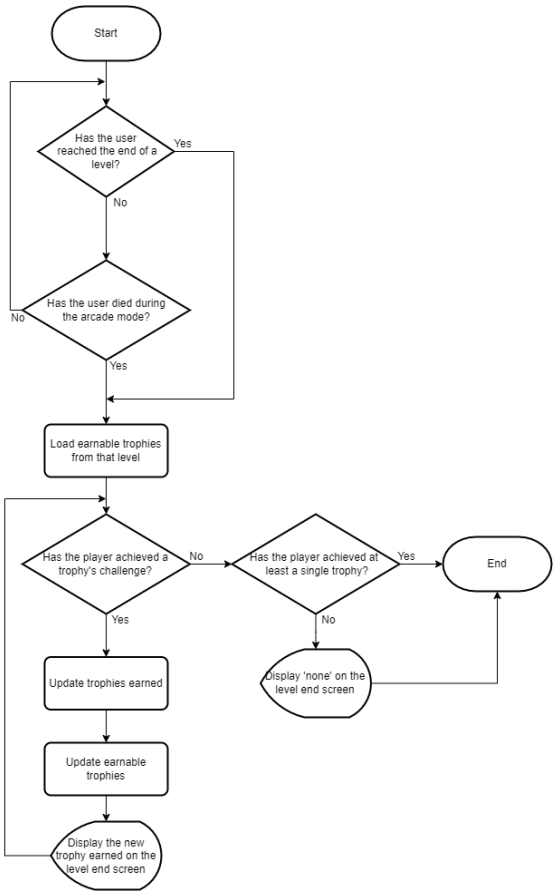
I have displayed flowchart diagrams for some of the more complicated algorithms within my game so I can effectively implement them using code since the flowchart would help me to plan and break down certain elements of the game. Although, many of the flowcharts may be simpler compared to the actual code because of the complexity of the code.

Menu

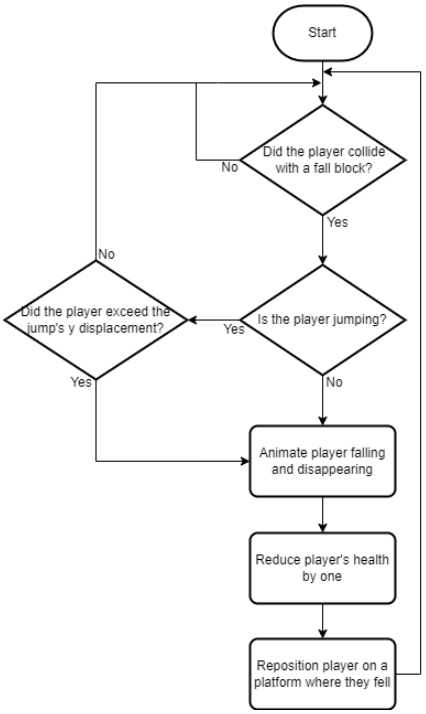


This flowchart doesn't have a terminator to end the process because it should continue until the user closes the game which means the user can access the menu at any time.

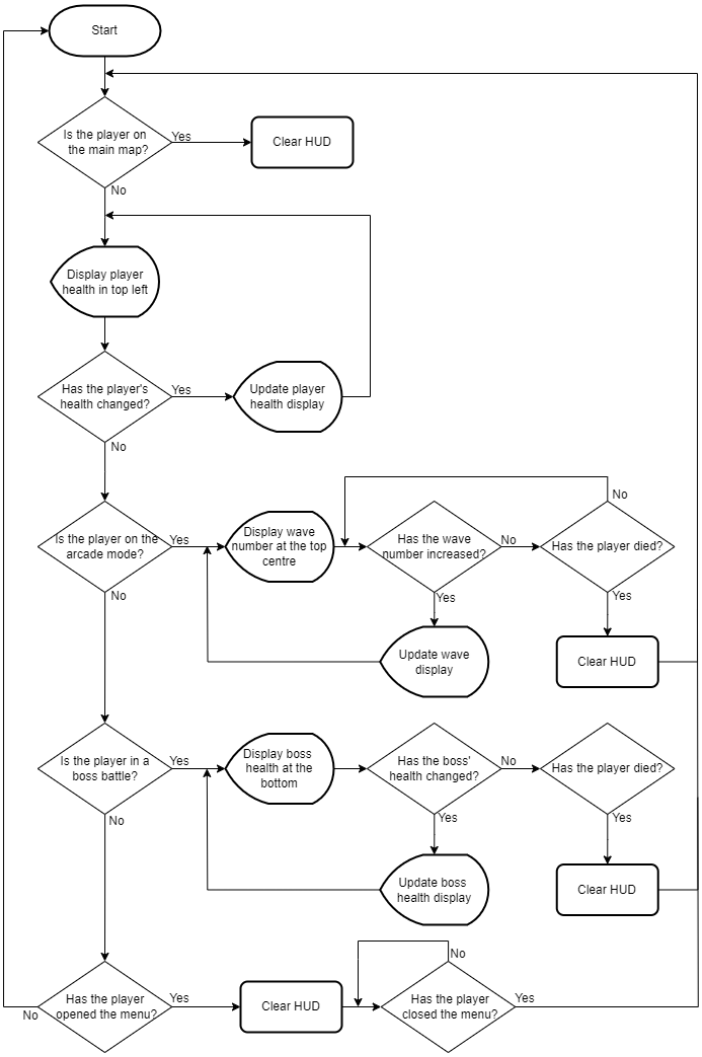
Checking if a trophy has been earned



Checking if the player has fell off the map

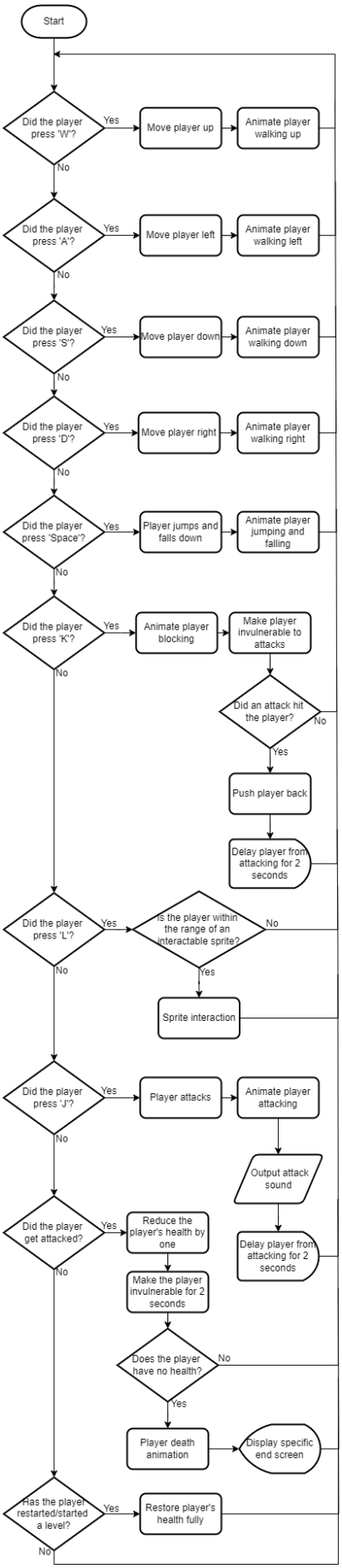


Updating HUD

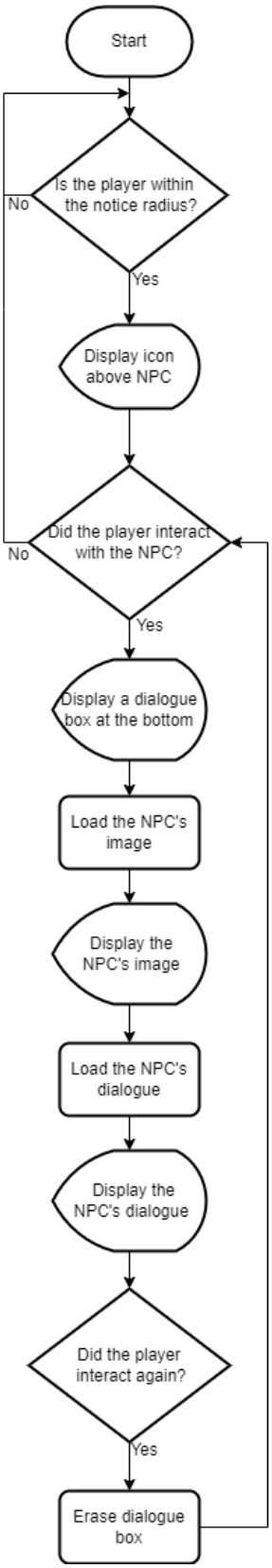


Updating enemies

Updating the player

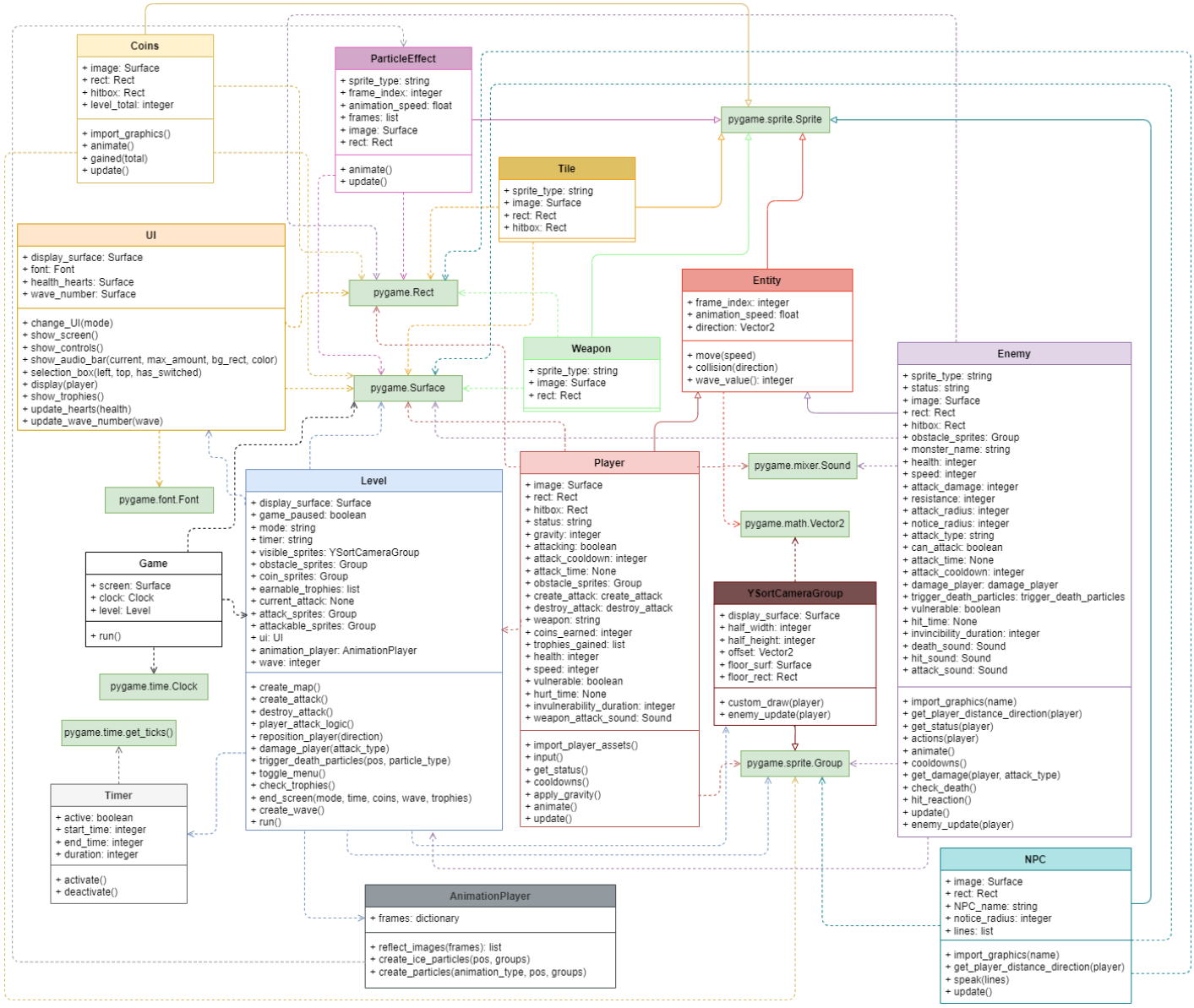


Updating NPCs



Key Variables and Classes

UML Class Diagram



This UML diagram breaks down some of the potential classes with their attributes and methods as well as the relationships between them. To create the classes, I've used my knowledge of the structure of prior games that have used Pygame. Furthermore, the only two relations I have used in the UML diagram are to indicate inheritance and dependence so as not to over complicate the diagram. The small green boxes represent the use of Pygame's classes and their methods since many of the classes are dependent or inherit from those classes/methods. Many of the classes' attributes may use other methods and therefore the data type of them may be hard to define but since the attribute is another class' method, the data type is the object type of that method. This means that I've kept some of the attributes' methods as their object type where it would be hard to define but I have tried to put the correct data type for the attributes that would use methods when I know what data type the method will return. However, this is an estimate of the structure of the classes, and it may look different at the end of the game's creation.

Game

This class is the main class that would help run the game by updating the screen using the FPS (Frames Per Second) it's given and the clock, writing the name of the game on the top of the tab and allowing the player to exit the tab by pressing the X on the tab. Its clock attribute is dependent on pygame.time.Clock because clock is the variable name that stores the data from the pygame.time.Clock method. The attribute level is also dependent on a class, but it is a class that will be made by myself: Level. This is so that the level is updated in the main running of the game.

Level

This class would deal with the main aspects of all the levels (including arcade mode) such as the basic gameplay features such as attacking, dealing damage, loading the maps, allowing the player to access the menu, and display end screens. I have decided not to separate the levels into multiple classes for the main map area, arcade mode, level mode and boss mode since the main logic is very similar and there'll just be some extra methods and attributes that will or will not be used according to the level's mode. This stops the game's logic from being overcomplicated and speeds up the development since I won't have to write as much code. The Level class is dependent on two of Pygame's classes (pygame.Surface and pygame.sprite's Group method) but is also dependent on the YSortCameraGroup, AnimationPlayer and Timer classes. Pygame's Surface class would be used to create the maps since it allows the screen to be drawn on and pygame.sprite.Group would be used to have access to all game's sprite groups so they can be updated, drawn, added and removed. This would stop the game from crashing since sprites are properly removed. YSortCameraGroup is the camera of the player and would be used to update the visible_sprites attribute since only the sprites that are visible to the player will be drawn to increase efficiency of the game and reduce the memory it would take to run. The AnimationPlayer class would be used by Level to animate the player's movement, and the Timer class would be used to time the player during levels.

YSortCameraGroup

This class would deal with what the user will view while playing the game as they move their character: it would make sure that the user can see their player character in the middle of the screen always, even when they move. In addition, the class would help give depth to the level of the game as it would allow the player sprite to overlap with other sprites. Therefore, pygame.sprite.Group would be inherited since the class needs to have access to the sprite groups to be able to overlap them in the Y direction, meaning that the player looks like it's in front of or behind other sprites. It would be dependent on pygame.math.Vector2 for creating the camera that follows the player since it would help create an offset that draws the map so that it looks like the camera is following the player.

AnimationPlayer

This class would contain dictionaries of several animation frames for attacks and enemies and a method to reflect images, so the animations are facing the right way as well as a method to create particles, so they appear on the screen for the user using the ParticleEffect class. This is to reduce slowing down the game whenever particles are created, therefore making AnimationPlayer dependent on ParticleEffect.

ParticleEffect

This class would deal with animating the particles created in the game and destroys them when they reach a certain length. It would be dependent on pygame.sprite.Sprite to manipulate the particle as a sprite so it can be updated, created and killed.

Timer

This class would be made to time the player when they play levels since Pygame doesn't have a timer that can do that, meaning I would have to create it as a separate class. It would still require to be dependent on pygame.time.get_ticks to get a time value when the player starts and completes the level to find out the duration.

UI

This class would deal with the UI (User Interface) aspects of the game such as the HUD, menu and screens however it would only need to be dependent on Pygame's classes and methods like pygame.Rect, pygame.Surface and pygame.font.Font. Both pygame.Rect and pygame.Surface would be used for drawing on the screen so the player can see the menu and HUD and pygame.font.Font would be used to apply fonts to the any text within the UI.

Tile

This class would use the imported version of the game's maps from the application that I will use to create my maps (Tiled) to place each of the tiles within the game. It would be dependent on pygame.Rect, pygame.Surface, pygame.sprite.Sprite because it needs to draw and add all of the elements of the map such as the player, enemies, objects, tiles and more.

Coins

This class would deal with the collectibles in the game: Coins. This means that it would animate the coins on the map as well as establish a hitbox which the player can walk into to gain the coin. Therefore this class would be dependent on pygame.Rect and pygame.Surface to draw the coins as well as pygame.sprite.Sprite and pygame.sprite.Group since it might be beneficial to make the coins as another sprite that's grouped for ease of coding.

Weapon

The weapon within the game may need a class for placing itself correctly on the player's character while the player moves which can be done in the constructor method of the class since it wouldn't have any other methods. It would need to inherit pygame.sprite.Sprite so the weapon can be manipulated and pygame.Rect and pygame.Surface so the weapon can be drawn.

NPC

This class would allow the NPCs (Non-Playable Characters) to be shown on the main map and be interacted with if the player is near enough to the NPC. This class would inherit pygame.sprite.Sprite to be manipulated as a sprite and be dependent on pygame.Rect, pygame.Surface and pygame.sprite.Group since the NPCs would have to be drawn on the screen and they may need to be grouped as another type of group since they can talk.

Entity

This would be a parent class of both the Player class and Enemy class since it covers the basic methods and attributes that both will like collisions and basic movement. This would then reduce the amount of repeated code within the game as well as speeding up the development process.

Entity would be dependent on pygame.math.Vector2 so the direction of the entity (the object of Entity) can be known to alter the hitboxes and collision boxes accordingly. Entity would also inherit pygame.sprite.Sprite so any entity can be manipulated using it's methods and any use of this is inherited by the children classes.

Player

This class would deal with all the aspects of the player such as animating itself, updating itself, displaying itself, attacking, health, jumping, coins and more. Player would inherit Entity and be dependent on pygame.Surface, pygame.Rect, pygame.sprite.Group and pygame.mixer.Sound. The first two classes it would be dependent on are for drawing the player character so the user can see it and pygame.sprite.Group would be used to place the player sprite in its individual group. Furthermore, pygame.mixer.Sound would be used by the Player class so there are sounds when the user interacts with the game through the player character such as attack sounds.

Enemy

This class would deal with all the aspects of the enemies such as displaying itself, animating itself, updating itself, attacking, noticing the player and more. Enemy would also inherit Entity for the basic features of both the player and enemies and is dependent on pygame.Surface, pygame.Rect, pygame.sprite.Group and pygame.mixer.Sound for the same reasons that the Player class would be. I decided that, like the Level class, I may not create a separate class for the mini boss enemy since it would have similar code, and I could add the code to the Enemy class instead to reduce repetition.

Key Variables

game

This would be the variable name of an instantiation of the Game class which means that the object game would allow the game to run for the user.

FPS

This variable would be an integer that would determine the Frames Per Second of the game which means that what the user will see is a specific number of images in a second that makes the game look like it's moving, and that number will be what the FPS is equal to.

WIDTH and HEIGHT

Both variables would be integers that indicate the size of the tab that the game will open as and would be passed an argument when the screen is created.

HITBOX_OFFSET

This would contain a dictionary that would indicate the offset for all the different sprites to create a sense of depth and would be used by the YSortCameraGroup class.

weapon_data

This variable would contain a dictionary of the weapon's data such as the graphics, damage and cooldown. This is so that the Player and Weapon classes can reference this data easily.

enemy_data

This variable would also contain a dictionary of each enemy's data such as its health, damage, speed, attack radius, notice radius, attack type, attack sound and more. This is also so that all this data can be easily referenced by the Enemy class when instantiating enemies.

Player.gravity

This attribute is important to determine how fast the player is falling and jumping and would be used to show this movement.

Level.ui

This attribute is an instantiation of the UI class and therefore allows all the UI elements to be displayed while within a level.

Entity.direction

This attribute is also important for dealing with collisions between the player, enemies and objects. It may also help when determining where to replace the player if they fall off the map.

Player.hitbox and Enemy.hitbox

Both attributes are very important when it comes to attacking enemies or being attacked because it establishes how close the player must attack the enemy for it to do damage as well as how close the enemy must attack the player for it to do damage.

Game.clock

This is a very important attribute that allows the game to update itself using the Clock object and allows you to set the FPS of the game. Without this attribute, the game wouldn't update itself and would only show the user an image.

Test Data

Software development lifecycle

For the development of my game, I have decided to use a mix of a RAD (Rapid Application Development) and Agile software development methodology. This is because of the combination of an iterative and sequential development as well as continuous prototyping will be the most beneficial development process for the creation of Hunted Horizon. This means that I will iterate through the different phases of software development while creating a prototype each iteration using the original plan with the clients' feedback. These iterations will continue until I have met the success criteria, and the clients are happy with the solution. Each prototype of the game will have an additional element of the game implemented which would then be unit tested for any errors and to ensure that the element is working correctly – this would speed up the development time since I wouldn't need to test the entire game repeatedly. Once the clients are satisfied, I will also test the game again to ensure that there are no bugs or errors while playing the game.

Different inputs

This is a display of the different inputs and variables within the game that can change during the game as well as the validity of the inputs.

General Controls

Input data	Type
‘W’	Valid
‘A’	Valid
‘S’	Valid
‘D’	Valid
‘Space’	Valid
‘J’	Valid
‘K’	Valid
‘L’	Valid
‘M’	Valid
‘P’	Invalid
1	Invalid
Tab close button	Valid

Controls when menu open

Input data	Type
‘W’	Valid
‘A’	Boundary
‘S’	Valid
‘D’	Boundary
‘Space’	Invalid
‘J’	Invalid
‘K’	Invalid
‘L’	Valid
‘M’	Valid
‘P’	Invalid
1	Invalid
Tab close button	Valid

‘A’ and ‘D’ are boundary because they are only valid while the user is altering the audio settings. This means that the keys wouldn’t do anything if the user isn’t selecting the audio levels.

Player

status

Input data	Type
“up”	Valid
“down”	Valid
“left”	Valid
“right”	Valid
“up_attack”	Valid
“down_attack”	Valid
“left_attack”	Valid
“right_attack”	Valid
“up_idle”	Valid
“down_idle”	Valid
“left_idle”	Valid
“right_idle”	Valid
“up_jump”	Valid
“down_jump”	Valid
“left_jump”	Valid
“right_jump”	Valid
“”	Invalid
“diagonal”	Invalid
5	Erroneous

health

Input data	Type
4	Valid
3	Valid
2	Valid
1	Valid
0	Boundary
5	Invalid

0 is a boundary input because the player’s health reaching 0 is a special case and will affect the game different to having a health of 1 to 4.

-1	Invalid
1.5	Erroneous
“1”	Erroneous

gravity

Input data	Type
10	Valid
-13	Valid
20	Boundary
-20	Boundary
5.8	Erroneous
“8”	Erroneous
200	Invalid
-50	Invalid

20 and -20 are boundary because they are the maximum and minimum of the input data.

coins_earned

Input data	Type
5	Valid
0	Valid
-1	Invalid
“10”	Erroneous
2.3	Erroneous
100	Valid

speed

Input data	Type
5	Valid
6	Invalid
4	Invalid
-1	Invalid
“10”	Erroneous
2.3	Erroneous

vulnerable

Input data	Type
True	Valid
False	Valid
“False”	Erroneous
0	Erroneous
1	Erroneous

weapon

Input data	Type
“sword”	Valid
“dagger”	Invalid
“”	Invalid
True	Erroneous

trophies_gained

Input data	Type
“Rich”	Valid
“Speedy”	Valid
“One-take”	Valid
“”	Invalid
“trophy”	Invalid
1	Erroneous

The valid inputs for this are potential trophy names.

Because trophies_gained will be an array, this is about the data that can be input within trophies_gained.

invulnerability_duration

Input data	Type
500	Valid
-100	Invalid
501	Invalid
499	Invalid
200.5	Erroneous

attack_cooldown

Input data	Type
400	Valid

-100	Invalid
401	Invalid
399	Invalid
200.5	Erroneous

Enemy

sprite_type

Input data	Type
“enemy”	Valid
“player”	Invalid
“npc”	Invalid
“”	Invalid
False	Erroneous

status

Input data	Type
“idle”	Valid
“attack”	Valid
“move”	Valid
“jump”	Boundary
“”	Invalid
“diagonal”	Invalid
3	Erroneous

“jump” is a boundary input since it’s a valid input for specific enemies.

health

Input data	Type
1	Valid
2	Valid
3	Boundary
0	Boundary
-1	Invalid
1.4	Erroneous

3 is a boundary input since it’s a valid input for specific enemies. 0 is a boundary input since it’s a special case.

speed

Input data	Type
2	Valid
3	Valid
4	Valid
-1	Invalid
1	Invalid
5	Invalid
“10”	Erroneous
2.3	Erroneous

attack_damage

Input data	Type
1	Valid
0	Invalid
2	Invalid
-1	Invalid
“1”	Erroneous
1.2	Erroneous

resistance

Input data	Type
3	Valid
2	Invalid
4	Invalid
-1	Invalid
“3”	Erroneous
3.4	Erroneous

attack_radius

Input data	Type
50	Valid
120	Valid
100	Valid

30	Invalid
500	Invalid
-100	Invalid
115.9	Erroneous
“110”	Erroneous

notice_radius

Input data	Type
300	Valid
400	Valid
500	Invalid
200	Invalid
-100	Invalid
“300”	Erroneous
355.5	Erroneous

attack_type

Input data	Type
“slash”	Valid
“ice”	Valid
“claw”	Valid
“bash”	Valid
“”	Invalid
“attack”	Invalid
5	Erroneous

can_attack

Input data	Type
True	Valid
False	Valid
0	Erroneous
1	Erroneous
“true”	Erroneous

attack_cooldown

Input data	Type
400	Valid
401	Invalid
399	Invalid
-200	Invalid
400.6	Erroneous
“400”	Erroneous

vulnerable

Input data	Type
True	Valid
False	Valid
0	Erroneous
1	Erroneous
“false”	Erroneous

invincibility_duration

Input data	Type
300	Valid
301	Invalid
299	Invalid
-300	Invalid
300.56	Erroneous
“300”	Erroneous

Level

game_paused

Input data	Type
True	Valid
False	Valid
0	Erroneous
1	Erroneous
“true”	Erroneous

mode

Input data	Type
“arcade”	Valid
“level”	Valid
“main_map”	Valid
“boss”	Valid
“multiplayer”	Invalid
4	Erroneous

timer

Input data	Type
“07:23:19”	Valid
“60:00:00”	Boundary
“5”	Invalid
3	Erroneous
5.283	Erroneous

“60:00:00” is a boundary input because it would be the timer’s maximum limit. This means that the timer would stop once it reaches this time.

earnable_trophies

Input data	Type
“Scavenger”	Valid
“Speedy”	Valid
“trophy”	Invalid
“”	Invalid
3	Erroneous

wave

Input data	Type
5	Valid
0	Invalid
32	Valid
-1	Invalid
“6”	Erroneous
14.5	Erroneous

NPC

NPC_name

Input data	Type
“Amy”	Valid
“Inspector”	Valid
“”	Invalid
“npc”	Invalid
4	Erroneous

notice_radius

Input data	Type
300	Valid
200	Invalid
-300	Invalid
300.2	Erroneous
“300”	Erroneous

lines

The valid inputs are examples of what might be an NPC’s dialogue, but the actual dialogue may be much longer.

Input data	Type
“Hello!”	Valid
“You should be careful…”	Valid
“”	Invalid
“line”	Invalid
3	Erroneous

Because lines would be an array, this is the validation of the inputs within the array.

AnimationPlayer

frames

Because frames would be a dictionary, this is the validation of the inputs within the dictionary.

Input data	Type
“slash”: import_folder(‘..graphics/particles/slash’)	Valid
“ice”: import_folder(‘..graphics/particles/ice’)	Valid
“claw”: import_folder(‘..graphics/particles/claw’)	Valid
“bash”: import_folder(‘..graphics/particles/bash’)	Valid
“attack”: import_folder(‘..graphics/particles/attack’)	Invalid

“”: import_folder(‘..graphics/particles’)	Invalid
“attack”	Erroneous
1	Erroneous

YSortCameraGroup

half_width

Input data	Type
640	Valid
641	Invalid
639	Invalid
-640	Invalid
640.5	Erroneous
“640”	Erroneous

half_height

Input data	Type
360	Valid
361	Invalid
359	Invalid
-360	Invalid
360.5	Erroneous
“360”	Erroneous

Coins

level_total

Input data	Type
9	Valid
30	Borderline
-1	Invalid
0	Invalid
“10”	Erroneous
15.3	Erroneous

30 is a borderline input because it’s the maximum number of coins there can be on a level map.

Entity

frame_index

Input data	Type
0	Valid
1	Valid
2	Valid
4	Valid
5	Borderline
13	Borderline
2.3	Erroneous
-2	Invalid
“2”	Erroneous

Inputs between 5-13 inclusive are borderline because they’re only valid when it’s the player’s frames

animation_speed

Input data	Type
0.15	Valid
0.14	Invalid
0.16	Invalid
-0.15	Invalid
1	Erroneous
“0.15”	Erroneous

Tile

sprite_type

Input data	Type
“enemy”	Valid
“npc”	Valid
“weapon”	Valid
“object”	Valid
“magic”	Valid
“sprite”	Invalid

“”	Invalid
1	Erroneous

Weapon

sprite_type

Input data	Type
“weapon”	Valid
“object”	Invalid
“magic”	Invalid
“”	Invalid
2	Erroneous

ParticleEffect

sprite_type

Input data	Type
“magic”	Valid
“object”	Invalid
“weapon”	Invalid
“”	Invalid
3	Erroneous

frame_index

Input data	Type
0	Valid
1	Valid
4	Valid
7	Valid
8	Invalid
-2	Invalid
3.4	Erroneous
“5”	Erroneous

animation_speed

Input data	Type
0.15	Valid
0.14	Invalid
0.16	Invalid
-0.15	Invalid
1	Erroneous
“0.15”	Erroneous

frames

Input data	Type
import_folder(‘..graphics/particles/ice’)	Valid
(‘..graphics/particles/ice/icespike1.png’)	Valid
(‘..graphics/particles/ice/icespike2.png’)	Valid
(‘..graphics/particles/ice/icespike8.png’)	Valid
“ice”	Invalid
“”	Invalid
1	Erroneous

Timer

active

Input data	Type
True	Valid
False	Valid
0	Erroneous
1	Erroneous
“True”	Erroneous

start_time

Input data	Type
0	Valid
1200	Valid
62874	Valid

-1	Invalid
“4284”	Erroneous
78825.9	Erroneous

end_time

Input data	Type
67237	Valid
73487	Valid
789801	Valid
0	Invalid
-63248	Invalid
372428.24	Erroneous
“44788742”	Erroneous

duration

Input data	Type
27954	Valid
1313	Valid
724	Valid
0	Invalid
-3897	Invalid
78830.593	Erroneous
“53875”	Erroneous

Acceptance testing

Test No.	Test Description	Inputs (Examples)	Expected Outcome
1	Check basic WASD movement and the diagonal movement.	**Valid**: ‘W’ key moves player up, ‘S’ key moves player down, ‘A’ key moves player left, ‘D’ key moves player right. **Invalid**: Any other key on the keyboard.	**Valid**: Player moves according to the valid input at a constant speed. **Invalid**: The other key pressed doesn’t do anything.
2	Check player camera.	**Valid**: player object **Invalid**: display_surface size	**Valid**: The camera (what the user can view) follows the player so the player is always in the middle of the screen. The player is

			also given depth, so it looks like the player is walking in front of or behind objects. **Invalid**: A syntax error is generated, and the game can't be run.
3	Check the main map and the objects within the map.	**Valid**: main map assets **Invalid**: player assets	**Valid**: The main map correctly appears when the game is run, and the player can correctly collide with objects. **Invalid**: A syntax error is generated, and the game can't be run.
4	Check the main map's barriers.	**Valid**: barrier tiles **Invalid**: grass tiles	**Valid**: There are invisible barriers around the map that the player can't walk past. **Invalid**: The main map's edges visibly look like grass and the player can walk past them.
5	Check player jumping movement.	**Valid**: 'Space' key **Invalid**: Any other key on the keyboard	**Valid**: The player jumps upwards, can move midair and fall back down imitating gravity. **Invalid**: The other key pressed doesn't do anything if it doesn't already have a use.
6	Check that player can attack, block and interact using placeholders.	**Valid**: 'J' key for attacking, 'K' key for blocking and 'L' key for interacting **Invalid**: Any other key on the keyboard	**Valid**: A placeholder output is run when the corresponding key is pressed. **Invalid**: The other key pressed doesn't do anything if it doesn't already have a use.
7	Check player animations are working for all current movement.	**Valid**: 'up_0.png' **Invalid**: ''	**Valid**: The player's movements are all accompanied with the correct animation. **Invalid**: A syntax error is generated because it can't find '' and the game can't be run.
8	Check that the weapon is working properly.	**Valid**: 'sword' **Invalid**: ''	**Valid**: The player's weapon correctly appears when the user attacks.

			Invalid: A syntax error is generated because it can't find '' and the game can't be run.
9	Check that blocking is working properly.	**Valid**: 'K' key **Invalid**: Any other key on the keyboard	**Valid**: The player is shown blocking which stops them from being damaged when attacked but they can't move while blocking. **Invalid**: The other key pressed doesn't do anything if it doesn't already have a use.
10	Check that interacting and interactable objects are working properly.	**Valid**: 'L' key **Invalid**: Any other key on the keyboard	**Valid**: If the player is near an interactable object, the object is interacted with. If they aren't near an interactable object, then nothing happens. **Invalid**: The other key pressed doesn't do anything if it doesn't already have a use.
11	Check that the HUD is working properly.	**Valid**: levelHUD **Invalid**: HUD	**Valid**: The level or main map HUD should display. The HUD should be updated using data such as the player's health when there are changes. **Invalid**: A syntax error is generated since the HUD variable isn't defined/doesn't exist and the game can't be run.
12	Check that the first level map loads properly.	**Valid**: level_map1 **Invalid**: map	**Valid**: The first level's map and its objects should load onto the user's screen after deleting the main map and its objects to reduce the game from slowing down. The player should also be placed onto the map properly. **Invalid**: A syntax error is generated since the map variable isn't defined/doesn't exist and the game can't be run.
13	Check that falling works properly.	**Valid**: gravity **Invalid**: fall	**Valid**: The player should be able to fall off the map

			appropriately with the correct animation and logic if the user moves the player off the walkable areas of the map. After falling, the player should also be repositioned properly with less health than they did prior to falling. **Invalid**: A syntax error is generated since the fall variable isn't defined/doesn't exist and the game can't be run.
14	Check that enemies appear properly.	**Valid**: image **Invalid**: object	**Valid**: The correct type of enemy should appear in the level at their specific position when the level starts. **Invalid**: A syntax error is generated since the object variable isn't defined/doesn't exist and the game can't be run.
15	Check that enemies move properly.	**Valid**: status **Invalid**: attack_type	**Valid**: The enemy should be still until the player is near, then the enemy should follow the player at a constant speed. **Invalid**: This would be a logical error, and the enemies may not move properly since their attack_type isn't required within their movement.
16	Check that the raccoon enemy's attacks are working properly.	**Valid**: attack_type **Invalid**: enemy_eat	**Valid**: The raccoon enemy should be able to do any of its attacks to the player with the correct cooldowns and type of attack. The attacks should also be deleted once it has happened. **Invalid**: A syntax error is generated since the enemy_eat variable isn't defined/doesn't exist and the game can't be run.
17	Check that the cyclops enemy's attacks are working properly.	**Valid**: attack_time **Invalid**: enemy_exp	**Valid**: The cyclops enemy should be able to do any

			of its attacks to the player with the correct cooldowns and type of attack. The attacks should also be deleted once it has happened. **Invalid**: A syntax error is generated since the enemy_exp variable isn't defined/doesn't exist and the game can't be run.
18	Check that the slime enemy's attacks are working properly.	**Valid**: attack_cooldown **Invalid**: enemy.hitbox	**Valid**: The slime enemy should be able to do any of its attacks to the player with the correct cooldowns and type of attack. The attacks should also be deleted once it has happened. **Invalid**: A logical error may occur by using the enemy's hitbox instead of the player's hitbox and may mean that the enemy can hurt the player just by attacking randomly.
19	Check that the enemies can die properly.	**Valid**: health **Invalid**: status	**Valid**: Once an enemy's health reaches zero, they must be deleted. **Invalid**: A syntax error is generated since a string input wouldn't be required to check this.
20	Check that the enemies can fall properly.	**Valid**: status **Invalid**: damage_player	**Valid**: The enemy should fall off the map using the correct logic if the enemy moves off the walkable areas of the map. **Invalid**: A syntax error may be generated because the damage_player variable may not fit any of the inputs' data types within checking this.
21	Check that the enemy animations are all working properly.	**Valid**: animations **Invalid**: resistance	**Valid**: All the enemies' movements should be accompanied with the corresponding animation. **Invalid**: This may cause a syntax or logical error

			depending on where the resistance variable is used. Overall, this would either make the game crash or hard to understand since the movements aren't properly conveyed.
22	Check that enemy-player interactions are working properly.	**Valid**: can_attack **Invalid**: display_surface	**Valid**: All the interactions between the enemy and player should work as intended: attacking, blocking, resistance, killing, etc. **Invalid**: A syntax error will be generated since there isn't a need for a variable of data type Surface in this section of code.
23	Check that the death screen works properly.	**Valid**: display_surface **Invalid**: 3	**Valid**: Once the player dies, a death screen should be displayed which allows the player to return to the main map. **Invalid**: A syntax error would be generated, and the game wouldn't be able to be run.
24	Check that coins appear and work properly.	**Valid**: hitbox **Invalid**: weapon	**Valid**: The player should be able to see coins within levels that disappear once the player touches them because the player has taken the coin. The coin should be deleted once it has been found and the player's coin total should increase. **Invalid**: A syntax error would be generated because there wouldn't be a string input in this part of the code, and the game wouldn't run.
25	Check that the user can open a menu and interact with it.	**Valid**: 'M' key **Invalid**: Any other key on the keyboard	**Valid**: When the user presses the 'M' key, the menu should open. While the menu is open, the only keys that do something when pressed are the WASD, M and L

			keys (A and D only when selecting audio). They can use W and S to switch between options on the menu and L to select an option. The currently selected option must be highlighted in a different colour. To escape the menu, the user can press M again. **Invalid**: The other key pressed doesn't do anything if it doesn't already have a use.
26	Check that the menu pauses the game properly.	**Valid**: game_paused **Invalid**: hitbox	**Valid**: While the menu is open, the game should be paused in the background so the user can access the menu. When the user closes the menu, the game should be un-paused from where it left off from. **Invalid**: A syntax error would be generated since the menu can't have a hitbox as it's not a sprite and the game wouldn't run.
27	Check that the controls part of the menu works properly.	**Valid**: 'controls.png' **Invalid**: ''	**Valid**: When the player selects the controls through the menu, there should be an image of the controls displayed as well as a back button so the user can return to the main menu. **Invalid**: A syntax error is generated because it can't find '' and the game can't be run.
28	Check that the first level can end properly.	**Valid**: complete **Invalid**: display_surface	**Valid**: Once the player reaches the end of the level, the level should end and indicate it has ended. **Invalid**: A syntax error will be generated since there isn't a need for a variable of data type Surface in this section of code.

29	Check that the timer works properly.	**Valid**: start_time **Invalid**: hours	**Valid**: When the player starts a level, the time should be note and when they reach the end of the level, the time should be noted again. The end time is subtracted from the start time and that is the duration. The duration is then converted into minutes, seconds and milliseconds and made into a string which is returned. The time should also not go above an hour, if it does, the time should stop at an hour and return that as the time. **Invalid**: A syntax error is generated since the hours variable isn't defined/doesn't exist and the game can't be run.
30	Check that reaching the end of a level has a working completion screen.	**Valid**: time **Invalid**: enemies_killed	**Valid**: The completion screen should show the time taken, coins gained from the level and a placeholder for any trophies gained. It should also have an option to return to the main map and a placeholder for restarting the level. This screen should appear once the player reaches the end of a level. **Invalid**: A syntax error is generated since the enemies_killed variable isn't defined/doesn't exist and the game can't be run.
31	Check that the trophies for the first level work properly.	**Valid**: earnable_trophies **Invalid**: coins_earned	**Valid**: If the user has completed a specific trophy's task that is achievable within the first level, the user should gain that trophy. **Invalid**: A syntax error is generated since there

			isn't a need for an integer variable in this part of the code and the game wouldn't run.
32	Check that the arcade mode map loads properly.	**Valid**: arcade_map **Invalid**: level_map1	**Valid**: The arcade map and its objects should load onto the user's screen after deleting the main map and its objects to reduce the game from slowing down. The player should also be placed onto the map properly. **Invalid**: A logical error would occur because of the mismatch of the maps which means that the game would still run but not as intended.
33	Check that enemies can be summoned in waves in the arcade mode.	**Valid**: wave **Invalid**: coins_earned	**Valid**: The waves should start from one and increment when all the enemies from the previous wave have been killed. When the wave increments, another set of random enemies should spawn. This should repeat until the player dies. **Invalid**: This variable wouldn't be used yet and may cause a logical error if it was input into this section of the code.
34	Check that the player's death in arcade mode results in a completion screen instead.	**Valid**: coins_earned **Invalid**: duration	**Valid**: When the player dies within arcade mode, instead of the death screen displayed in the levels, a completion screen showing the number of waves completed, any trophies gained and how many coins the user has earned from arcade mode. The screen will also have a return to main map option and a placeholder to replay the arcade mode.

			Invalid: This would cause a logical error because the arcade mode wouldn't have a timer active while the user plays it.
35	Check that the arcade mode HUD works properly.	**Valid**: arcadeHUD **Invalid**: HUD	**Valid**: The arcade HUD should display. The HUD should be updated using data such as the player's health and the wave number when there are changes. **Invalid**: A syntax error is generated since the HUD variable isn't defined/doesn't exist and the game can't be run.
36	Check that the boss level map loads properly.	**Valid**: boss_map1 **Invalid**: arcade_map	**Valid**: The boss map and its objects should load onto the user's screen after deleting the main map and its objects to reduce the game from slowing down. The player should also be placed onto the map properly. **Invalid**: A logical error would occur because of the mismatch of the maps which means that the game would still run but not as intended.
37	Check that the boss' attacks are working properly.	**Valid**: attack_type **Invalid**: ''	**Valid**: The boss' attack should be done periodically and across the specific area on the map. **Invalid**: A syntax error is generated because '' doesn't exist.
38	Check that the boss can summon other enemies properly.	**Valid**: attack_type **Invalid**: can_summon	**Valid**: The boss should summon enemies randomly while attacking within the boss area. **Invalid**: A syntax error is generated since the can_summon variable isn't defined/doesn't exist and the game can't be run.

39	Check that the buttons work properly in the boss battle.	**Valid**: 'L' key **Invalid**: 'M' key	**Valid**: The buttons on the boss map should be able to be interacted with, one by one, by the player after a certain time has passed. When pressed, the buttons harm the boss and can't be pressed again. **Invalid**: The menu would open for the player instead of interacting with any of the buttons.
40	Check that the boss' animations work properly.	**Valid**: animations **Invalid**: attack_cooldown	**Valid**: All the enemies' movements should be accompanied with the corresponding animation. **Invalid**: This may cause a syntax or logical error depending on where attack_cooldown is used. Overall, this would either make the game crash or hard to understand since the movements aren't properly conveyed.
41	Check that the boss battle HUD works properly.	**Valid**: bossHUD **Invalid**: HUD	**Valid**: The boss battle HUD should display. The HUD should be updated using data such as the player's and boss' health when there are changes. **Invalid**: A syntax error is generated since the HUD variable isn't defined/doesn't exist and the game can't be run.
42	Check that defeating the boss ends the level and shows a completion screen.	**Valid**: health (boss) **Invalid**: enemies_killed	**Valid**: Once the boss' health is depleted and the boss' death animation finishes, the completion screen should appear. The completion screen should be like the level completion screen except the title would be different and the placeholder option would be a restart boss battle option.

			Invalid: A syntax error is generated since the enemies_killed variable isn't defined/doesn't exist and the game can't be run.
43	Check that the trophy area on the main map works properly.	**Valid**: trophies_gained **Invalid**: trophy_desc	**Valid**: At the trophy area, the player can interact to see how many trophies they've gained and how many they haven't earned. **Invalid**: A syntax error is generated since the trophy_desc variable isn't defined/doesn't exist and the game can't be run.
44	Check that the second level loads and works properly.	**Valid**: level_map2 **Invalid**: boss_map1	**Valid**: The second level map and its objects should load onto the user's screen after deleting the main map and its objects to reduce the game from slowing down. The player should also be placed onto the map properly. **Invalid**: A logical error would occur because of the mismatch of the maps which means that the game would still run but not as intended.
45	Check that the third level loads and works properly.	**Valid**: level_map3 **Invalid**: level_map2	**Valid**: The third level map and its objects should load onto the user's screen after deleting the main map and its objects to reduce the game from slowing down. The player should also be placed onto the map properly. **Invalid**: A logical error would occur because of the mismatch of the maps which means that the game would still run but not as intended.
46	Check that the NPCs load properly.	**Valid**: NPC_name **Invalid**: lines	**Valid**: The NPCs should appear on the main map and the player shouldn't

			be able to walk over them. **Invalid**: A syntax error is generated since the lines variable isn't used in this part of the code and it doesn't match with any of the data types used in this section of code.
47	Check that NPC interaction works properly.	**Valid**: lines **Invalid**: display_surface	**Valid**: The player should be able to interact with the NPC when they are near. The NPC should say a few lines of dialogue at the bottom of the screen and the player can press interact again to get to the next line. The dialogue only appears for a while and is then deleted. If the user continues the dialogue while another line is running, then the previous line is deleted and replaced with the new one. **Invalid**: A syntax error will be generated since there isn't a need for a variable of data type Surface in this section of code.
48	Check that player progress is saved properly.	**Valid**: trophies_gained **Invalid**: enemies_killed	**Valid**: After the user has completed a level/the arcade mode/a boss level, that data is saved for when the player returns to the game. **Invalid**: A syntax error is generated since the enemies_killed variable isn't defined/doesn't exist and the game can't be run.
49	Check that players can return to the main map through the menu properly.	**Valid**: 'L' key **Invalid**: Any other key on the keyboard when trying to select return to main map on the menu	**Valid**: The player can return to the main map through the menu but if they are in a level/arcade mode, they lose their progress. **Invalid**: The other key pressed doesn't do

			anything if it doesn't already have a use.
50	Check that players can restart levels/arcade mode/boss battles.	**Valid**: 'L' key **Invalid**: 'M' key	**Valid**: All the restart placeholders on the completion screens should now work. Restarting from a completion screen doesn't mean you lose your previous data. **Invalid**: The 'M' key shouldn't do anything while the completion screen is active.
51	Check that the start screen works properly.	**Valid**: 'start.png' **Invalid**: ''	**Valid**: When the game is loaded, the first thing the user must see is a start screen from which the user can then start playing by pressing the spacebar. **Invalid**: A syntax error is generated because it can't find '' and the game can't be run.
52	Check that the loading screens work properly.	**Valid**: 'loading.png' **Invalid**: ''	**Valid**: In between different modes there should be a loading screen displayed while the game deletes the previous map and loads the new one. **Invalid**: A syntax error is generated because it can't find '' and the game can't be run.
53	Check that trophies work properly.	**Valid**: earnable_trophies **Invalid**: weapon	**Valid**: Trophies can be gained by the player once they complete the task for that trophy. Once they have gained that trophy, they can't gain it again. **Invalid**: A logical error may occur that may confuse users on the trophies.
54	Check that completion screens display information properly.	**Valid**: coins **Invalid**: enemies_killed	**Valid**: The completion screen should correctly display the information for the specific game mode's completion screen. If there is no data

			then it should display none instead. **Invalid**: A syntax error is generated since the enemies_killed variable isn't defined/doesn't exist and the game can't be run.
55	Check that levels can be appropriately blocked.	**Valid**: complete **Invalid**: level_map2	**Valid**: The levels should be played in linear, and they only unlock once other levels have been completed. This is also true for the boss battle. If the player hasn't met the requirements, then the entrance to the level is blocked. **Invalid**: A syntax error is generated since the data type wouldn't match any of the code in this section.
56	Check that the main map can update to show available levels.	**Valid**: main_map **Invalid**: level_map1	**Valid**: The main map should alter slightly to show available levels and non-available levels for the user. **Invalid**: A logic error would occur since the alteration of the map may not work as intended.
57	Check that player audio works properly.	**Valid**: weapon_attack_sound **Invalid**: ''	**Valid**: Most actions made by the player should be accompanied by a matching sound. **Invalid**: A syntax error is generated because it can't find '' and the game can't be run.
58	Check that enemy audio works properly.	**Valid**: death_sound **Invalid**: weapon_attack_sound	**Valid**: Most actions made by an enemy should be accompanied by a matching sound. **Invalid**: The game will still run but this would be a logic error since the sounds wouldn't match up.
59	Check that system audio works properly.	**Valid**: coin_sound **Invalid**: 'coin.png'	**Valid**: Any other actions must be accompanied by a matching sound.

			Invalid: A syntax error is caused since the data type wouldn't match up in this section of code.
60	Check that game music works properly.	**Valid**: main_sound **Invalid**: death_sound	**Valid**: There should be music for each of the levels in the game that loops while the user plays. **Invalid**: The game will still run but this would be a logic error since the audio wouldn't match up.
61	Check that the audio settings menu works properly.	**Valid**: WASD keys **Invalid**: Any other key on the keyboard	**Valid**: The audio levels are preset for the user however they can change according to what the user has set. **Invalid**: The other key pressed doesn't do anything if it doesn't already have a use.
62	Check that player progress can be loaded properly.	**Valid**: player_progress **Invalid**: display_surface	**Valid**: The game should check if the user has saved progress on the game already and load it if that is the case. **Invalid**: A syntax error will be generated since there isn't a need for a variable of data type Surface in this section of code.

These tests will be carried out as unit tests. After all these tests, I will test the game overall to ensure that everything is working as it should. If there are extra errors, I will document them while I fix them. I will also do some beta testing using my client and stakeholders so I can identify potential errors and improvements to Hunted Horizon. Any extra features to the game will also be documented and tested similarly to the rest of the already determined features.

Sign off proposal

Interview with main client:

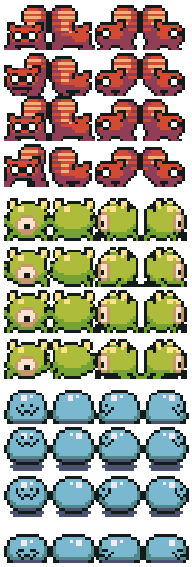
> Is there anything that you would change to the game?
>
> "No, I'm impressed with the work done so far. I really like the initial designs and feel this would exceed my expectations."
>
> Is there anything that you would like to add to the game?
>
> "I'm not sure if there's anything I'd like to add since there seems to be a lot to already implement."

Do you confirm to go ahead with the development of the game?

"As I am happy with the overall design of the game, I would like to advance to the next stages of development."

Development

Before starting the coding of Hunted Horizon, I did another survey using the different player sprites to see which sprite my stakeholders liked the most. There were 18 different sprites that I picked from the Ninja Adventure Asset Pack, and I gave the survey's participants a choice to rate each of the sprites 1-5 stars. The sprite that was the highest rated (4.57 average rating) is the sprite that I will use for the player and is displayed on the left.



I also chose what the enemies' sprites would look like to have a range of attacks and so that none of the enemies were too similar. These were the three sprites that I ended up choosing: a raccoon sprite, cyclops sprite and slime sprite. For the user to easily see the enemies, I chose sprites that had bright colours and were all different colours. I based each of the sprites on what their attacks would be:

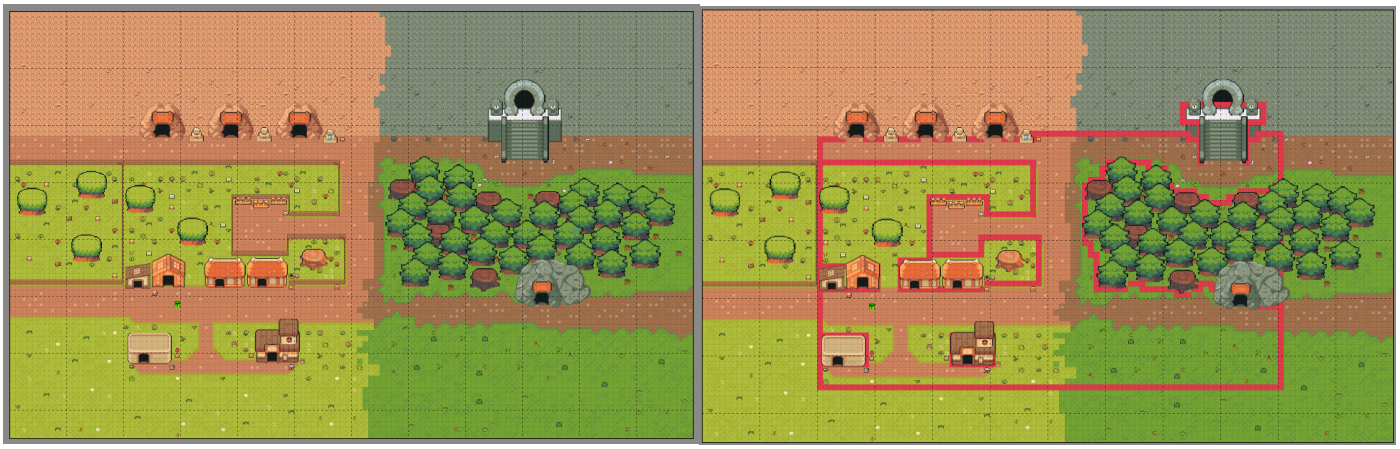
The raccoon enemy would be the short-range attacking enemy, and its attack would be a claw attack.

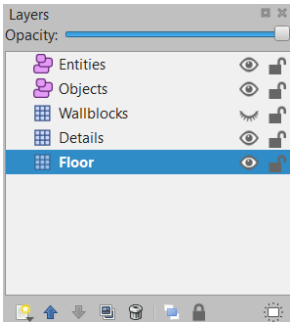
The cyclops enemy would be the short-range and mid-range attacking enemy with a headbutt and stomp attack.

The slime enemy would be the long-range attacking enemy and would attack by shooting ice spikes.

I chose to create some of the levels using Tiled and the Ninja Adventure Asset Pack. Because I hadn't used Tiled before, I watched tutorials to help me understand how to use it specifically for Pygame. I planned what each of the levels may look like as well as the enemies that would be in them. To increase the difficulty within the levels, a new enemy would be introduced each level, therefore I based each of the levels on the enemy introduced. I created sketches for all the levels to help me create them on Tiled however I only created the main map and level 1 map in case alterations may be needed. This is because when creating the maps, I also had to add extra tiles to indicate invisible barriers for the player and I may need extra tiles on the levels to help indicate valid jumps for the player since the logic of jumping must mimic 3D logic on a 2D platform.

These are screenshots of what the main map looks like on Tiled:

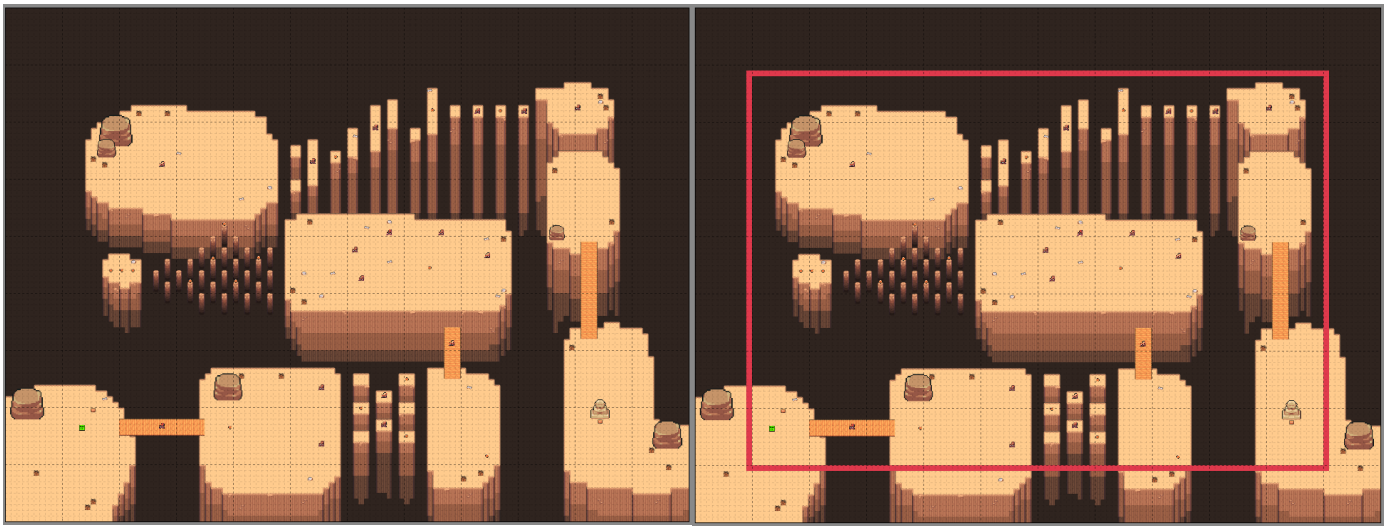


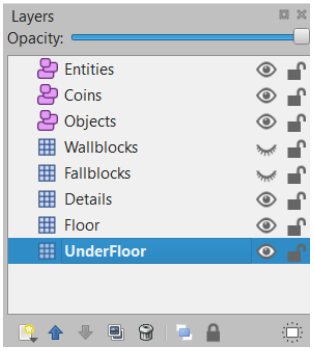


The red lines are the invisible barriers and there are extra parts of the map beyond the barriers so when the player is next to the barrier, they can still see the map and it isn't just cut off. The three caves on the top left will be the entrances for the levels and the statues next to them will indicate whether the level can be accessed. The entrance with a staircase is the entrance to the boss fight and the cave at the bottom is the entrance to the arcade mode. There is also a small trophy area where the player can view their trophies. In addition, there's also a marker to represent where the player will spawn on the map.

The layers of the Tiled map are specifically split up so they can be easily converted for Pygame. However, these layers might be different for each map and may be altered during the development of the game.

These are the screenshots of what the first level looks like on Tiled:





The red lines again represent the invisible barriers and there's also a marker to indicate where the player will spawn. This map isn't the same size as the main map because it requires more space upwards to contain all the obstacles however it may need resizing once the coding starts so that the map doesn't look too big when playing Hunted Horizon.

This map also has the enemies and coins on them, and its layers may look more like this instead to contain the logic of coins and jumping but this may be altered when coding the logic.

To aid with the coding and creation of my game, I am going to use sources such as Clear Code's YouTube channel[9] and their videos[10] [11] [12] as well as Atlas' YouTube channel[13] and their video[14] to help understand how I will create my game in addition to any other sources I will use that will be mentioned. I will also be using the Pygame documentation website[15] to help me.

Before developing the code, I have also created and structured the file folders for the game code and its assets. I have split the game into four folders called 'audio', 'code', 'graphics' and 'map' to make accessing and locating files easier during the development. The code within the 'code' folder will also be split up into sections so that the code isn't input into one single large file to improve clarity during development. So that all the code can be accessed between the code files, they will be imported into the necessary files.

The code will be created within Visual Studio Code's IDE.

Basic Movement

The initial code has been split into three Python files: main, settings and debug. The main program is what will be run to start the game and will contain the main loop. The settings file will contain information about the game such as the FPS, height and width of the screen, and more. This is so that they can be altered outside of the main code. The debug code file isn't required for the game to work however it would allow the developers to view the changes of certain variables while the game is running.

main.py

```python
import pygame, sys
from settings import *

class Game:
    def __init__(self):

        # General setup
        self.screen = pygame.display.set_mode((WIDTH,HEIGHT))
        pygame.display.set_caption('Hunted Horizon')
        self.clock = pygame.time.Clock()

    def run(self):
        while True:
            for event in pygame.event.get():

                # Allows user to close program
                if event.type == pygame.QUIT:
                    pygame.quit()
                    sys.exit()

            # Screen
            self.screen.fill('black')
            pygame.display.update()
            self.clock.tick(FPS)

if __name__ == '__main__':
    game = Game()
    game.run()
```

settings.py

```python
# game setup
WIDTH = 1280
HEIGHT = 720
FPS = 60
TILESIZE = 64

WORLD_MAP = [
['x','x','x','x','x','x','x','x','x','x','x','x','x','x','x','x','x','x','x','x'],
['x',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ','x'],
['x',' ','p',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ','x'],
['x',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ','x','x','x','x',' ',' ',' ',' ','x'],
['x',' ',' ','x',' ',' ',' ',' ',' ',' ',' ',' ',' ','x',' ',' ',' ',' ',' ','x'],
['x',' ',' ','x',' ',' ',' ',' ',' ',' ',' ',' ',' ','x',' ',' ',' ',' ',' ','x'],
['x',' ',' ','x',' ',' ',' ',' ',' ',' ',' ',' ',' ','x',' ',' ',' ',' ',' ','x'],
['x',' ',' ','x',' ',' ',' ','x',' ',' ',' ',' ',' ','x',' ',' ',' ',' ',' ','x'],
['x',' ',' ','x',' ',' ',' ',' ',' ',' ',' ',' ',' ','x',' ',' ',' ',' ',' ','x'],
['x',' ',' ','x',' ',' ',' ',' ',' ',' ',' ',' ',' ','x',' ',' ',' ',' ',' ','x'],
['x',' ',' ','x',' ',' ',' ',' ',' ',' ',' ',' ',' ','x',' ',' ',' ',' ',' ','x'],
['x',' ',' ','x',' ',' ',' ',' ',' ',' ',' ',' ',' ','x',' ',' ',' ',' ',' ','x'],
['x',' ',' ','x',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ','x'],
['x',' ',' ',' ',' ',' ','x',' ','x',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ','x'],
['x',' ',' ',' ',' ',' ','x',' ','x',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ','x'],
['x',' ',' ',' ',' ',' ','x','x','x',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ','x'],
['x',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ','x','x','x','x',' ',' ',' ','x'],
['x',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ','x'],
['x',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ','x'],
['x','x','x','x','x','x','x','x','x','x','x','x','x','x','x','x','x','x','x','x'],
]
```

debug.py

```python
# Displays information for debugging in
import pygame
pygame.init()
font = pygame.font.Font(None,30)

def debug(info, y=10, x=10):
    display_surface = pygame.display.get_surface()
    debug_surf = font.render(str(info),True,'White')
    debug_rect = debug_surf.get_rect(topleft=(x,y))
    pygame.draw.rect(display_surface,'Black',debug_rect)
    display_surface.blit(debug_surf,debug_rect)
```

This initial code is used to set the basic screen of the game, which is a black screen with tab name 'Hunted Horizon' currently, but it uses the dimensions from the settings file and the user should be able to close the

screen. The settings file also contains a test world map so that I can code the player's basic movement without implementing the actual map first. The test world map is a 2D array where each item is a string that's either empty (indicating an empty tile), has an x (indicating an object tile) or a p (indicating the player). The debug file isn't initially being used but it creates a text box at the top left of the screen so I can view any information that has passed as an argument within the debug function while the game is running. As this process isn't specific to the development of Hunted Horizon and is like the layout of many other Pygame programs, there was no errors in creating the code and I have omitted the testing for this initial code.

To display the world test map, I had to create three other Python files: level, tile and player. The main, settings and debug file didn't require to be changed. This code allows the user to see the map and the player however they can't interact with the game yet.

The level file requires these imports from Pygame and the other Python files because it uses their functions and variables.

This simply establishes the two current sprite groups: visible and obstacle sprites. The visible_sprites attribute is a list that will contain the visible sprites that are visible to the user. The obstacle_sprites attribute is another list of sprites that can interact with the player. They are split because some obstacle sprites like a game barrier won't be visible to the user.

The create_map function takes each value within the 2D array and uses it to map out the test world map tiles. Each of the values' index requires to be multiplied by the tile size to get its coordinates so that the world map is displayed on the screen like a grid, not overlapping each other. The two if statements allow either a rock or the player to be displayed using the Tile and Player methods with the specific tile's coordinates and groups. The Player method only requires visible_sprites since there isn't any part of the Player that won't be visible, and the user (player) wouldn't need to interact with itself.

main.py | settings.py | debug.py | level.py | tile.py | player.py

level.py > ...

```python
import pygame
from settings import *
from tile import Tile
from player import Player

class Level:
    def __init__(self):

        # Get display surface
        self.display_surface = pygame.display.get_surface()

        # Sprite group setup
        self.visible_sprites = pygame.sprite.Group()
        self.obstacle_sprites = pygame.sprite.Group()

        # Sprite setup
        self.create_map()

    def create_map(self):
        for row_index,row in enumerate(WORLD_MAP):  # Enumerate returns row index number for each row as row_index
            for col_index,col in enumerate(row):
                x = col_index * TILESIZE
                y = row_index * TILESIZE
                if col == 'x':
                    Tile((x,y),[self.visible_sprites, self.obstacle_sprites])
                if col == 'p':
                    Player((x,y),[self.visible_sprites])


    def run(self):
        # Update + draw game
        self.visible_sprites.draw(self.display_surface)
```

This gets the display surface of the current screen so that it can be drawn on.

This just displays the visible_sprites on the screen.

This class creates the visual and interactable parts of each tile. This is why the position and groups are needed as arguments for the object. Tile inherits the Sprite class for later use. The groups attribute uses super init because the attribute is an object of type Group, meaning that Tile is inheriting from the Group class.

main.py | settings.py | debug.py | level.py | tile.py | player.py

tile.py > ...

```python
import pygame
from settings import *

class Tile(pygame.sprite.Sprite):
    def __init__(self,pos,groups):
        super().__init__(groups)
        self.image = pygame.image.load('../graphics/test/rock.png').convert_alpha()
        self.rect = self.image.get_rect(topleft = pos)
```

This simply loads the image and uses the image to create the Tile object's rectangle (rect). The convert alpha is used on the image to optimize the image for fast blitting – the image is drawn quicker.

The player file is currently the exact same as the tile file except the image used is different and the class' name is Player.

main.py | settings.py | debug.py | level.py | tile.py | player.py

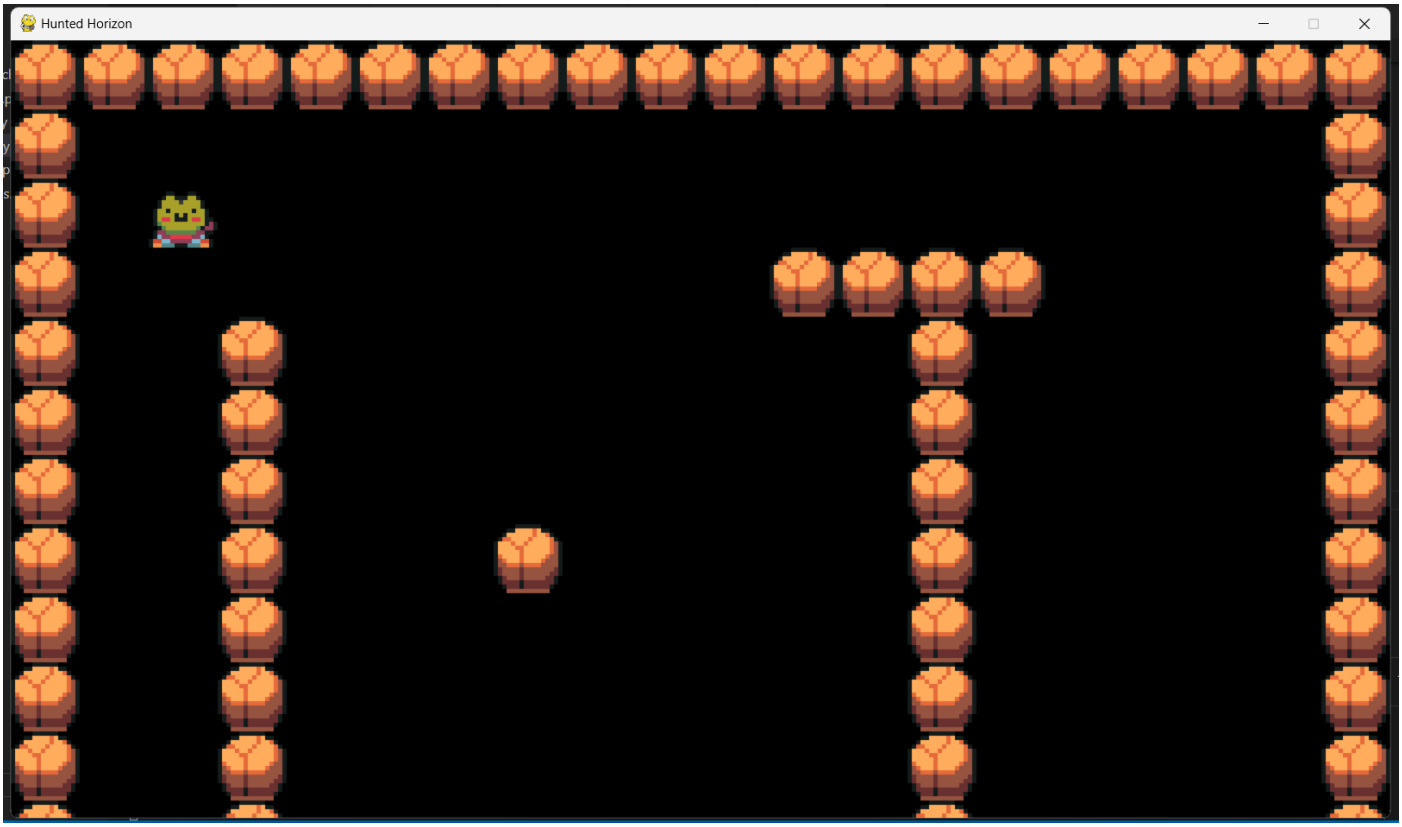
player.py > ...

```
1   import pygame
2   from settings import *
3
4   class Player(pygame.sprite.Sprite):
5       def __init__(self,pos,groups):
6           super().__init__(groups)
7           self.image = pygame.image.load('../graphics/test/player.png').convert_alpha()
8           self.rect = self.image.get_rect(topleft = pos)
```

There was only one simple error that was encountered that was a TypeError because I had forgotten to add the brackets after super in line 6 of the tile file.

```
TypeError: descriptor '__init__' requires a 'super' object but received a 'list'
PS C:\Users\gursi\OneDrive - The Holt School\Year 12\Computing\Coursework\Game\code>
```

When the current code runs, this is what the user can see, and it is what I expect to see. The rocks are currently being used as placeholders for barriers.



To make the player move, I need to first find out how to check if the user has pressed the specific key and the directional change of the player. This only required me to edit the level and player files in addition to utilising the debug file.

level.py > ...

```python
import pygame
from settings import *
from tile import Tile
from player import Player
from debug import debug
```

To utilise the debug function from the debug file, I had to import it into the level file.

level.py > ...

```python
class Level:
    def create_map(self):
        for row_index,row in enumerate(WORLD_MAP):  # Enumerate returns row index number for each row as row_index
            for col_index,col in enumerate(row):
                x = col_index * TILESIZE
                y = row_index * TILESIZE
                if col == 'x':
                    Tile((x,y),[self.visible_sprites, self.obstacle_sprites])
                if col == 'p':
                    self.player = Player((x,y),[self.visible_sprites])

    def run(self):
        # Update + draw game
        self.visible_sprites.draw(self.display_surface)
        self.visible_sprites.update()
        debug(self.player.direction)
```

The player object was placed inside a variable to make it easier to reference throughout the code and development.

The update method called is a part of the Pygame Group class and simply updates the sprites as the game runs. The debug function allows us to see the player's direction attribute change while the game is running.

The comment explains how the Vector2 class works but because the later code alters its data, it can be used to figure out the direction of where the player must move/is moving.

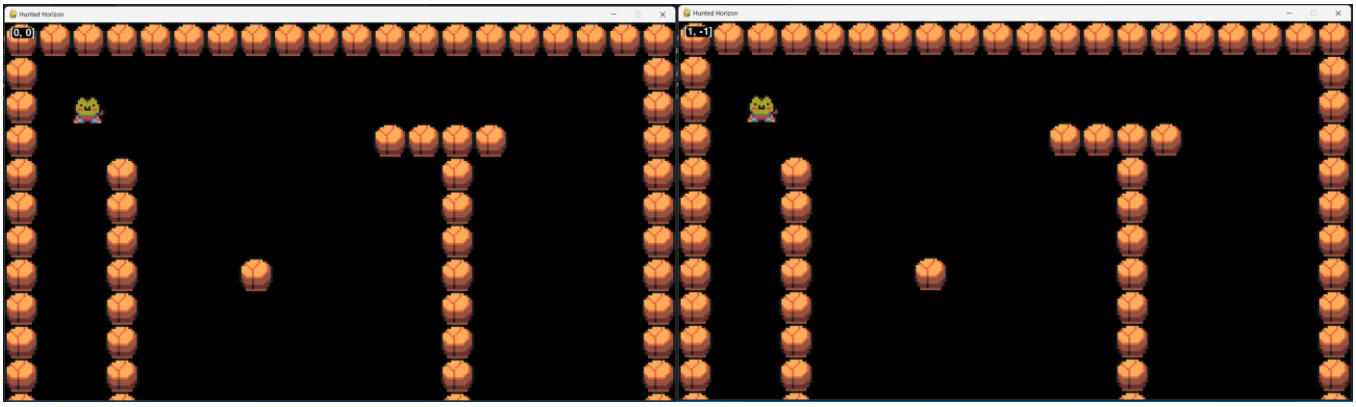
player.py > ...

```python
class Player(pygame.sprite.Sprite):
    def __init__(self,pos,groups):
        # Player movement
        self.direction = pygame.math.Vector2()  # An array containing the x and y coords of the player

    def input(self):
        keys = pygame.key.get_pressed()

        if keys[pygame.K_w]:
            self.direction.y = -1
        elif keys[pygame.K_s]:
            self.direction.y = 1
        else:
            self.direction.y = 0

        if keys[pygame.K_a]:
            self.direction.x = -1
        elif keys[pygame.K_d]:
            self.direction.x = 1
        else:
            self.direction.x = 0

    def update(self):
        self.input()
```

This method is used for the different keyboard inputs that the user might give. The get_pressed function allows us to check what button was pressed so the user can interact with the game. Currently the basic WASD controls alters the direction's coordinates so the direction changes using these buttons, which can be used to help move the character. The x and y are separated since they can be changed at the same time.

This allows the user's inputs and attributes in the Player object to be updated while the game is running.

Now when the game is running, a rectangle with the player direction can be seen in the top left and it's updated while the movement keys are pressed. The left image is what is shown when a non-interactable key is pressed and the right image is when the 'W' and 'S' key are pressed simultaneously.

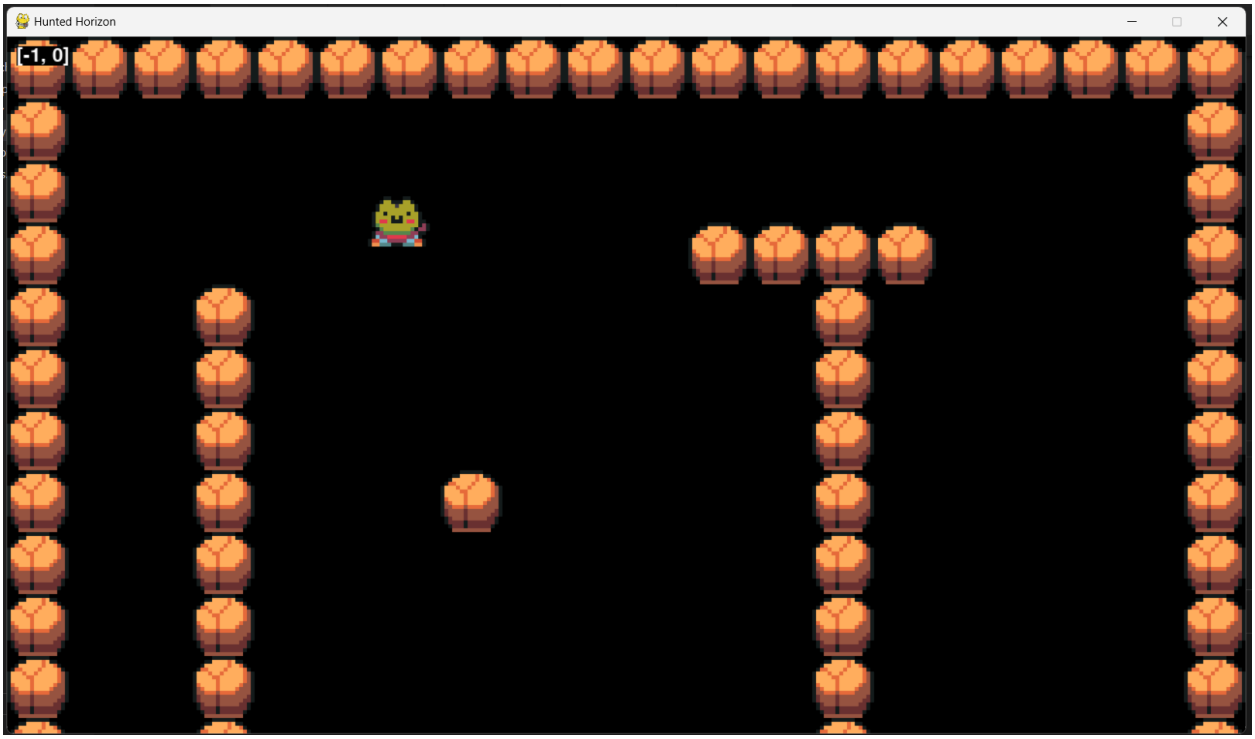


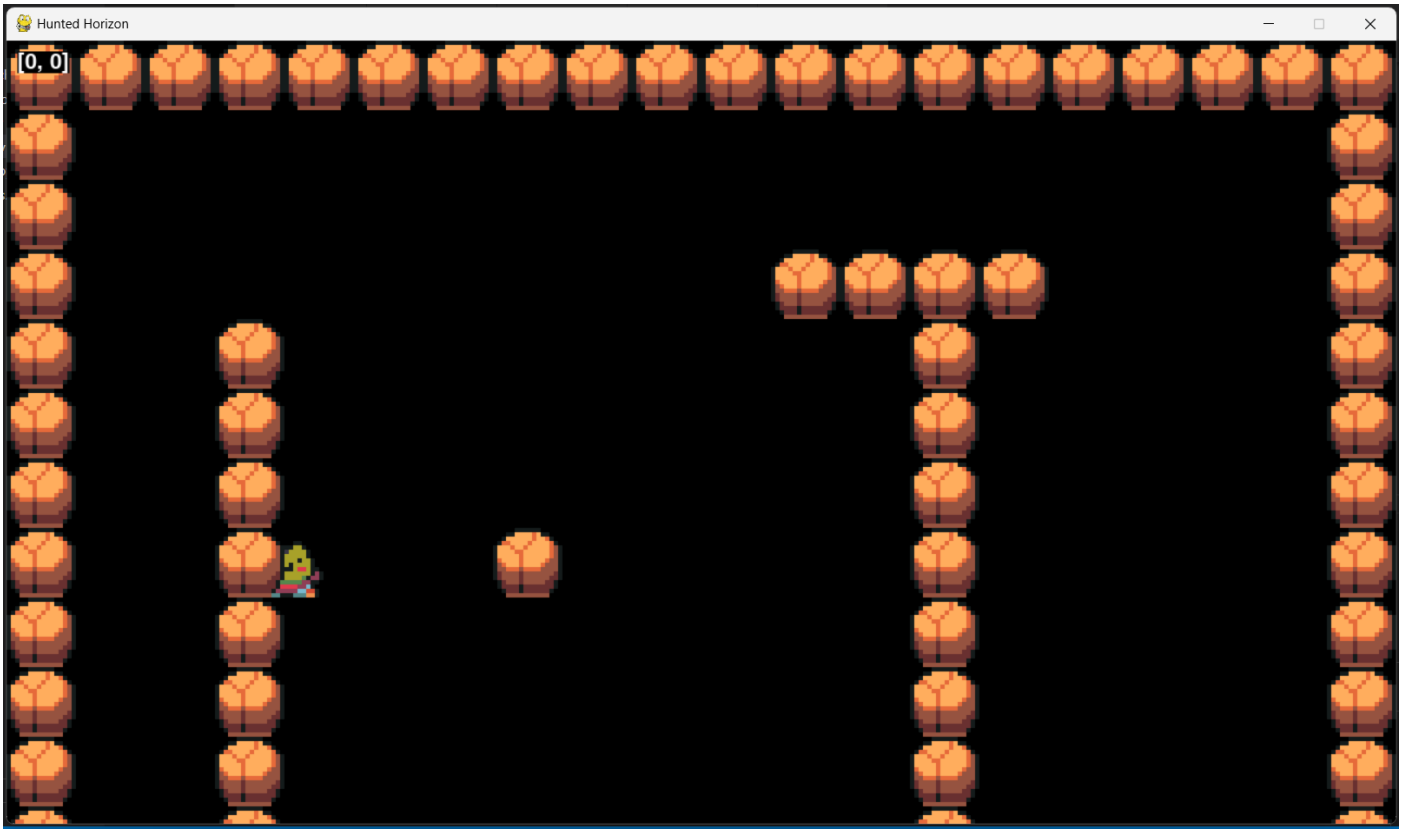
To simply allow the player to move, I added a speed attribute, created a move method and placed that move method within the update method so that the player can move when playing the game. The move function simply takes the centre of the player object's rectangle and alters its coordinates using the incrementation of the direction multiplied by the speed. The speed attribute simply indicates how many pixels the player would cross while moving. Later in the development, the speed attribute will be removed because the player's data will be stored within an array which will be accessed to get the player's speed. The speed method will also be moved to the future Entity class so it can be inherited by both the Player and the Enemy classes which is why I have kept the speed method more general.

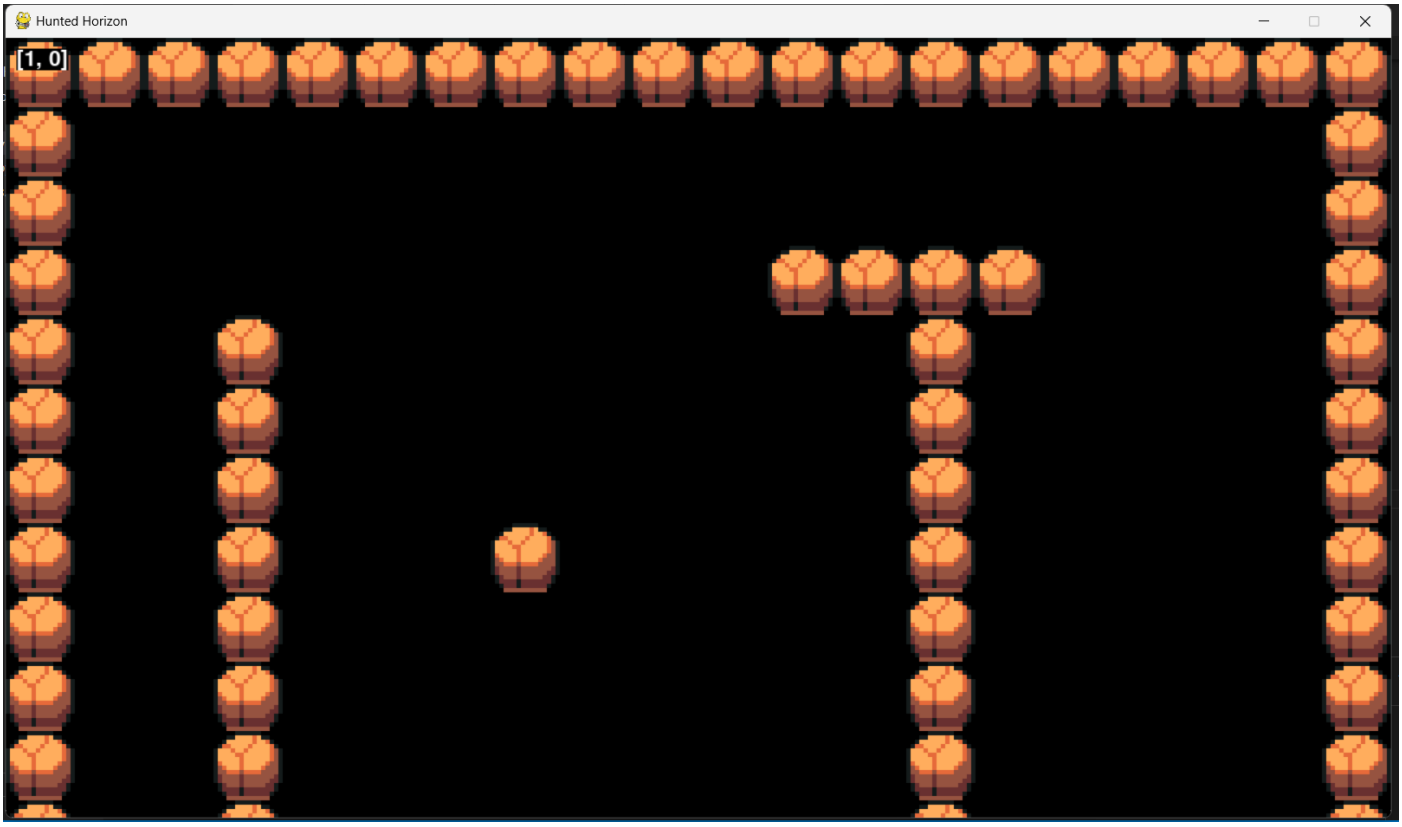
player.py > ...

```
4   class Player(pygame.sprite.Sprite):
5       def __init__(self,pos,groups):
10          # Player movement
11          self.direction = pygame.math.Vector2()  # An array containing the x and y coords of the player
12          self.speed = 5
13
14      def input(self):
15          keys = pygame.key.get_pressed()
16
17          if keys[pygame.K_w]:
18              self.direction.y = -1
19          elif keys[pygame.K_s]:
20              self.direction.y = 1
21          else:
22              self.direction.y = 0
23
24          if keys[pygame.K_a]:
25              self.direction.x = -1
26          elif keys[pygame.K_d]:
27              self.direction.x = 1
28          else:
29              self.direction.x = 0
30
31      def move(self,speed):
32          self.rect.center += self.direction * speed
33
34      def update(self):
35          self.input()
36          self.move(self.speed)
```

Now the player can move, however the player can walk off screen and walk over/under the rocks.


Hunted Horizon
[-1, 0]


Hunted Horizon
[0, 0]



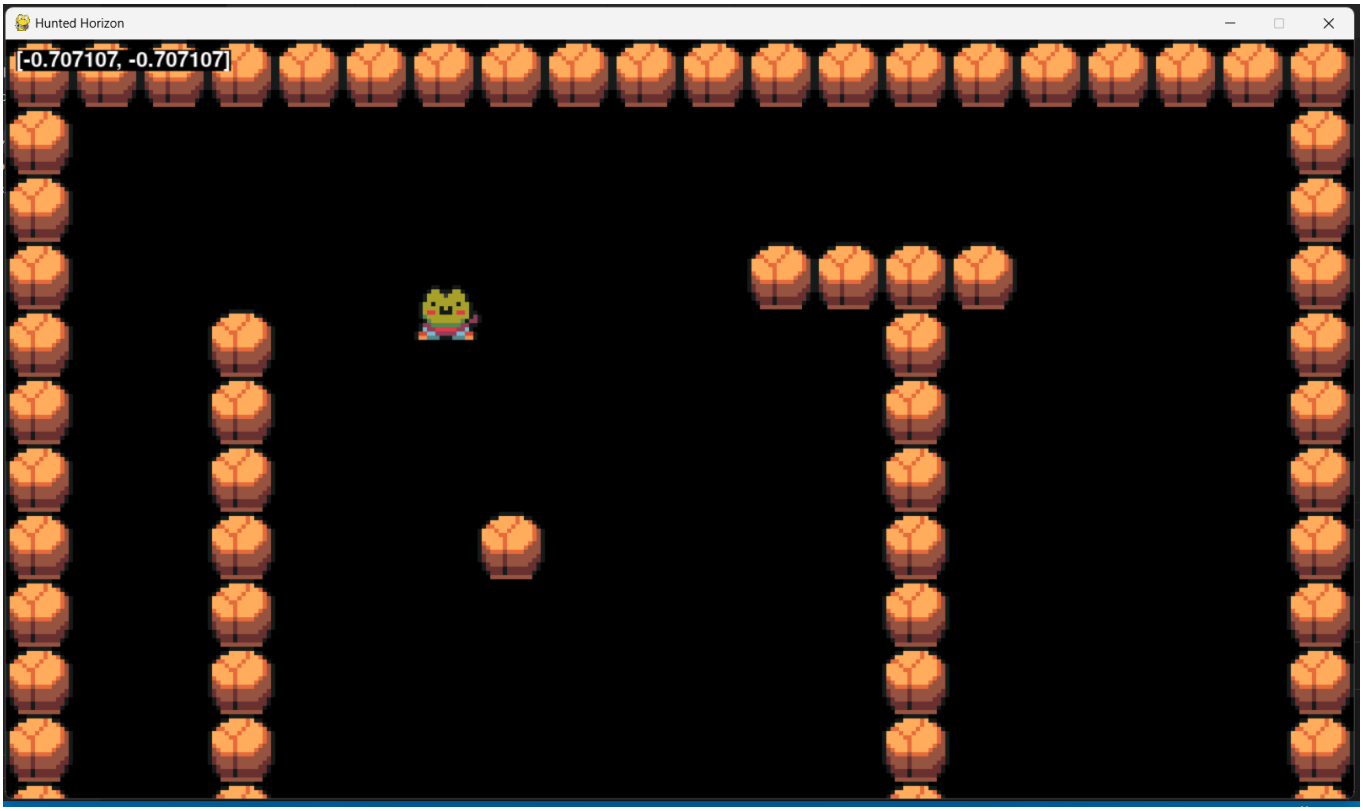
Another problem that can be solved first is that when the player moves only up and down or left and right, the player's speed is as expected but, when the player moves diagonally, the player's speed is way faster than expected. This is because of trigonometry, and it can be fixed by normalising the direction. By normalising the direction, it essentially changes the length of the vector to 1 and means that the player's speed will remain constant as the direction no longer affects the speed in that way.

player.py > ...

```
4   class Player(pygame.sprite.Sprite):
31      def move(self,speed):
32          if self.direction.magnitude() != 0:
33              self.direction = self.direction.normalize()
34
35          self.rect.center += self.direction * speed
```

The direction's magnitude can't be 0 because a vector of 0 can't be normalised and it would result in an error.

Now the player's speed has been fixed but the problem of the player walking offscreen and over/under the rocks remains. The normalised direction can be seen through the debug information when the user moves the player diagonally.



To improve the movement of the player, I needed to add collisions which would stop the player from being able to walk over/under the rocks. Because Pygame can't always clearly identify where the collision is occurring (since two sprites could be colliding on two sides of each other, meaning that Pygame wouldn't know which side the object should be moved to), I needed to split the horizontal movement and collisions from the vertical movement and collisions so the collisions would make logical sense and the player wouldn't look like it's randomly teleporting.

The movement has been split into the horizontal and vertical movement so that collisions can be checked immediately after movement and for the logical error mentioned above to not occur.

To deal with collisions, I created a collision method and split the collisions into horizontal and vertical collisions. This part of the method checks if the direction is horizontal, accesses each sprite within obstacle_sprites, checks if the sprite is colliding with the player and if true, the player will be moved so there's a physical barrier that makes logical sense. This is because logically, if the player is moving to the right, they can only collide with the sprite's left side. The same logic applies to moving left and it helps to simplify where the player will collide with obstacles.

player.py > Player

```
4   class Player(pygame.sprite.Sprite):
33      def move(self,speed):
34          if self.direction.magnitude() != 0:
35              self.direction = self.direction.normalize()  # Makes the direction vector always 1
36
37          self.rect.x += self.direction.x * speed
38          self.collision('horizontal')
39          self.rect.y += self.direction.y * speed
40          self.collision('vertical')
41          #self.rect.center += self.direction * speed
42
43      def collision(self,direction):
44          if direction == 'horizontal':
45              for sprite in self.obstacle_sprites:
46                  if sprite.rect.colliderect(self.rect):  # Checks if obstacle and player are colliding
47                      if self.direction.x > 0:  # Moving right
48                          self.rect.right = sprite.rect.left
49                      if self.direction.x < 0:  # Moving left
50                          self.rect.left = sprite.rect.right
51
52          if direction == 'vertical':
53              for sprite in self.obstacle_sprites:
54                  if sprite.rect.colliderect(self.rect):
55                      if self.direction.y > 0:  # Moving down
56                          self.rect.bottom = sprite.rect.top
57                      if self.direction.y < 0:  # Moving up
58                          self.rect.top = sprite.rect.bottom
```

This does the exact same as the code for horizontal collisions but for vertical collisions. The same logic is applied to find where the collision occurs and moves the player accordingly.

Acceptance Test 1: Check basic WASD movement and the diagonal movement.

Inputs	Outputs	Validity
'W' key	Player moves up and is shown moving up at a constant speed. Any collisions while moving up occur as expected.	Valid
'S' key	Player moves down and is shown moving down at a constant speed. Any collisions while moving down occur as expected.	Valid
'A' key	Player moves left and is shown moving left at a constant speed. Any collisions while moving left occur as expected.	Valid
'D' key	Player moves right and is shown moving right at a constant speed. Any collisions while moving right occur as expected.	Valid
'WD'/'WA'/'SD'/'SA' keys (pressed together)	Player moves diagonally using the two keys and is shown moving diagonally at a constant speed. Any collisions while moving diagonally occur as expected.	Valid
'WS'/'AD' keys (pressed together)	Player moves using the last key pressed and is shown moving at a constant speed. Any collisions while moving occur as expected. It doesn't seem possible for the opposite moving keys to be pressed simultaneously and cancel each other out.	Valid
'WASD' keys (pressed together)	Player moves using the last key pressed and is shown moving at a constant speed. Any collisions while moving occur as expected. It doesn't seem possible for the keys to be pressed simultaneously and cancel each other out.	Valid
Other keyboard keys	There is no change to the game when other keys are pressed.	Valid

The tests are all valid for this first test which means that the movement is currently working as expected for the player. However, the player can still walk off the screen which can be fixed by implementing the camera.

Player Camera

The player camera should follow the player and allow sprites to overlap so the game looks like it has depth. To create this, it needs to inherit the Pygame Group class but use polymorphism to override some of the methods such as the draw method. The class is called YSortCameraGroup because it will create a camera and use the y coordinates of sprites to create depth. It will also be placed within the level file because it will affect the levels but wouldn't need its own file.

To create the camera, you need to offset the sprites' rect and therefore blit (draw) the sprites' image elsewhere using the player's position. This would mean that the part of the map shown will move as the player moves and it will keep the player in the middle of the screen.

level.py > ...

```
7   class Level:
8       def __init__(self):
13          # Sprite group setup
14          self.visible_sprites = YSortCameraGroup()
```

The visible_sprites can use the YSortCameraGroup class instead of the pygame.sprite.Group class since it's a child of Group. For the camera to work, it needs to offset the visible sprites, and this is why YSortCameraGroup is used instead.

level.py > ...

```
 7  class Level:
31      def run(self):
32          # Update + draw game
33          self.visible_sprites.custom_draw(self.player)
34          self.visible_sprites.update()
35
36  class YSortCameraGroup(pygame.sprite.Group):
37      def __init__(self):
38
39          # General setup
40          super().__init__()
41          self.display_surface = pygame.display.get_surface()
42          self.half_width = self.display_surface.get_size()[0] // 2
43          self.half_height = self.display_surface.get_size()[1] // 2
44          self.offset = pygame.math.Vector2()
45
46      def custom_draw(self,player):
47              # Getting offset
48              self.offset.x = player.rect.centerx - self.half_width
49              self.offset.y = player.rect.centery - self.half_height
50
51
52              for sprite in self.sprites():
53                  offset_pos = sprite.rect.topleft - self.offset
54                  self.display_surface.blit(sprite.image,offset_pos)
```

The draw method is replaced by the custom_draw method and no longer needs the display surface to be passed into the method. This is since the display surface is defined in the YSortCameraGroup class. However, the player object needs to be passed into the method so that it can access the player's position.

The super initialisation simply initialises all the methods and attributes of its parent class (Group). The display surface is defined so it can be accessed within the class instead of being passed as an argument from elsewhere. Half of the height and width of the display surface are defined as attributes, and they're rounded down using the //. The offset is used as a placeholder that's altered in the methods which means it's initially [0,0].

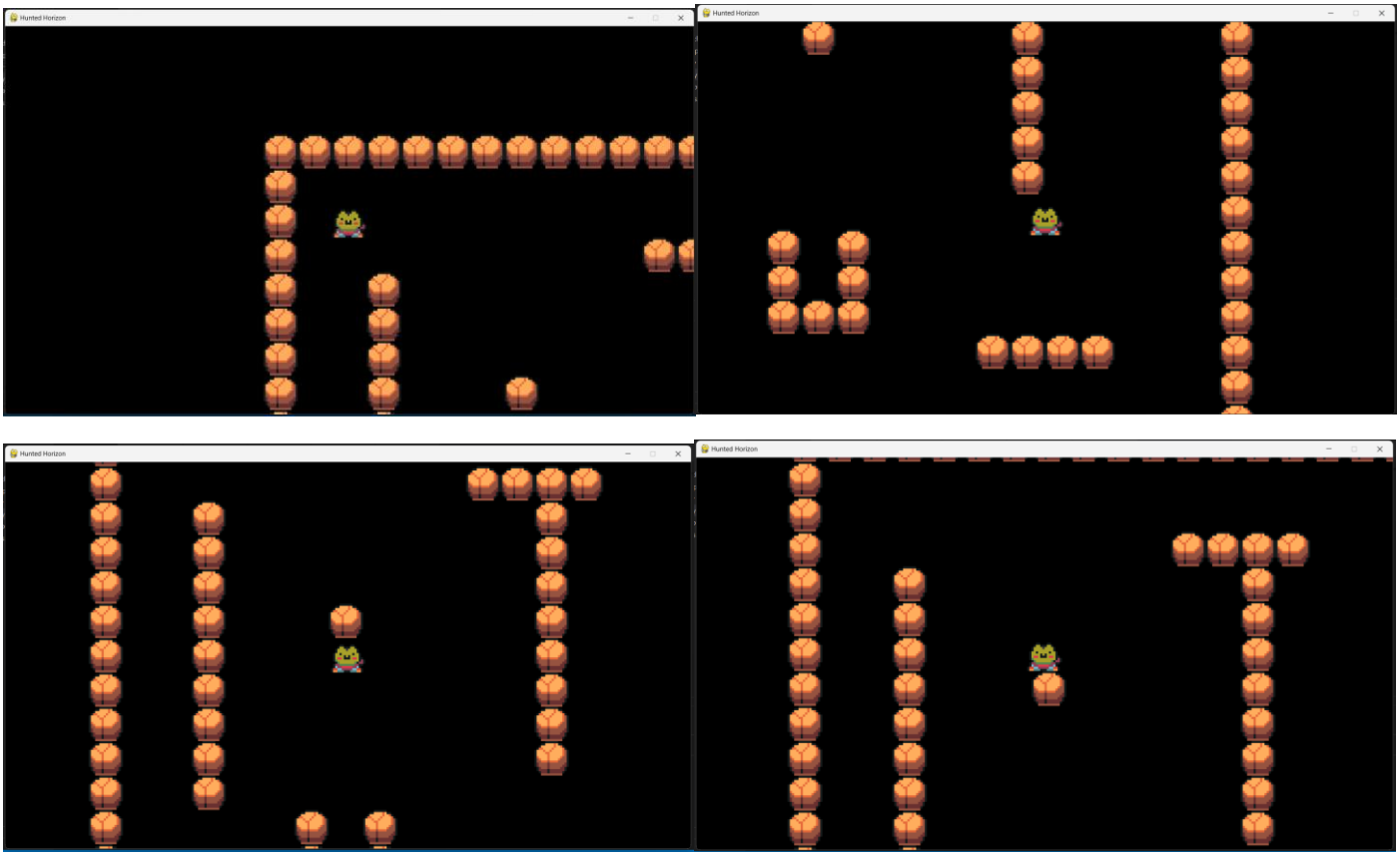
This method is the actual functionality of the camera. First the offset of the camera is gained and set in the offset variable. This is gained by using the player object's coordinates and subtracting the height/width of the screen from that – making the player always appear in the middle of the screen. The method then loops through the sprites (visible_sprites) and resets the position of the sprites using the offset coordinates. The blit method simply draws/blits the sprite image in their new position.

The debug function was also removed in the Level's run method since the movement of the player no longer needs to be checked.

While developing this part of the code, I had a small syntax error where I had accidentally placed the custom_draw method of YSortCameraGroup within its constructor when it should be within the class but outside the constructor. This was easy to fix as I just had to unindent the custom_draw method.

```
  File "c:\Users\gursi\OneDrive - The Holt School\Year 12\Computing\Coursework\Game\code\main.py", line 32, in <module>
    game.run()
  File "c:\Users\gursi\OneDrive - The Holt School\Year 12\Computing\Coursework\Game\code\main.py", line 26, in run
    self.level.run()
  File "c:\Users\gursi\OneDrive - The Holt School\Year 12\Computing\Coursework\Game\code\level.py", line 35, in run
    self.visible_sprites.custom_draw()
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'YSortCameraGroup' object has no attribute 'custom_draw'
PS C:\Users\gursi\OneDrive - The Holt School\Year 12\Computing\Coursework\Game\code>
```

After implementing this code, the screen follows the player as they move, therefore meaning the camera is working as wanted and the player can no longer walk off the screen. However, there is a lack of depth in the game since the sprites don't overlap – when the player is behind a sprite, the sprite should overlap over the player, making it seem like the player is within a 3D world.



To add depth, first hitboxes should be implemented for collisions instead of using the entire image as a hitbox. The hitboxes would be smaller than the image of the sprite. The layer order of the image and the hitbox needs to also be established so there aren't any logical errors in displaying sprites.

```
tile.py > ...
4   class Tile(pygame.sprite.Sprite):
5       def __init__(self,pos,groups):
8           self.rect = self.image.get_rect(topleft = pos)
9           self.hitbox = self.rect.inflate(0,-10)
```

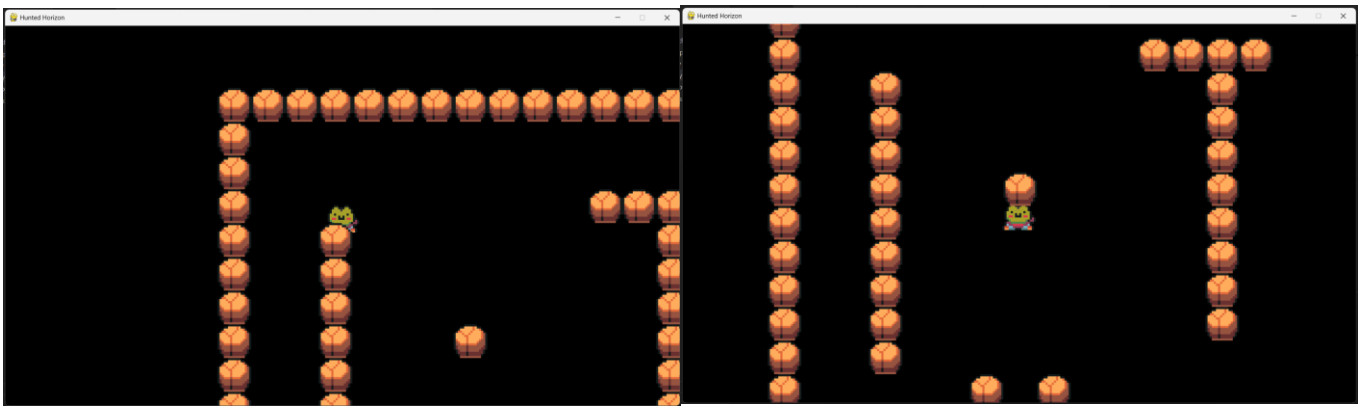
I've defined the hitbox for the sprites (except from the player). The inflate method uses the dimensions of the rect and alters it using what is passed as arguments for the width (x) and height (y) in that order. The width of the player wouldn't need to change since the sides of the player don't need to overlap with other sprites. The -10 means that 5 pixels are subtracted from the top and bottom of the rect.

```
player.py > Player > __init__
4   class Player(pygame.sprite.Sprite):
5       def __init__(self,pos,groups,obstacle_sprites):
9           self.hitbox = self.rect.inflate(0,-26)
```

The hitbox for the player was also defined as an attribute. Again, the width (x) of the rect wasn't altered but the height (y) was. The height was altered more for the player because the player is taller than the other sprites and 13 pixels are subtracted from the top and bottom of the rect.

```
player.py > Player > _init_
4   class Player(pygame.sprite.Sprite):
34      def move(self,speed):
38          self.hitbox.x += self.direction.x * speed
39          self.collision('horizontal')
40          self.hitbox.y += self.direction.y * speed
41          self.collision('vertical')
42          self.rect.center = self.hitbox.center
43
44      def collision(self,direction):
45          if direction == 'horizontal':
46              for sprite in self.obstacle_sprites:
47                  if sprite.hitbox.colliderect(self.hitbox):  # Checks if obstacle and player are colliding
48                      if self.direction.x > 0:  # Moving right
49                          self.hitbox.right = sprite.hitbox.left
50                      if self.direction.x < 0:  # Moving left
51                          self.hitbox.left = sprite.hitbox.right
52
53          if direction == 'vertical':
54              for sprite in self.obstacle_sprites:
55                  if sprite.hitbox.colliderect(self.hitbox):
56                      if self.direction.y > 0:  # Moving down
57                          self.hitbox.bottom = sprite.hitbox.top
58                      if self.direction.y < 0:  # Moving up
59                          self.hitbox.top = sprite.hitbox.bottom
```

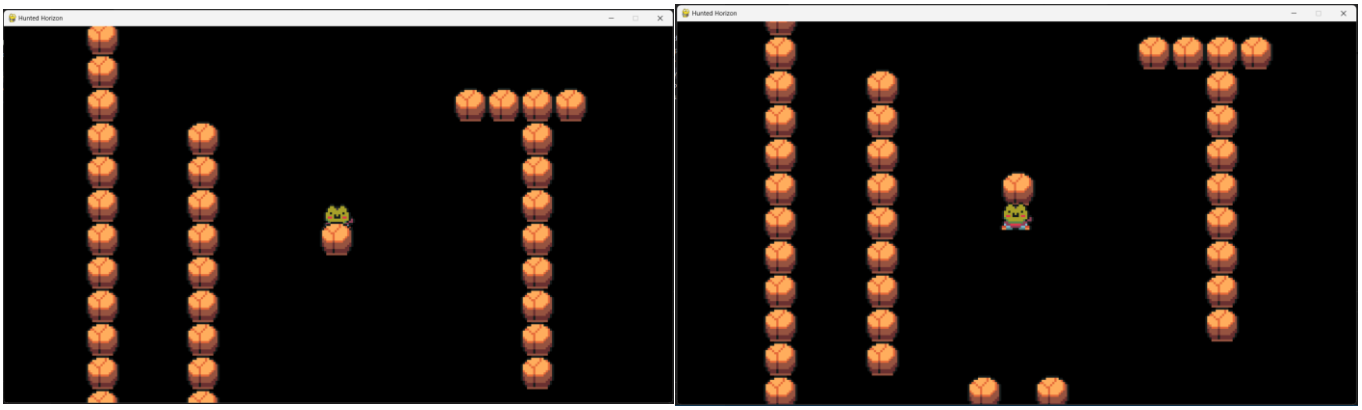
All the Player methods have been altered so that instead of it using the player's rect, it's using the hitbox. This allows for the overlapping to occur and doesn't result in any unintended consequences.



Now, the player can correctly overlap when behind sprites, but it still won't overlap properly when the player is in front of the sprite as the player is now always behind the sprites. This is because of the layering order of the sprites since the rocks at the top do correctly overlap because they are loaded in first and it can be fixed by changing the order. Another minor issue is that the sprite image has some blank space at the top which makes the top of the player not look like its overlapping but, this only occurs when the player is facing forward and will not seem weird once the player animations have been added.

```
level.py > ...
36  class YSortCameraGroup(pygame.sprite.Group):
46      def custom_draw(self,player):
47              # Getting offset
48              self.offset.x = player.rect.centerx - self.half_width
49              self.offset.y = player.rect.centery - self.half_height
50
51
52              for sprite in sorted(self.sprites(),key=lambda sprite: sprite.rect.centery):
53                  offset_pos = sprite.rect.topleft - self.offset
54                  self.display_surface.blit(sprite.image,offset_pos)
```

Instead of drawing the sprites by going through each sprite within visible_sprites, I have rewritten the code to go through the sorted list of visible_sprites. The sorted function takes in the list and then sorts it by the key which in this case is the y-position of the sprites. Lambda is a function that's used to return the result of an expression so the code would return the y-position of the sprite. This code line sorts the sprites by their y-position and logically fixes the overlapping issue.



Now the player camera is working as expected and there seems to be no other errors caused by adding the camera.

Acceptance Test 2: Check player camera.

Inputs	Outputs	Validity
'W' key	Player moves up but remains in centre of the screen, the camera following the player.	Valid
'A' key	Player moves left but remains in centre of the screen, the camera following the player.	Valid
'S' key	Player moves down but remains in centre of the screen, the camera following the player.	Valid

‘D’ key	Player moves right but remains in centre of the screen, the camera following the player.	Valid
Any combination of ‘WASD’ keys	Player moves in the correct direction according to which ‘WASD’ keys are pressed but remains in centre of the screen, the camera following the player.	Valid
Top sprite collision	Player sprite is under the top of the other sprite for a set number of pixels.	Valid
Bottom sprite collision	Player sprite is above the bottom of the other sprite for a set number of pixels.	Valid
Side sprite collision	Player collides with the sides of other sprites, and nothing has been changed since the final implementation of the basic movement.	Valid
Other keyboard keys	There is no change to the game when other keys are pressed.	Valid

Loading the Main Map and Barriers

I have already created the main map on Tiled and showed it previously within the documentation. To load the main map, I first had to export the floor of the main map as an image and place it on the screen within the custom_draw method of the YSortCameraGroup class and ensure that it’s always under the other sprites. To reduce complexity, the code will currently be made to only implement the main map and will be later altered when implementing the other maps.

level.py > ...

```
36  class YSortCameraGroup(pygame.sprite.Group):
37      def __init__(self):
46          # Floor Creation
47          self.floor_surf = pygame.image.load('../graphics/tilemap/mainmap_ground.png').convert()
48          self.floor_rect = self.floor_surf.get_rect(topleft=(0,0))
49
50      def custom_draw(self,player):
51              # Getting offset
52              self.offset.x = player.rect.centerx - self.half_width
53              self.offset.y = player.rect.centery - self.half_height
54
55              # Drawing the Floor
56              floor_offset_pos = self.floor_rect.topleft - self.offset
57              self.display_surface.blit(self.floor_surf,floor_offset_pos)
58
59              # Drawing the Sprites
60              for sprite in sorted(self.sprites(),key=lambda sprite: sprite.rect.centery):
61                  offset_pos = sprite.rect.topleft - self.offset
62                  self.display_surface.blit(sprite.image,offset_pos)
```

This sets the map to be behind the rest of the sprites and objects that are drawn afterwards. The map doesn’t require convert_alpha since it already would already load in quickly.

The same logic of adding the offset to the sprite positions for the camera is applied to the map floor. Therefore, ensuring that the camera still works properly.

This is what it looks like after running the code. The camera still runs as expected and there are no issues except from the image size of the map being too small. This was due to a mistake I had made when creating the main map on Tiled since I didn’t put the correct tile width and height. This meant that I had to change the sizes of the assets, and it should have fixed itself. Although, the main map had incorrectly saved

and I was required to recreate some parts of the map – meaning that the map doesn't look entirely the same but still close to the original. After completing this and then running the code again, the issue has been solved.

Before implementing the objects of the map, I needed to remove the test rocks and place the player elsewhere on the screen. This requires me to edit the create_map method within the Level class. Now the player can move around freely on the screen but there are no collisions.

level.py > YSortCameraGroup > __init__

```
7   class Level:
20      def create_map(self):
21          #for row_index,row in enumerate(WORLD_MAP):  # Enumerate returns row index number for each row as row_index
22          #    for col_index,col in enumerate(row):
23          #        x = col_index * TILESIZE
24          #        y = row_index * TILESIZE
25          #        if col == 'x':
26          #            Tile((x,y),[self.visible_sprites, self.obstacle_sprites])
27          #        if col == 'p':
28          #            self.player = Player((x,y),[self.visible_sprites],self.obstacle_sprites)
29          self.player = Player((1500,3280),[self.visible_sprites],self.obstacle_sprites)
```

Next, I would need to alter the Tile class so all the objects can be displayed onto the map. This means that it should work with objects of varying size and invisible objects which will be the wall blocks. To access the objects however, the map needs to be exported as a csv file for each of the layers and then converted into a matrix for Python to understand – requiring another function to be created to do this (import_csv_layout). Tiled can only export tile layers which also meant that I had to manually map the items within object layers to a csv file.

The sprite_type is being defined here using the argument and the image of the Tile object is now its surface.

```python
# tile.py > ...
import pygame
from settings import *

class Tile(pygame.sprite.Sprite):
    def __init__(self,pos,groups,sprite_type,surface=pygame.Surface((TILESIZE,TILESIZE))):
        super().__init__(groups)
        self.sprite_type = sprite_type
        self.image = surface
        self.rect = self.image.get_rect(topleft = pos)
        self.hitbox = self.rect.inflate(0,-10)
```

I added two more parameters for the Tile class: one for determining the sprite type (e.g. 'enemy' or 'invisible') and another for the surface of the object. When creating an object without a surface argument, the default surface is used. The surface argument ensures that the tile is properly drawn.

The import_csv_layout function was placed in another Python file named 'support' because its only purpose is to convert the csv file into a matrix.

The reader function is imported from the csv Python module so that the csv file can be read.

The only parameter required is the path of the csv file so it can be accessed. The terrain_map is a variable that will store the converted map.

```python
# support.py > ...
from csv import reader

def import_csv_layout(path):
    terrain_map = []
    with open(path) as level_map:
        layout = reader(level_map,delimiter=',')
        for row in layout:
            terrain_map.append(list(row))
        return terrain_map
```

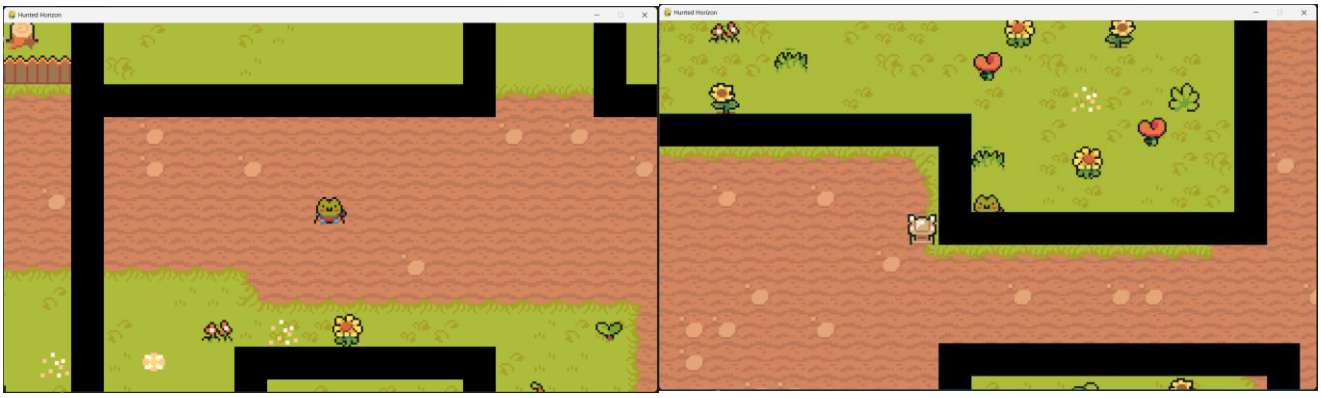
A with statement automatically closes the file when the following indented code has finished. The open function is an in-built Python function that opens the file so it can be handled within Python. The reader function takes the file as an argument and takes a delimiter which is just what the values will be separated by. Then each row of the map is added as a list to terrain_map until there are no rows left. The terrain_map, now storing the 2D array of the map, is then returned.

This is what the terrain_map would look like: the -1 representing an empty space and any other number representing the ID of the Tiled tile. The rows can be seen as individual arrays within the overall array.

The different layers of the map are placed within this dictionary so they can each be easily accessed.

```python
# level.py > Level
class Level:
    def create_map(self):
        layouts = {
            'boundary': import_csv_layout('../map/mainmap/mainmap_Wallblocks.csv')
        }
        for style,layout in layouts.items():  # Items function retrieves the key and value pair
            for row_index,row in enumerate(layout):  # Enumerate returns row index number for each row as row_index
                for col_index,col in enumerate(row):
                    if col != '-1':
                        x = col_index * TILESIZE
                        y = row_index * TILESIZE
                        if style == 'boundary':
                            Tile((x,y),[self.visible_sprites,self.obstacle_sprites],'invisible')

        #        if col == 'x':
        #            Tile((x,y),[self.visible_sprites, self.obstacle_sprites])
        #        if col == 'p':
        #            self.player = Player((x,y),[self.visible_sprites],self.obstacle_sprites)
        self.player = Player((1500,3280),[self.visible_sprites],self.obstacle_sprites)
```

A two variable for loop is used to define the key and value for each pair within layouts. Then another two for loops are used to go through each value of the map, like before. The value is checked to make sure it isn't '-1' since that represents an empty tile. The style of the layer is checked to place the right tile type. Currently the boundary tiles would be visible since they've been placed in both groups, but this is just to see that the tiles are correctly positioned and will be removed from the code.



This is what the map looks like while the boundary tiles still have self.visible_sprites as one of their groups. The tiles are in the correct positions and the collisions and overlapping still works as expected. When the self.visible_sprites is taken out of the group, it looks like the GIF shown below.

GIF (1) Checking barriers

Now to create the objects, I must further add to the create_map method as well as create another function that would help return the images of a file as a list.

The walk function is imported from the Python os module. It accesses a file's directory path, directory names and file names – taking the path of the file as an argument.

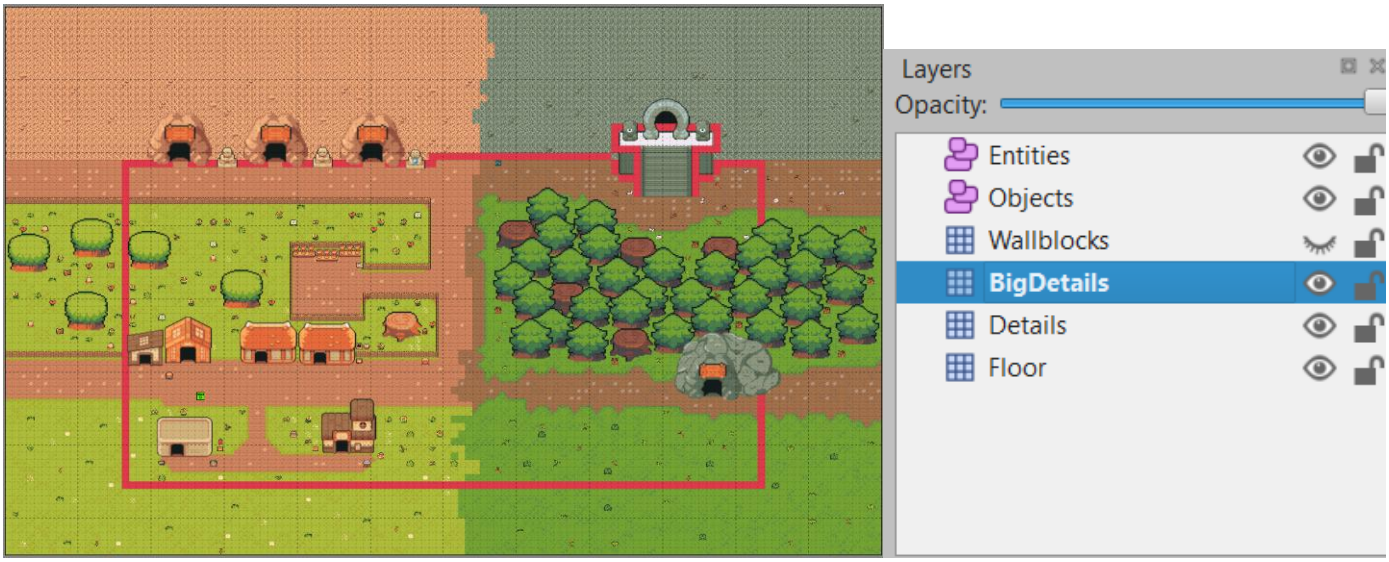
The import_folder function takes the path of the file as its only argument. surface_list is an empty list that will contain the surfaces of all the files within the folder. A three variable for loop is used when the walk function is called but we only need the third variable from walk(), therefore I have kept the other variables as underscores since they aren't needed.

support.py > ...

```
1   import pygame
2   from csv import reader
3   from os import walk
4
5   def import_csv_layout(path):
6       terrain_map = []
7       with open(path) as level_map:
8           layout = reader(level_map,delimiter=',')
9           for row in layout:
10              terrain_map.append(list(row))
11          return terrain_map
12
13  def import_folder(path):
14      surface_list = []
15      for _,__,img_files in walk(path):
16          for image in img_files:
17              full_path = path + '/' + image
18              image_surf = pygame.image.load(full_path).convert_alpha()
19              surface_list.append(image_surf)
20      return surface_list
```

The for loop goes through all the files within the folder and finds their path so that their surface can be made. That surface is added to the surface_list and once the for loop has ended, the surface_list is returned so it can be used when creating the objects.

During the creation of the map, I altered the tiled map slightly to help the map look better. I removed unnecessary barriers and created a separate layer for tiles that would require collisions but are within the tile sets and need a tile layer rather than an object layer. This would help the game look better since otherwise the player could walk over them or it would require me to place invisible barriers over them which would look clunky. That's another reason why I removed some of the barriers like on the fence and instead made the fence like an object that can be collided with.



A 'detail' and 'object' style are added to the layouts, and they simply store the map layout of the layer like it was used for the 'boundary' style.

Another dictionary is defined as graphics to store the surface_list of the folders. The surface_list would contain the surfaces for each of the images within the folder chosen.

Like what was done when implementing the barriers, the two if statements check the style (key) of the current layout. Then the layout map is used to find any values within the map that aren't -1 and create the tile for that object which then appears when the game is run. Both the if statements for details and objects are the same except for using their specific surfaces from their own folders. The surface of the object is found using the surface_list (value pair) within the graphics dictionary and the col (the value inside the current position). The surface is used since the images are varied in size and using the default surface would cause a logical error. When instantiating an object, they also both use the visible_sprites group so the object can be seen on the map.

level.py > Level > create_map

```
8   class Level:
21      def create_map(self):
22          layouts = {
23              'boundary': import_csv_layout('../map/mainmap/mainmap_Wallblocks.csv'),
24              'detail': import_csv_layout('../map/mainmap/mainmap_BigDetails.csv'),
25              'object': import_csv_layout('../map/mainmap/mainmap_Objects.csv')
26          }
27          graphics = {
28              'details': import_folder('../graphics/details'),
29              'objects': import_folder('../graphics/objects')
30          }
31
32          for style,layout in layouts.items():  # Items function retrieves the key and value pair
33              for row_index,row in enumerate(layout):  # Enumerate returns row index number for each row as row_index
34                  for col_index,col in enumerate(row):
35                      if col != '-1':
36                          x = col_index * TILESIZE
37                          y = row_index * TILESIZE
38                          if style == 'boundary':
39                              Tile((x,y),[self.obstacle_sprites],'invisible')
40                          if style == 'detail':
41                              surf = graphics['details'][int(col)]
42                              Tile((x,y),[self.visible_sprites,self.obstacle_sprites],'detail',surf)
43                          if style == 'object':
44                              surf = graphics['objects'][int(col)]
45                              Tile((x,y),[self.visible_sprites,self.obstacle_sprites],'object',surf)
```

The Tile class also needed altering so that the objects' rect would be slightly repositioned in the middle of the surface instead of the top left to allow for better overlapping.

tile.py > ...

```
4   class Tile(pygame.sprite.Sprite):
5       def __init__(self,pos,groups,sprite_type,surface=pygame.Surface((TILESIZE,TILESIZE))):
9           if sprite_type == 'object':
10              self.rect = self.image.get_rect(topleft=(pos[0],pos[1]-TILESIZE))
11          else:
12              self.rect = self.image.get_rect(topleft = pos)
```

With this implementation, the test map's data and code can be removed from the settings and level file: the WORLD_MAP 2D array and checking if col is equal to 'x' or 'p'. This is now what the map looks like:

GIF (2) Showing mainmap

I had encountered a logical error while coding this section where one of the objects wasn't appearing with the correct image and surface. I had found out that this was because the folder of object images was meant to be named sequentially but I had missed out two numbers, causing the error to occur. This was easily fixed by renaming the images so they are in sequential order and then adjusting the objects csv file to match the new name.

```
  File "c:\Users\gursi\OneDrive - The Holt School\Year 12\Computing\Coursework\Game\code\level.py", line 41, in create_map
    surf = graphics['details'][int(col)]
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^
IndexError: list index out of range
```

I also had a simple syntax error within my code since I had forgotten a slash when writing the path for the details folder, causing the list index to be out of range since the value for the key 'details' then became an empty list instead of a list containing the surfaces of the images within the details folder.

Acceptance Test 3: Check the main map and the objects within the map.

Inputs	Outputs	Validity
Main map floor image	Main map floor image is displayed behind any other objects as the correct size.	True
Floor offset	The main map floor is correctly given an offset which allows the player camera to continue working as expected.	True
Sprite surface	Sprites have their correct surface (including their image) according to the map layer.	True
Sprite position	Sprites are shown to be in their correct position according to the map layer and what is expected.	True
Sprite offset	The sprites are correctly given an offset which allows the player camera to continue working as expected.	True
Sprite rect	Sprites have their rect within the expected place and are adjusted if needed which means that they also overlap properly.	True
Sprite side collisions	Player sprite correctly collides with the side of other sprites.	True
Sprite top/bottom collisions	Player sprite correctly collides with the top and bottom of other sprites, overlapping with them slightly for depth as coded.	True

Acceptance Test 4: Check the main map's barriers.

Inputs	Outputs	Validity
Barrier position	Barrier tiles are within their correct position according to the map layer.	True
Barrier collisions	Player correctly collides with any of the barrier's edges, stopping the player from advancing.	True
Barrier surface	Barrier tiles have an invisible surface to the user.	True

Jumping

To implement jumping, I needed to give the player character some more attributes, create a method to apply the gravity and for jumping, and add the spacebar key to the list of keyboard keys that are usable – making sure that the spacebar also correctly performs the jump.

The gravity attribute is used to alter the vertical speed so that it looks like there is gravity when the player jumps. The jump_force attribute is a constant for indicating how high the player can jump. The vertical_speed is used to alter the player's position, so it looks like the player is jumping. The vertical_speed remains zero and only changes when the player is jumping.

player.py > Player > __init__

```
4   class Player(pygame.sprite.Sprite):
5       def __init__(self,pos,groups,obstacle_sprites):
15          self.current_ground = self.hitbox.y
16          self.gravity = 0.8
17          self.jump_force = -15
18          self.vertical_speed = 0
19          self.jumping = False
20          self.down = False
```

This attribute stores the ground level of the player so that they can return down to that level when jumping. Although since there is no actual 3D aspect, the attribute takes the y position of the player instead.

The jumping attribute is simply to show whether the player is jumping or not (it would be True when they are and False when they aren't). The down attribute is similarly to show whether the player is moving down or not so the jumping can be separated from downward movement.

The current_ground attribute is updated whenever the player moves so that the player can jump from anywhere. It's only updated when the player moves vertically since it only stores the y position of the player.

player.py > ...

```
4   class Player(pygame.sprite.Sprite):
24      def input(self):
25          keys = pygame.key.get_pressed()
26
27          if keys[pygame.K_w]:
28              self.direction.y = -1
29              self.current_ground = self.hitbox.y
30          elif keys[pygame.K_s]:
31              self.direction.y = 1
32              self.current_ground = self.hitbox.y
33              self.down = True
34          else:
35              self.direction.y = 0
36              self.down = False
37
38          if keys[pygame.K_a]:
39              self.direction.x = -1
40          elif keys[pygame.K_d]:
41              self.direction.x = 1
42          else:
43              self.direction.x = 0
44
45          if keys[pygame.K_SPACE]:
46              self.jump()
```

The down attribute is set to True if the player is moving down and False if they aren't.

This checks if the spacebar was pressed. If it was, the jump method runs.

player.py > ...

```
4   class Player(pygame.sprite.Sprite):
77      def jump(self):
78          if not self.jumping:
79              self.jumping = True
80              self.vertical_speed = self.jump_force
```

The class method is a simple method that simply sets whether the player is jumping (the jumping attribute) to True and sets the vertical_speed attribute to the jump_force constant (-15). The if statement is to ensure that the values like vertical_speed don't reset if the player is already within a jumping state – this would cause a logical error and make the player look like they are teleporting.

This if statement checks if the player is jumping and moving downwards. If the player is then the vertical_speed that would be -15 is increased by the gravity (0.8) every frame. The player is moved using this vertical_speed by increasing the player's y coordinate since the player is moving down.

This if statement is like the other one within this method but this is for any jumping where the player isn't moving down at the same time. The logic is the same except the player's y coordinates are decreasing from the vertical_speed and the player stops falling when their y coordinate is greater or equal to the current_ground. This is simply because the player is moving in an upward direction instead.

player.py > ...

```
4   class Player(pygame.sprite.Sprite):
82      def apply_gravity(self):
83          if self.jumping and self.down:
84              self.vertical_speed += self.gravity
85              self.hitbox.y -= self.vertical_speed
86
87              if self.hitbox.y <= self.current_ground:
88                  self.hitbox.y = self.current_ground
89                  self.jumping = False
90                  self.vertical_speed = 0
91
92          elif self.jumping:
93              self.vertical_speed += self.gravity
94              self.hitbox.y += self.vertical_speed
95
96              if self.hitbox.y >= self.current_ground:
97                  self.hitbox.y = self.current_ground
98                  self.jumping = False
99                  self.vertical_speed = 0
100
101
102     def update(self):
103         self.input()
104         self.move(self.speed)
105         self.apply_gravity()
```

This if statement is to stop the player from falling infinitely. This is because once the player's y coordinate is less than or equal to the current_ground (the y value where the player jumped from), jumping is no longer True, and the vertical_speed is returned to 0. Once this happens, the gravity is no longer applied, and the player doesn't continue falling.

The apply_gravity method is placed in the update method since it should be checked for at every frame so there's no input lag when the player presses the spacebar to jump.

GIF (3) Jumping

This is what the jumping looks like in game. Jumping in all directions seems to be fine and the basic WASD movement isn't being affected. The only problem is that the player can 'clip' through objects, but this

won’t pose much of a problem in the rest of the game since the levels won’t have many objects and the jumping will be disabled when on the main map.

Acceptance Test 5: Check player jumping movement.

Inputs	Outputs	Validity
‘Spacebar’ key	The player jumps once up and falls back to the position where they jumped from.	True
Holding ‘Spacebar’ key	The player performs a single jump and if ‘Spacebar’ is still being held once the player returns down, the player jumps again.	True
‘Spacebar’ and ‘W’ keys	The player jumps upwards and falls to a position higher than they were before.	True
‘Spacebar’ and ‘S’ keys	The player jumps downwards and falls to a position lower than they were before.	True
‘Spacebar’, ‘W’ and ‘A’/’D’ keys	The player jumps upwards diagonally (depending on if they were moving left or right) and falls to a position higher than they were before and diagonal to where they jumped from.	True
‘Spacebar’, ‘S’ and ‘A’/’D’ keys	The player jumps downwards diagonally (depending on if they were moving left or right) and falls to a position lower than they were before and diagonal to where they jumped from.	True
‘Spacebar’ and ‘A’ keys	The player jumps while moving left and their y position remains the same when the player falls back down.	True
‘Spacebar’ and ‘D’ keys	The player jumps while moving right and their y position remains the same when the player falls back down.	True
Holding ‘Spacebar’ and movement (WASD) keys	The player performs a single jump in the direction and if ‘Spacebar’ is still being held once the player returns down, the player jumps again in the movement direction.	True
Other keyboard keys	There’s no change to the game.	True

Action Placeholders

I need to implement placeholders for attacking, blocking and interacting just so they can be developed later in the game. This is very easy to do and is the same as implementing the basic movement and jumping. However, the keys for now will just print a string within the Python console – since they are placeholders.

This code is placed in the input method along with the other movement keys. It simply prints the string when the player presses those keys. Once other features are implemented, I will come back to this code.

Acceptance Test 6: Check that player can attack, block and interact using placeholders.

Inputs	Outputs	Validity
'J' key	The string 'attack' is printed in Python's console for however long the key is pressed.	True
'K' key	The string 'block' is printed in Python's console for however long the key is pressed.	True
'L' key	The string 'interact' is printed in Python's console for however long the key is pressed.	True
Other keyboard keys	There's no change to the game.	True

Player Animations

Currently, the player does move and jump but there are no animations that go along with the movement. This makes the game look odd and makes it harder for the player's actions and movement to be clearly conveyed to the user. Therefore, using animations can help the game look better and be easier to understand.

To do this, there must be an attribute in Player that keeps track of the player's status which can then be used to show the correct animation. Before implementing the animations, I need to set cooldowns for some of the actions like attacking.

player.py > Player > __init__

```
4   class Player(pygame.sprite.Sprite):
5       def __init__(self,pos,groups,obstacle_sprites):
22          self.attacking = False
23          self.attack_cooldown = 400
24          self.attack_time = None
```

These attributes were added to the player. The attacking attribute is to indicate whether the player is currently attacking or not. The attack_cooldown is the number of milliseconds that the player will have to wait before attacking again. The attack_time would store the time at which the player pressed attack. The attacking and attack_time are set to the default values: False and None.

player.py > Player > __init__

```
4   class Player(pygame.sprite.Sprite):
32      def input(self):
56          if keys[pygame.K_j] and not(self.attacking):
57              self.attacking = True
58              self.attack_time = pygame.time.get_ticks()
59              print('attack')
```

If the player is already attacking, they shouldn't be able to continuously attack – this is why it checks that the attacking attribute is false. Then it sets attacking as True and the time when the player attacks is stored in the attack_time attribute so the cooldown can be applied later in the code.

player.py > Player > __init__

```
4   class Player(pygame.sprite.Sprite):
98      def cooldowns(self):
99          current_time = pygame.time.get_ticks()
100
101         if self.attacking:
102             if (current_time-self.attack_time) >= self.attack_cooldown:
103                 self.attacking = False
```

A cooldown method is created to set the cooldown for any required actions. The current_time variable gets the current time of the game.

This checks if the player is currently attacking and then checks if the time elapsed since the player has attacked is equal or greater than the attack_cooldown. If it is, then attacking is set to False and the player can attack again. This creates a pause between when the player can attack following a previous attack (a cooldown).

player.py > Player > __init__

```
4   class Player(pygame.sprite.Sprite):
135     def update(self):
136         self.input()
137         self.cooldowns()
138         self.move(self.speed)
139         self.apply_gravity()
```

The cooldowns method is added to the update method since it needs to be checked continuously.

Now there is a cooldown when attacking which means that the player can't infinitely attack. This is seen when running the code and when pressing the attack key 'J', there aren't several attack strings printed at once and there is a pause between when you can attack.

Now I need to import all the player assets for the animation and implement the animations.

```
player.py > Player
  5   class Player(pygame.sprite.Sprite):
  6       def __init__(self,pos,groups,obstacle_sprites):
 12           # Graphic Setup
 13           self.import_player_assets()
 14           self.status = 'down'
 15           self.frame_index = 0
 16           self.animation_speed = 0.15
```

The import_player_assets() method simply imports all of the different player animation frames. The status attribute is used to indicate what the player is currently doing so that the correct animation can be played. The frame_index is used to store the current frame of the animation, and the animation_speed is used when changing frame_index.

```
player.py > Player > _init_
  5   class Player(pygame.sprite.Sprite):
 35       def import_player_assets(self):
 36           character_path = '../graphics/player/'
 37           self.animations = {'up':[], 'down':[], 'left':[], 'right':[],
 38               'up_idle':[], 'down_idle':[], 'left_idle':[], 'right_idle':[],
 39               'up_attack':[], 'down_attack':[], 'left_attack':[], 'right_attack':[],
 40               'up_jump':[], 'down_jump':[], 'left_jump':[], 'right_jump':[]}
 41
 42           for animation in self.animations.keys():
 43               full_path = character_path + animation
 44               self.animations[animation] = import_folder(full_path)
```

The character_path variable just contains the file path to the player's assets. The animations attribute is a dictionary to store all the images within the different animation files. The for loop goes through each key in the dictionary and updates their value to store the path of each animation image within the correct folder using the import_folder function that was created before. This only works since the files for the player animation images are ordered to allow this to work.

```
player.py > Player > _init_
  5   class Player(pygame.sprite.Sprite):
 46       def input(self):
 49           if not self.attacking:
 50               if keys[pygame.K_w]:
 51                   self.direction.y = -1
 52                   self.current_ground = self.hitbox.y
 53                   self.status = 'up'
 54               elif keys[pygame.K_s]:
 55                   self.direction.y = 1
 56                   self.current_ground = self.hitbox.y
 57                   self.down = True
 58                   self.status = 'down'
 59               else:
 60                   self.direction.y = 0
 61                   self.down = False
 62
 63               if keys[pygame.K_a]:
 64                   self.direction.x = -1
 65                   self.status = 'left'
 66               elif keys[pygame.K_d]:
 67                   self.direction.x = 1
 68                   self.status = 'right'
 69               else:
 70                   self.direction.x = 0
```

The status is updated when moving with the WASD keys to indicate which way the player is facing.

This method is for changing the status of the player. The if statement checks if the player isn't moving and doesn't have 'idle', 'attack' or 'jump' already in their status to add _idle to the end of their status.

This is to return the status back to the basic status by removing 'attack' from the status.

```
player.py > Player
  5   class Player(pygame.sprite.Sprite):
 85       def get_status(self):
 86           # Idle status
 87           if self.direction.x == 0 and self.direction.y == 0:
 88               if not 'idle' in self.status and not 'attack' in self.status and not 'jump' in self.status:
 89                   self.status = self.status + '_idle'
 90
 91           if self.attacking:
 92               self.direction.x = 0   # Stops player from moving while attacking
 93               self.direction.y = 0   # ^
 94               if not 'attack' in self.status:
 95                   if 'idle' in self.status:
 96                       self.status = self.status.replace('_idle','_attack')
 97                   elif 'jump' in self.status:
 98                       self.status = self.status.replace('_jump','_attack')
 99                   else:
100                       self.status = self.status + '_attack'
101           else:
102               if 'attack' in self.status:
103                   self.status = self.status.replace('_attack','')
```

This checks if the player is attacking and then stops them from moving while attacking. Then if 'attack' isn't already within their status, it checks if either 'idle' or 'jump' are in the status and then replaces them. The last else is for when the player is simply moving.

```
player.py > Player
  5   class Player(pygame.sprite.Sprite):
 85       def get_status(self):
105           if self.jumping:
106               if not 'jump' in self.status:
107                   if 'idle' in self.status:
108                       self.status = self.status.replace('_idle','_jump')
109                   elif 'attack' in self.status:
110                       self.status = self.status.replace('_attack','_jump')
111                   else:
112                       self.status = self.status + '_jump'
113           else:
114               if 'jump' in self.status:
115                   self.status = self.status.replace('_jump','')
```

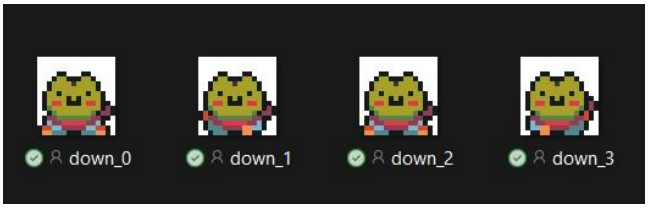
This code is the same as the code for the attacking except that it's for the jumping instead.

The animate method animates the character by going through various animation frames. The animation variable stores the current animation graphics for the player based on their status.

```
player.py > Player
  5   class Player(pygame.sprite.Sprite):
177       def animate(self):
178           animation = self.animations[self.status]
179
180           # Looping over frame index
181           self.frame_index += self.animation_speed
182           if self.frame_index >= len(animation):
183               self.frame_index = 0
184
185           self.image = animation[int(self.frame_index)]
186           self.rect = self.image.get_rect(center = self.hitbox.center)
187
188       def update(self):
189           self.input()
190           self.cooldowns()
191           self.get_status()
192           self.animate()
193           self.move(self.speed)
194           self.apply_gravity()
```

This slowly increases the frame until it reaches the end of the animation. The frame is then set back to 0 so the animation can start again. The frame_index is incremented by the animation_speed so that the animation frame changes after a while. The player's image and rect is then updated for the player to visibly change. The rect needs changing because some of the animation images may be smaller or bigger than others.

Both get_status and animate methods are added to the update method since they need to be run for every frame of the game.



Name	Status	Date modified	Type	Size
down		30/10/2024 21:08	File folder	
down_attack		30/10/2024 21:08	File folder	
down_idle		30/10/2024 21:08	File folder	
down_jump		30/10/2024 21:08	File folder	
left		30/10/2024 21:08	File folder	
left_attack		30/10/2024 21:08	File folder	
left_idle		30/10/2024 21:08	File folder	
left_jump		30/10/2024 21:08	File folder	
right		30/10/2024 21:08	File folder	
right_attack		30/10/2024 21:08	File folder	
right_idle		30/10/2024 21:08	File folder	
right_jump		30/10/2024 21:08	File folder	

This is what the folder setup looks like that helps to understand why the code works. If the folder wasn't set up like this, then there would be syntax or logical errors: e.g. not found errors.

While developing the animations, there were only two main errors. One was a syntax error that would cause the game to crash: this is because when some actions were done either together or one after another, it would cause the last action to be added to the end of the status. Because there is no key for that status, it causes a key error. This was fixed by ensuring that the code checks to make sure there isn't another action already in the status before adding to the status. I also made it that the player couldn't attack while jumping.

```
  File "c:\Users\gursi\OneDrive - The Holt School\Year 12\Computing\Coursework\Game\code\player.py", line 188, in update
    self.animate()
  File "c:\Users\gursi\OneDrive - The Holt School\Year 12\Computing\Coursework\Game\code\player.py", line 174, in animate
    animation = self.animations[self.status]
                ~~~~~~~~~~~~~~~^^^^^^^^^^^^^
KeyError: 'right_jump_attack'
```

Another error I encountered was a logical error where if the player attacked or jumped and didn't make any action after that, they would remain in the attack or jump frame rather than returning to the idle frames. This was easily fixed by adding the last else when the player is jumping or attacking within the get_status method.

Now the player movement looks like this:

GIF (4) Player animation

Acceptance Test 7: Check player animations are working for all current movement.

Inputs	Outputs	Validity
No input	Player idle frame is displayed in the correct direction.	True
'WASD' keys	Player walking animation is displayed in the correct direction.	True
'J' key	Player attack animation is displayed in the correct direction and then returns to idle frames if no other input occurs.	True
'Spacebar' key	Player jump animation is displayed for the duration of the player's jump.	True
'K' or 'L' keys	Nothing occurs other than the output in the Python console.	True
Other keyboard keys	Nothing occurs.	True

Weapon

To create the weapon, I needed to make a weapon class and two new methods for creating and destroying the attack. I created a new Python file for the Weapon class, and I put the attack creation/destruction methods in the Level class. I also added the weapon data to the settings so it can be accessed in the player class.

settings.py > ...

```python
# Game setup
WIDTH = 1280
HEIGHT = 720
FPS = 60
TILESIZE = 64

weapon_data = {
    'sword': {'cooldown':100, 'damage':1, 'graphic':'../graphics/weapons/sword/full.png'}
}
```

weapon.py > ...

```python
import pygame

class Weapon(pygame.sprite.Sprite):
    def __init__(self,player,groups):
        super().__init__(groups)
        direction = player.status.split('_')[0]

        # Graphics
        full_path = f'../graphics/weapons/sword/{direction}.png'
        self.image = pygame.image.load(full_path).convert_alpha()

        # Placement
        if direction == 'right':
            self.rect = self.image.get_rect(midleft=(player.rect.midright + pygame.math.Vector2(0,16)))
        elif direction == 'left':
            self.rect = self.image.get_rect(midright=(player.rect.midleft + pygame.math.Vector2(0,16)))
        elif direction == 'down':
            self.rect = self.image.get_rect(midtop=(player.rect.midbottom + pygame.math.Vector2(-10,0)))
        else:
            self.rect = self.image.get_rect(midbottom=(player.rect.midtop + pygame.math.Vector2(-10,0)))
```

The Weapon class inherits Pygame's Sprite class to make it easier to display the weapon and code it. It also inherits all the attributes from the group of the sprite which would be the visible_sprites group. The direction is split where the '_' is mentioned in the status to get the direction of the player which would be the first in the split list.

The path of the image based on the direction of the player is stored so the correct image can be displayed.

This changes the position of where the weapon is displayed near the player, so it makes logical sense. The position is based upon the player's status and uses the player rect's position to determine where to place the weapon. The pygame.math.Vector2 is added to the position as an offset so it looks like the player is actually holding the weapon when attacking.

player.py > Player

```python
class Player(pygame.sprite.Sprite):
    def __init__(self,pos,groups,obstacle_sprites,create_attack,destroy_attack):
        # Weapon
        self.create_attack = create_attack
        self.destroy_attack = destroy_attack
        self.weapon = list(weapon_data.keys())[0]
```

The create_attack and destroy_attack are added as parameters and attributes in the Player class. This is so the Player class can access both methods. The weapon is also stored as the beginning of the weapon_data dictionary so that any of its data can easily be accessed as a list.

player.py > Player

```python
class Player(pygame.sprite.Sprite):
    def input(self):
            if keys[pygame.K_j] and not self.jumping:
                self.attacking = True
                self.attack_time = pygame.time.get_ticks()
                self.create_attack()
```

The create_attack method is used when the player presses the 'J' key to attack.

```python
class Level:
    def __init__(self):
        # Attack Sprites
        self.current_attack = None

        # Sprite setup
        self.create_map()

    def create_map(self):...

    def create_attack(self):
        self.current_attack = Weapon(self.player,[self.visible_sprites])

    def destroy_attack(self):
        if self.current_attack:
            self.current_attack.kill()
        self.current_attack = None
```

This would store the current attack and its image.

The create_attack method simply instantiates the Weapon class and stores it in the current_attack attribute.

The destroy_attack method simply kills the weapon sprite if it exists and then sets current_attack back to its default. This stops the weapon from continuously displaying even after the attack has finished.

The main issue when developing this was a logic error of the weapon image remaining after the player had finished attacking but this was solved by implementing the create_attack and destroy_attack methods.

The attacking now looks like this:

GIF (5) Weapon

Acceptance Test 8: Check that the weapon is working properly.

Inputs	Outputs	Validity
'J' key	Weapon appears in the player's hand, facing in the direction the player is facing only while the player is attacking.	True
'W' and 'J' keys	The player stops moving and the weapon appears in the player's hand, facing in the upwards direction only while the player is attacking.	True

‘S’ and ‘J’ keys	The player stops moving and the weapon appears in the player’s hand, facing in the downwards direction only while the player is attacking.	True
‘A’ and ‘J’ keys	The player stops moving and the weapon appears in the player’s hand, facing in the left direction only while the player is attacking.	True
‘D’ and ‘J’ keys	The player stops moving and the weapon appears in the player’s hand, facing in the right direction only while the player is attacking.	True
Other keyboard keys	Nothing occurs.	True

I had realised that I should implement the blocking later in the game development since it can only be tested using the enemies. Therefore, I shall skip its implementation for now and go on to the interactable objects and interacting.

Interacting

Before implementing any of the actions, I had to add another layer to the map of the interactable sprites. This layer only included the position of the level entrances because they use the same object – this means that you can’t distinguish between the level entrances using their object ID, unlike the other objects such as the trophies, the boss entrance and the arcade entrance. This also meant that I had to add the ID of the objects as an attribute to the Tile class. The interactable sprites layer was added to the layouts like all the other layers.

level.py > Level > create_map

```
10   class Level:
30       def create_map(self):
31           layouts = {
32               'boundary': import_csv_layout('../map/mainmap/mainmap_Wallblocks.csv'),
33               'interactable': import_csv_layout('../map/mainmap/mainmap_Interactable.csv'),
34               'detail': import_csv_layout('../map/mainmap/mainmap_BigDetails.csv'),
35               'object': import_csv_layout('../map/mainmap/mainmap_Objects.csv')
36           }
```

level.py > Level

```
10   class Level:
30       def create_map(self):
42           for style,layout in layouts.items():  # Items function retrieves the key and value pair
43               for row_index,row in enumerate(layout):  # Enumerate returns row index number for each row as row_index
44                   for col_index,col in enumerate(row):
45                       if col != '-1':
46                           x = col_index * TILESIZE
47                           y = row_index * TILESIZE
48                           if style == 'boundary':
49                               Tile((x,y),[self.obstacle_sprites],'invisible',col)  # col is the actual value of the tile
50                           if style == 'interactable':
51                               Tile((x,y),[self.interactable_sprites],'interactable',col)
52                           if style == 'detail':
53                               surf = graphics['details'][int(col)]
54                               Tile((x,y),[self.visible_sprites,self.obstacle_sprites],'detail',col,surf)
55                           if style == 'object':
56                               surf = graphics['objects'][int(col)]
57                               Tile((x,y),[self.visible_sprites,self.obstacle_sprites],'object',col,surf)
```

The tiles from the interactable layer are added like the other layers. The col value is added to the Tile object arguments as its ID.

tile.py > ...

```
4   class Tile(pygame.sprite.Sprite):
5       def __init__(self,pos,groups,sprite_type,id,surface=pygame.Surface((TILESIZE,TILESIZE))):
6           super().__init__(groups)
7           self.sprite_type = sprite_type
8           self.image = surface
9           self.id = id
```

The object's id is added as a parameter for the Tile.

The tile's attribute id is set.

tile.py > ...

```
4   class Tile(pygame.sprite.Sprite):
5       def __init__(self,pos,groups,sprite_type,id,surface=pygame.Surface((TILESIZE,TILESIZE))):
16          self.hitbox = self.rect.inflate(0,-10)
17          self.notice_radius = None
```

The attribute notice_radius is set as None initially. This is since if the attribute isn't set prior, there would be a syntax error saying that the attribute self.notice_radius doesn't exist.

tile.py > ...

```
4   class Tile(pygame.sprite.Sprite):
5       def __init__(self,pos,groups,sprite_type,id,surface=pygame.Surface((TILESIZE,TILESIZE))):
19          if sprite_type == 'interactable':
20              if int(self.id) == 17:
21                  self.response = 'Level 3'
22              if int(self.id) == 18:
23                  self.response = 'Level 2'
24              if int(self.id) == 19:
25                  self.response = 'Level 1'
26
27          if sprite_type == 'object':
28              for objectid,objectresponse in interactable_objects.items():
29                  if int(self.id) == objectid:
30                      self.notice_radius = self.hitbox.inflate(50,100)
31                      self.response = objectresponse
```

This is for the tiles from the 'interactable' layer. This simply checks the id of the tile and sets the response attribute according to its id. The response attribute is a placeholder that will later be replaced when implementing the different features.

This is for the interactable objects that from the 'object' layer instead. This has the same logic as the other interactable tiles however it uses a dictionary for the corresponding id and response. It also creates a notice_radius so the player can notice the interactable object.

settings.py > ...

```
11  interactable_objects = {
12      23: 'Boss',
13      32: 'Arcade',
14      37: 'Trophies',
15      43: 'Inspector',
16      44: 'Old man',
17      45: 'Villager 1',
18      46: 'Villager 2'
19  }
```

This was added to the settings file and is used for finding the corresponding response for the object id when the player interacts with the object. This was created so there is less code in the tile file: otherwise, it would require multiple if statements like done for the tiles in the 'interactable' layer which is less efficient.

After this, I needed to create a way for the player to notice the interactable sprites, so they know to interact with them. This required me to create a notice bubble above the player and to create a function to allow the player to interact with the interactable sprites. To do this, I created another Python file named 'interact.py' and a Notice_Bubble class.

player.py > Player > check_notice

```
6   class Player(pygame.sprite.Sprite):
7       def __init__(self,pos,groups,obstacle_sprites,interactable_sprites,create_attack,destroy_attack,notice):
39          # Notice + Interacting
40          self.notice = notice
41          self.noticing = False
42          self.notice_id = None
43          self.notice_response = None
44
45          self.obstacle_sprites = obstacle_sprites
46          self.interactable_sprites = interactable_sprites
```

The notice attribute would store the notice method from the Level class. The noticing attribute is a Boolean that will indicate if the player is currently noticing or not. The notice_id would store the ID of the object the player is noticing, and the notice_response would store its corresponding response.

The interactable_sprites are added as an attribute to the Player so they can be accessed within the Player class.

player.py > Player > interact

```
6   class Player(pygame.sprite.Sprite):
161     def check_notice(self):
162         self.noticing = False
163         self.notice_id = None
164         self.notice_response = None
165         for sprite in self.interactable_sprites:
166             if sprite.hitbox.colliderect(self.hitbox):
167                 self.noticing = True
168                 self.notice_id = sprite.id
169                 self.notice_response = sprite.response
170
171         for object in self.obstacle_sprites:
172             if object.notice_radius:
173                 if object.notice_radius.colliderect(self.hitbox):
174                     self.noticing = True
175                     self.notice_id = object.id
176                     self.notice_response = object.response
```

The noticing and notice_id are set to their default values initially and if the player is colliding with the interactable sprite, noticing is set to true, and the notice_id is set to the sprite's ID in addition to the notice_response being set to the corresponding response.

The same is done for the interactable objects but it checks for a collision with the object's notice_radius if it has one.

The check_notice method is added to the end of the Player's move method since it is a form of collisions as well.

level.py > Level

```
7   from interact import Notice_Bubble
8   from debug import debug
9
10  class Level:
11      def __init__(self):
12
13          # Get display surface
14          self.display_surface = pygame.display.get_surface()
15
16          # Sprite group setup
17          self.visible_sprites = YSortCameraGroup()
18          self.obstacle_sprites = pygame.sprite.Group()
19          self.interactable_sprites = pygame.sprite.Group()
20
21          # Attack Sprites
22          self.current_attack = None
23
24          # Notice Sprite
25          self.current_notice = None
```

The Notice_Bubble class is imported into the level file and interactable_sprites and current_notice have been set as attributes for class Level. The interactable_sprites is a sprite group like visible_sprites and obstacle_sprites. The current_notice is set to None as a default value.

level.py > Level

```
10  class Level:
69      def notice(self):
70          if self.player.noticing and self.current_notice == None:
71              self.current_notice = Notice_Bubble(self.player,[self.visible_sprites])
72          elif self.current_notice and not self.player.noticing:
73              self.current_notice.kill()
74              self.current_notice = None
75
76          if self.current_notice:
77              self.current_notice.update()
```

This method creates the notice bubble if the player is noticing, and they don't already have a current notice. Otherwise, the current notice is destroyed.

This updates the current notice if it exists.

The notice method is placed in the Player class' update method, so it's checked every time the player is updated (every main loop).

interact.py > Notice_Bubble

```python
import pygame

class Notice_Bubble(pygame.sprite.Sprite):
    def __init__(self,player,groups):
        super().__init__(groups)

        self.animation = ['0','1','2','3']
        self.frame_index = 0
        self.animation_speed = 0.025

        self.direction = player.direction
        self.player = player
        self.player_top = player.rect.midtop

        self.image = pygame.image.load('../graphics/notice/0.png').convert_alpha()
        self.rect = self.image.get_rect(midbottom=self.player_top)
```

The Notice_Bubble class is created, and it inherits the Pygame Sprite class since it acts as a sprite. The attributes animation, frame_index and animation_speed all help the notice bubble to be correctly animated. The animation array contains the names of the frame images.

The direction, player and player_top attribute are required to know where and when to place the notice bubble.

This sets the notice bubble's default image and its rect so it can be displayed.

interact.py > ...

```python
class Notice_Bubble(pygame.sprite.Sprite):
    def follow_player(self,player):
        if self.direction != [0,0]:
            self.player_top = player.rect.midtop
            self.rect.midbottom = self.player_top

    def animate(self):
        self.frame_index += self.animation_speed
        if self.frame_index >= len(self.animation):
            self.frame_index = 0
        full_path = f'../graphics/notice/{self.animation[int(self.frame_index)]}.png'
        self.image = pygame.image.load(full_path).convert_alpha()

    def update(self):
        self.follow_player(self.player)
        self.animate()
```

This method checks if the player is moving and if they are, the player_top attribute and rect position are updated to keep the notice bubble above the player even when they move.

The animate method has the same logic as the animation method for the player. It increments the frame index by the animation speed and once it reaches the size of the animation array, it returns to zero. The notice bubble's image is updated as the frame_index is increased.

The update method simply runs the other two methods which makes it easier to update the notice bubble in the level code.

Initially when trying to develop the code for the notice bubble, I had tried another approach which was to create multiple notices that would delete themselves – creating the illusion that they were following the player. However, this didn't work since I couldn't animate the notice bubble, and the notice bubble would shutter since it was appearing and disappearing at the same time. This made me change the way I created the notice bubble to how it is now. I did encounter a few syntax errors: one was because I had forgotten that coding a while loop would cause an error since the code is already looping through the main loop.

```
  File "c:\Users\gursi\OneDrive - The Holt School\Year 12\Computing\Coursework\Game\code\interact.py", line 18, in show_notice
    while self.show_notice():
          ^^^^^^^^^^^^^^^^^^
  [Previous line repeated 989 more times]
RecursionError: maximum recursion depth exceeded
```

The other was a type error since I had accidentally added [1] to the end of the y direction of the player which wasn't required. Though, I ended up changing how I made the notice bubble follow the player since when done the other way, it carried on moving away from the player and not actually following the player.

```
  File "c:\Users\gursi\OneDrive - The Holt School\Year 12\Computing\Coursework\Game\code\interact.py", line 21, in follow_player
    self.rect.y += int(self.direction.y[1])
                       ~~~~~~~~~~~~~~~~~^^^
TypeError: 'float' object is not subscriptable
```

Next, I had to implement a placeholder for interacting which just prints the name of what is being interacted with.

```
player.py > Player
  6   class Player(pygame.sprite.Sprite):
178       def interact(self):
179           if self.noticing and self.notice_id:
180               print(self.notice_response)
```

This checks if the player is noticing an interactable object and it has an ID. If it does, then the corresponding notice response is printed in the Python console.

```
player.py > Player
  6   class Player(pygame.sprite.Sprite):
 59       def input(self):
 96               if keys[pygame.K_l]:
 97                   self.interact()
```

The interact method is run if the player presses the 'L' key.

Now the game looks like this when interacting:

GIF (6) Notice Bubble

If the player presses interact, the name of the object they're interacting with is printed in the Python console:

```
Arcade
Arcade
Arcade
Trophies
Trophies
Trophies
Trophies
Level 1
Level 1
Level 2
Level 2
Level 2
Level 3
Level 3
Level 3
Boss
Boss
Boss
```

Because this is a placeholder for the actual interactions, the corresponding response will print for every moment the 'L' key is pressed down – I didn't change the code so it doesn't for now.

Acceptance Test 10: Check that interacting and interactable objects are working properly.

Inputs	Outputs	Validity
'L' key	Nothing occurs.	True
'L' key while noticing	The corresponding placeholder response is output in the console.	True
'WASD' while noticing	Notice bubble follows the player while they move, keeping it above their head. The animation works as normal.	True
Interactable area collisions (noticing)	Notice bubble appears above the player and is animated as expected.	True
Other keyboard keys	Nothing occurs if they don't have a use. If they do, there is no change to its use.	True

Level HUD

To implement the level HUD, I just have to display the player's health in the top left corner of the screen. This is very simple to add, and I will add the other features of the HUD later in the development since they aren't currently required: arcade HUD and boss HUD. I also created another python file for the UI to make the level code less bulky.

player.py > Player > __init__

```
6   class Player(pygame.sprite.Sprite):
7       def __init__(self,pos,groups,obstacle_sprites,interactable_sprites,create_attack,destroy_attack,notice):
48          # Stats
49          self.health = 4
50          self.coins = 123
```

These attributes were added to the player as their stats.

settings.py > ...

```
7   # UI
8   UI_FONT = '../graphics/font/NormalFont.ttf'
9   UI_FONT_SIZE = 18
10  TEXT_COLOUR = '#EEEEEE'
```

These are a few variables that I've set in the settings that may be used when creating the UI later in development.

ui.py > ...

```
1   import pygame
2   from settings import *
3
4   class UI:
5       def __init__(self):
6           # General
7           self.display_surface = pygame.display.get_surface()
8           self.font = pygame.font.Font(UI_FONT,UI_FONT_SIZE)
9
10      def show_health(self,current):
11          self.health_img = pygame.image.load(f'../graphics/health/{current}.png').convert_alpha()
12          self.display_surface.blit(self.health_img,(10,10))
13
14      def display(self,player):
15          self.show_health(player.health)
```

Pygame and settings are required.

The UI class is defined with its init method. The display_surface is set as an attribute so the UI can be drawn on the screen and the font is set for later development.

This sets the image of the player's health using their current health. This works since I've set the image names of the different health to the actual number of lives the player would have – making it easy to code. The image is then blit to the screen at the coordinates.

The display method simply indicates what parts of the UI will be shown.

level.py > ...

```
11  class Level:
12      def __init__(self):
31          # User Interface
32          self.ui = UI()
```

The UI class is set as an attribute to the Level class so it can be accessed.

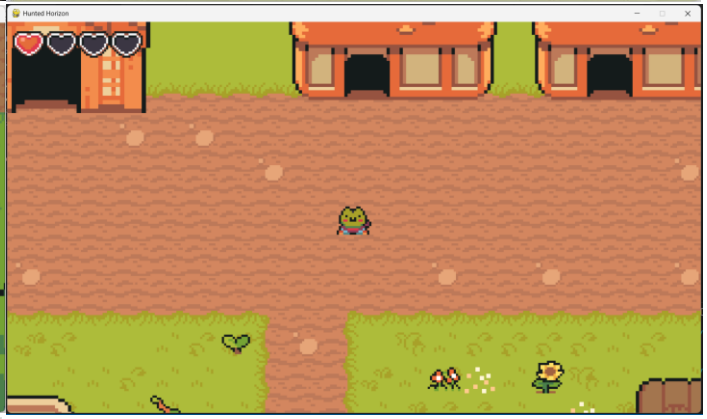
```
level.py > ...
11    class Level:
83        def run(self):
84            # Update + draw game
85            self.visible_sprites.custom_draw(self.player)
86            self.visible_sprites.update()
87            self.ui.display(self.player)
```

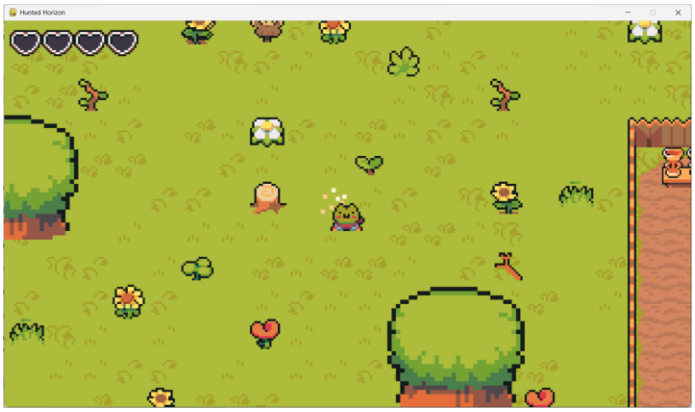
The UI is displayed through the Level's run method.

To test that the health updates while the game is running, I created extra inputs that would alter the health:

```
player.py > Player
6     class Player(pygame.sprite.Sprite):
63        def input(self):
103               if keys[pygame.K_UP]:
104                   if self.health < 4:
105                       self.health += 1
106
107               if keys[pygame.K_DOWN]:
108                   if self.health > 0:
109                       self.health -= 1
```

(The if statements are to ensure that the health doesn't go beyond the expected limits, otherwise it would cause a not found error.)





Because the health seems to work, I will remove the extra inputs.

Acceptance Test 11: Check that the HUD is working properly.

Inputs	Outputs	Validity
Player health increasing	The hearts at the top update to reflect the player's new health.	True
Player health decreasing	The hearts at the top update to reflect the player's new health.	True
Heart position	The hearts are placed and displayed in the correct position.	True

Loading Level 1

To load different maps, I need to separate the maps and their own layers so they can be loaded only when required. I also needed to create a simple class in the settings file so that all the other files could easily alter the game mode as well as an additional method in the Level class to destroy the previous map and load the new one.

level.py > Level > create_map

```
11  class Level:
35      def create_map(self):
36          graphics = {
37              'details': import_folder('../graphics/details'),
38              'objects': import_folder('../graphics/objects')
39          }
40
41          map_layouts = {
42              'mainmap': {'boundary': import_csv_layout('../map/mainmap/mainmap_Wallblocks.csv'),
43                          'interactable': import_csv_layout('../map/mainmap/mainmap_Interactable.csv'),
44                          'detail': import_csv_layout('../map/mainmap/mainmap_BigDetails.csv'),
45                          'object': import_csv_layout('../map/mainmap/mainmap_Objects.csv')},
46              'level1': {'boundary': import_csv_layout('../map/level1/level1_Wallblocks.csv'),
47                          'detail': import_csv_layout('../map/level1/level1_BigDetails.csv'),
48                          'object': import_csv_layout('../map/level1/level1_Objects.csv')}
49          }
50
51          for map,layouts in map_layouts.items():
52              if map == game_mode.mode:
53                  current_layouts = layouts
```

I placed the layouts in another dictionary to make it easy to access only the ones that are needed for the map. The graphics dictionary didn't need to be changed.

This sets the current layouts required for the map that is needed so it can be used to load the map.

level.py > Level > create_map

```
11  class Level:
35      def create_map(self):
55          for style,layout in current_layouts.items():  # Items function retrieves the key and value pair
56              for row_index,row in enumerate(layout):  # Enumerate returns row index number for each row as row_index
57                  for col_index,col in enumerate(row):
```

I simply changed layouts to current_layouts so it only creates the sprites for that map.

settings.py > ...

```
7   class MODE:
8       def __init__(self):
9           self.mode = 'mainmap'
10
11  game_mode = MODE()
```

This is a class I created to help make changing the mode easier since otherwise it would be difficult to change the mode and may require importing the player file into the settings file or using cyclical dependency[16] which I thought would overcomplicate the programming or make the game run slower. The object game_mode is then created from the MODE class.

level.py > Level

```
11  class Level:
12      def __init__(self):
28          # Sprite setup
29          self.current_mode = game_mode.mode
30          self.create_map()
```

The current_mode attribute is added to the Level class to help indicate when the mode has changed.

```
level.py > Level > create_map
11  class Level:
35      def create_map(self):
72          if self.current_mode == 'mainmap':
73              playerpos = (1500,3280)
74          elif self.current_mode == 'level1':
75              playerpos = (1000,4800)
76
77          self.player = Player(playerpos,[self.visible_sprites],self.obstacle_sprites,self.interactable_sprites,self.create_attack,self.destroy_attack,self.notice)
```

The current_mode is checked in the create_map method to know where to place the player according to the map. The self.player is also altered slightly to have this new position.

```
level.py > ...
118 class YSortCameraGroup(pygame.sprite.Group):
119     def __init__(self):
128         # Floor Creation
129         self.floor_surf = pygame.image.load(f'../graphics/tilemap/{game_mode.mode}_ground.png').convert()
130         self.floor_rect = self.floor_surf.get_rect(topleft=(0,0))
```

I had to also alter the floor_surf in YSortCameraGroup so that the floor of the game is the correct one according to the game mode.

```
level.py > ...
11  class Level:
79      def reload_map(self):
80          if self.current_mode != game_mode.mode:
81              for sprite in self.visible_sprites:
82                  sprite.kill()
83              for sprite in self.obstacle_sprites:
84                  sprite.kill()
85              for sprite in self.interactable_sprites:
86                  sprite.kill()
87
88              self.current_mode = game_mode.mode
89              self.visible_sprites = YSortCameraGroup()
90              self.create_map()
```

This method was added to destroy the old map and create the new map. First, it checks if the current_mode is different from the game_mode.mode (meaning that the map has changed). Then if so, the sprites are all killed.

The current_mode is updated, and the visible_sprites are reset so the floor of the map can be reset. The map is then recreated.

```
level.py > ...
11  class Level:
110     def run(self):
111         # Update + draw game
112         self.visible_sprites.custom_draw(self.player)
113         self.visible_sprites.update()
114         self.ui.display(self.player)
115         self.reload_map()
```

This method is put in the run method since a change in the game mode needs to be checked each loop.

```
player.py > Player
6   class Player(pygame.sprite.Sprite):
182     def interact(self):
183         if self.noticing and self.notice_id:
184             print(self.notice_response)
185             if int(self.notice_id) == 19:
186                 game_mode.mode = 'level1'
```

The interact method in the Player is updated so that if the ID of the sprite the player is interacting with the same to take them to the first level, the game mode is changed to reflect that.

After implementing this, the first level loads as expected. However, I also changed the UI so it only appears when the player is on the level.

```
ui.py > ...
4   class UI:
14      def display(self,player):
15          if game_mode.mode[:5] == 'level':
16              self.show_health(player.health)
```

The if statement checks if the first five letters of the game mode are ‘level’ and only displays the player health if that’s true now.

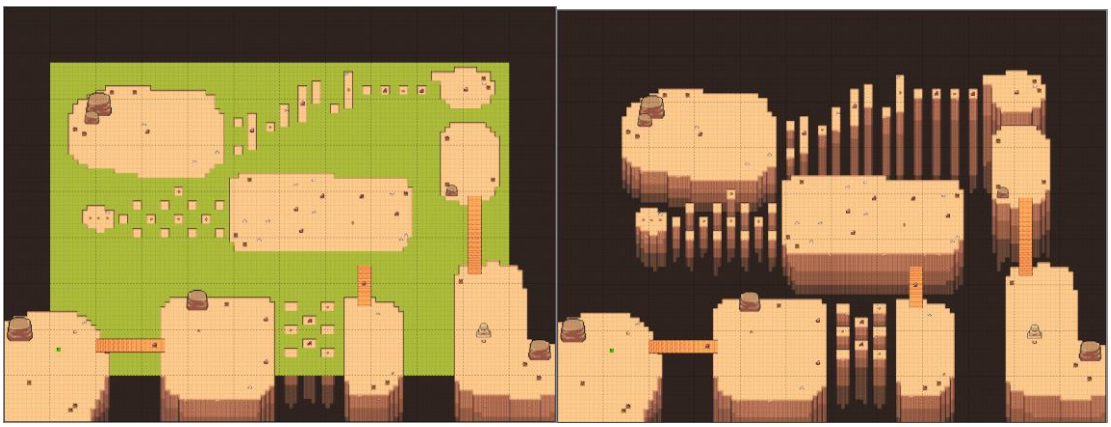
GIF (7) Loading level 1

Acceptance Test 12: Check that the first level map loads properly.

Inputs	Outputs	Validity
New map floor	Old map floor must be replaced with new map floor.	True
New objects	Old map objects must be deleted, and new map objects must be placed.	True
Player object collisions	Player should correctly collide with objects like done before.	True
HUD change	HUD should change with the map change and correctly appear.	True
Player position	Player should be positioned correctly on the new map when it is loaded.	True

Falling

To create the falling, I had to add another layer of sprites to the level that would help indicate when the player isn't on the floor and should be falling. I then created a prototype of how the falling may look as an initial prototype – this is simply to speed up the overall development since I can come back to the falling animation. In addition, I had to change part of the first level as I realised it was too difficult, and it was causing errors (the pillar section which now has wider platforms). Below shows the extra layer of sprites and the first level map after the changes.



```
level.py > Level > create_map
11    class Level:
12        def __init__(self):
17            # Sprite group setup
18            self.visible_sprites = YSortCameraGroup()
19            self.obstacle_sprites = pygame.sprite.Group()
20            self.interactable_sprites = pygame.sprite.Group()
21            self.fall_sprites = pygame.sprite.Group()
```

Another group for the fall_sprites is created so I can easily access each of the sprites.

```
level.py > Level > create_map
11    class Level:
36        def create_map(self):
42            map_layouts = {
43                'mainmap': {'boundary': import_csv_layout('../map/mainmap/mainmap_Wallblocks.csv'),
44                            'interactable': import_csv_layout('../map/mainmap/mainmap_Interactable.csv'),
45                            'detail': import_csv_layout('../map/mainmap/mainmap_BigDetails.csv'),
46                            'object': import_csv_layout('../map/mainmap/mainmap_Objects.csv')},
47                'level1': {'boundary': import_csv_layout('../map/level1/level1_Wallblocks.csv'),
48                            'detail': import_csv_layout('../map/level1/level1_BigDetails.csv'),
49                            'object': import_csv_layout('../map/level1/level1_Objects.csv'),
50                            'fallblock': import_csv_layout('../map/level1/level1_Fallblocks.csv')}
51            }
```

The fallblock layer of the first level is imported from a csv file within the level1 dictionary key.

```
level.py > Level > create_map
11    class Level:
36        def create_map(self):
57            for style,layout in current_layouts.items():  # Items function retrieves the key and value pair
58                for row_index,row in enumerate(layout):  # Enumerate returns row index number for each row as row_index
59                    for col_index,col in enumerate(row):
60                        if col != '-1':
61                            x = col_index * TILESIZE
62                            y = row_index * TILESIZE
63                            if style == 'boundary':
64                                Tile((x,y),[self.obstacle_sprites],'invisible',col)  # col is the actual value of the tile
65                            if style == 'fallblock':
66                                Tile((x,y),[self.fall_sprites],'fallblock',col)
```

The fall sprites are created and added to the group like the other layers are.

```
level.py > Level
11    class Level:
36        def create_map(self):
81            self.player = Player(playerpos,[self.visible_sprites],self.obstacle_sprites,self.interactable_sprites,self.fall_sprites,self.create_attack,self.destroy_attack,self.notice)
```

The fall_sprites are added to the instantiation of the Player object so that it can be accessed within the Player class.

```
level.py > Level
11    class Level:
83        def reload_map(self):
84            if self.current_mode != game_mode.mode:
85                for sprite in self.visible_sprites:
86                    sprite.kill()
87                for sprite in self.obstacle_sprites:
88                    sprite.kill()
89                for sprite in self.interactable_sprites:
90                    sprite.kill()
91                for sprite in self.fall_sprites:
92                    sprite.kill()
```

The fall_sprites are destroyed when the map is reloaded to ensure there no logical errors.

```
player.py > ...
6   class Player(pygame.sprite.Sprite):
7       def __init__(self,pos,groups,obstacle_sprites,interactable_sprites,fall_sprites,create_attack,destroy_attack,notice):
8           super().__init__(groups)
9           self.image = pygame.image.load('../graphics/test/player.png').convert_alpha()
10          self.rect = self.image.get_rect(topleft = pos)
11          self.hitbox = self.rect.inflate(0,-26)
12          self.fall_hitbox = self.rect.inflate(-50,-62)
13          self.fall_hitbox.bottom = self.hitbox.bottom
14
15          self.width = 64
16          self.height = 64
```

The fall_sprites are added as a parameter.

I added a separate hitbox for the player when dealing with falling since the fall_hitbox would have to be smaller than the actual hitbox. The fall_hitbox is also set to the bottom of the player's hitbox, so it's positioned properly.

This was added as attributes to the player since this will be used for the prototype falling animation. They are both 64 since, in this case, the height and width of the player never change during animations.

```
player.py > Player > __init__
6   class Player(pygame.sprite.Sprite):
7       def __init__(self,pos,groups,obstacle_sprites,interactable_sprites,fall_sprites,create_attack,destroy_attack,notice):
28          self.current_ground = self.hitbox.y
29          self.gravity = 0.8
30          self.jump_force = -15
31          self.vertical_speed = 0
32          self.jumping = False
33          self.down = False
34          self.jump_released = True
35          self.jump_pos = None
36          self.falling = False
```

The jump_released attribute was added since I had realized the player shouldn't be able to continuously hold down the space bar and jump – this stops the player from being able to fall and then they're able to complete the level by just holding jump and skipping to the end. Therefore, I needed to added a flag that indicates that the jump had ended which is jump_released. The jump_pos is used for storing a position used when repositioning the player after they fall and have jumped. The falling attribute indicates when the player should be falling.

```
player.py > Player > __init__
 6  class Player(pygame.sprite.Sprite):
 7      def __init__(self,pos,groups,obstacle_sprites,interactable_sprites,fall_sprites,create_attack,destroy_attack,notice):
53          self.obstacle_sprites = obstacle_sprites
54          self.interactable_sprites = interactable_sprites
55          self.fall_sprites = fall_sprites
```

The fall_sprites attribute is defined using the fall_sprites group, allowing the fall_sprites to be accessed within the Player class.

```
player.py > Player > __init__
 6  class Player(pygame.sprite.Sprite):
 7      def __init__(self,pos,groups,obstacle_sprites,interactable_sprites,fall_sprites,create_attack,destroy_attack,notice):
61          self.spawn_pos = self.hitbox.center
62          self.repos = None
```

The spawn position of the player on the level is also set as an attribute for repositioning the player. The repos attribute is initially set as None but it would store the position for the player to be repositioned when needed.

```
player.py > Player
 6  class Player(pygame.sprite.Sprite):
75      def input(self):
101             if keys[pygame.K_SPACE] and self.jump_released:
102                 self.jump()
103                 self.jump_released = False
104
105             if not keys[pygame.K_SPACE]:
106                 self.jump_released = True
```

The space input now only works when the jump has ended/space has been released.

Once space is pressed and jump_released is True, the jump method is run and jump_released is set to False to show that the player is currently jumping.

If the user is no longer pressing space, jump_released is set as True. This stops the user from holding space continuously.

This checks if the player is colliding with the fall blocks and whether they are jumping. If they are colliding and not jumping, the falling attribute is set to True and the player's direction is set to zero to stop the player from moving. The jumping attribute is checked since the fall animation shouldn't trigger if the player is currently jumping – helping to emulate the 3D aspect of the game. This is also placed in the move method because it's a form of collision and needs to be checked during player movement.

```
player.py > Player
 6  class Player(pygame.sprite.Sprite):
151     def move(self,speed):
161         self.fall_hitbox.midbottom = self.hitbox.midbottom
162
163         if self.fall_hitbox.collidelist(list(self.fall_sprites)) != -1 and not self.jumping:
164             self.falling = True
165             self.direction.x = 0
166             self.direction.y = 0
```

This updates the position of the fall_hitbox to ensure that it's always always in the correct position. This is put in the move method since it will be updated each time the player is updated, and the fall_hitbox will only change position when the player moves.

```
player.py > Player
 6  class Player(pygame.sprite.Sprite):
219     def jump(self):
220         if not self.jumping:
221             self.jumping = True
222             self.vertical_speed = self.jump_force
223             self.jump_pos = self.hitbox.center
```

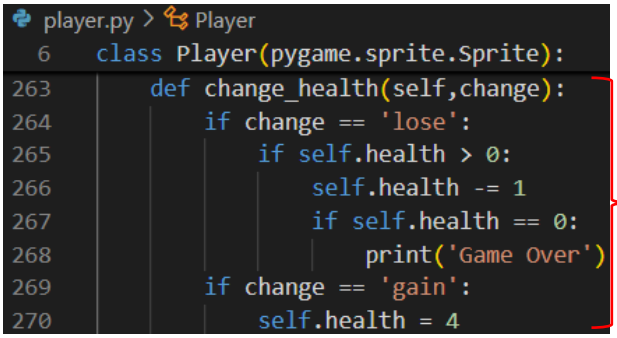
When the jump method is run, the position of the player is set as the jump_pos attribute – showing where the player jumped from.

```
player.py > Player
 6  class Player(pygame.sprite.Sprite):
244     def fall(self):
245         if self.falling:
246             pos = self.rect.topleft
247             if self.width > 0 and self.height > 0:
248                 self.image = pygame.transform.scale(self.image,(int(self.width),int(self.height)))
249                 self.rect = self.image.get_rect(topleft = pos)
250                 self.width -= 1.5
251                 self.height -= 1.5
252                 self.hitbox.y += 3
253             else:
254                 self.falling = False
255                 self.change_health('lose')
256                 if self.jump_pos:
257                     self.hitbox.center = self.jump_pos
258                 else:
259                     self.hitbox.center = self.spawn_pos
260                 self.width = 64
261                 self.height = 64
```

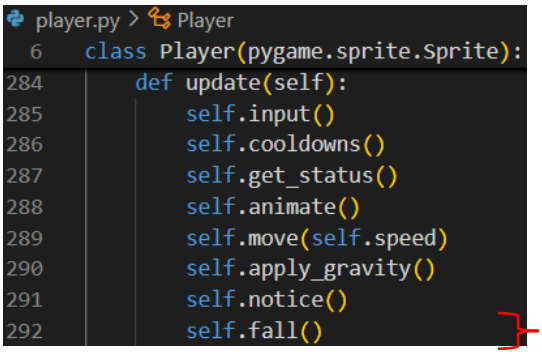
I created another method in the Player class for the falling. It's only run when the falling attribute is True and the position of the player is stored in the variable pos. This is used later for making the player appear smaller as if they're falling.

The if statement acts as a while statement because the code block will run every time the player is updated if falling is True and the player's width and height are greater than 0. The player's image is then scaled using the attributes and the rect is updated. The width and height are then decremented by 1.5 as well as the y position of the player being moved down by 3. This gives the illusion of the player falling.

Else, this means that the player has fell completely so the falling attribute is set back to False and the player loses some of their health. The player is then repositioned where they last jumped if they have jumped or where their spawn position is. Their height and width are also restored back to normal.



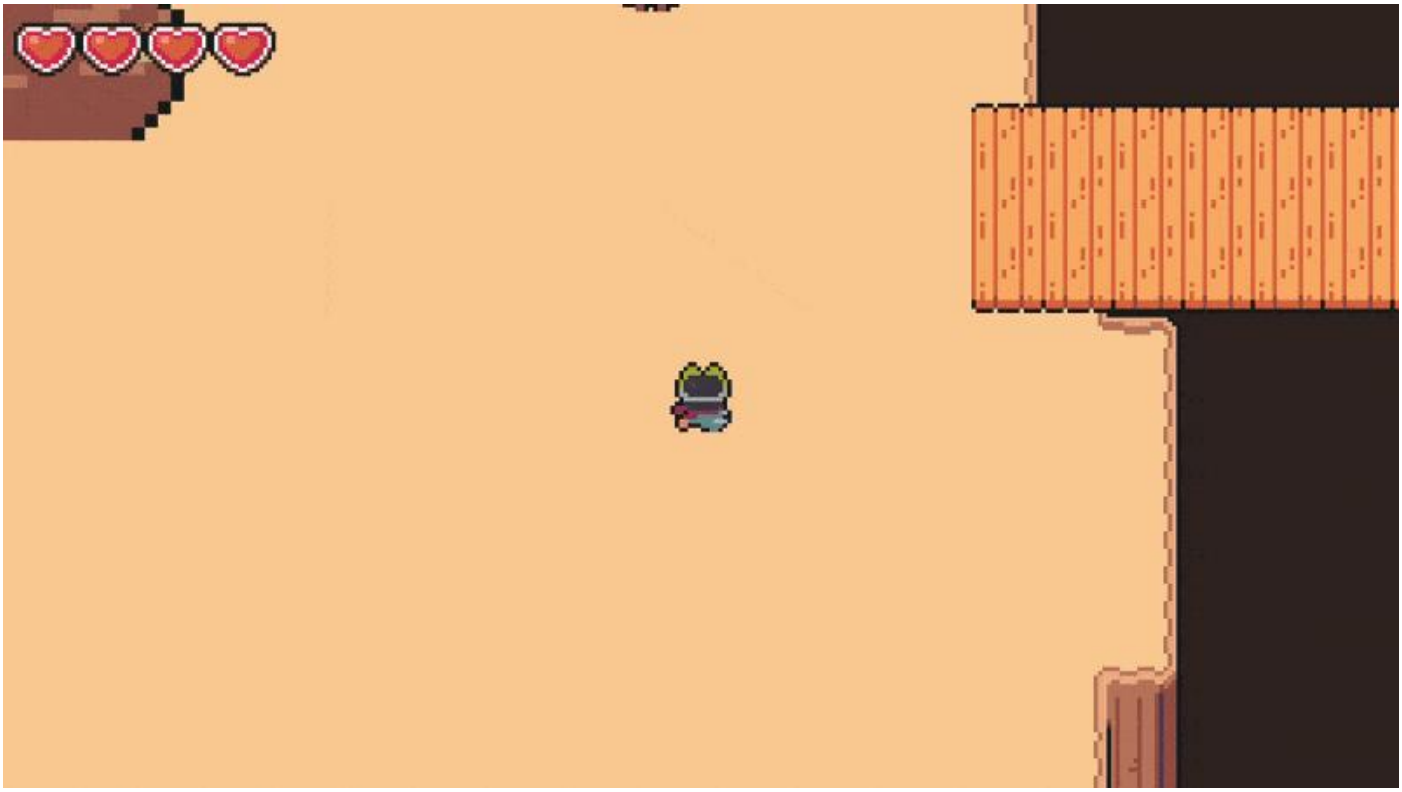
The change_health method was also added to the Player class so that when the player falls off the map, they also lose their health. The change parameter just represents how the health will change and if it's 'lose', the player's health will decrease. If the player's health reaches 0, 'Game Over' is printed as a placeholder for now. I also added what would happen when the player gains health which would mean that they have restarted a level or have leaved a level, and their health needs to be reset to 4.



The fall method is added to the update method since it needs to be checked continuously.

During the development of the falling, I encountered lots of logical errors such as the exploit that I had fixed – the holding jump exploit. I also tried the opposite of the fall_blocks and instead made sprites that would cover the floor, however this resulted in the game becoming very laggy and slow – this may have been because the collision was being continuously checked within the if statement while the player plays instead of the one-time collision with a fall_block being checked. Therefore, I had to stick to the fall_blocks instead. In addition, it was difficult to get a clear position for repositioning the player, so I had used the position where they last jumped or the spawn point instead – keeping the repositioning fairly near to where the player had fell.

This feature of the game is quite difficult to make so I decided to keep the falling as a prototype and I would come back to it later in development. Currently, the falling serves its purpose of indicating that the player has fallen and repositioning them with a small animation. Although, the falling doesn't fully emulate a 3D space since the player can't fall behind platforms.



GIF (8) Testing falling

Acceptance Test 13: Check that falling works properly.

Inputs	Outputs	Validity
Player and fall_block collision while jumping is False	Player should be animated as falling and then lose a life and is repositioned.	True
Player and fall_block collision while jumping is True	Player should jump as normal unless they land off the map which causes them to fall.	True
No player and fall_block collision	Player should function as normal.	True
Player health becoming 0	'Game Over' should be printed in the Python console.	True

Enemies

To make my game more modular, I need to create a separate class called Entity which would store the methods and attributes that both the player and enemies will have. This way there is no repeated code, and the Player and Enemy classes will be able to inherit from the Entity class instead. I placed the Entity class in a different Python file called entity.py as well as the Enemy class (enemy.py).

entity.py > ...

```python
import pygame

class Entity(pygame.sprite.Sprite):
    def __init__(self,groups):
        super().__init__(groups)
        self.frame_index = 0
        self.animation_speed = 0.15
        self.direction = pygame.math.Vector2()

    def move(self,speed):
        if self.direction.magnitude() != 0:
            self.direction = self.direction.normalize()  # Makes the direction vector always 1

        self.hitbox.x += self.direction.x * speed
        self.collision('horizontal')
        self.hitbox.y += self.direction.y * speed
        self.collision('vertical')
        self.rect.center = self.hitbox.center
```

These attributes will remain the same for the player and the enemies so they can be inherited from Entity instead.

The move method from the Player class is moved to Entity instead since they'll be shared. I had to remove the notice method and checking if the player was falling since they are only applicable for the player.

entity.py > ...

```python
class Entity(pygame.sprite.Sprite):
    def collision(self,direction):
        if direction == 'horizontal':
            for sprite in self.obstacle_sprites:
                if sprite.hitbox.colliderect(self.hitbox):  # Checks if obstacle and player are colliding
                    if self.direction.x > 0:  # Moving right
                        self.hitbox.right = sprite.hitbox.left
                    if self.direction.x < 0:  # Moving left
                        self.hitbox.left = sprite.hitbox.right

        if direction == 'vertical':
            for sprite in self.obstacle_sprites:
                if sprite.hitbox.colliderect(self.hitbox):
                    if self.direction.y > 0:  # Moving down
                        self.hitbox.bottom = sprite.hitbox.top
                    if self.direction.y < 0:  # Moving up
                        self.hitbox.top = sprite.hitbox.bottom
```

The collision method is also moved since it can be shared between the player and enemies too. However, there wasn't a need to change the method in any way.

player.py > Player

```python
class Player(Entity):
    def check_fall(self):
        self.fall_hitbox.midbottom = self.hitbox.midbottom

        if self.fall_hitbox.collidelist(list(self.fall_sprites)) != -1 and not self.jumping:
            self.falling = True
            self.direction.x = 0
            self.direction.y = 0
```

I created an extra method to check for falling which is simply the code that used to be within the move method.

player.py > Player

```python
class Player(Entity):
    def update(self):
        self.input()
        self.cooldowns()
        self.get_status()
        self.animate()
        self.move(self.speed)
        self.apply_gravity()
        self.check_notice()
        self.notice()
        self.check_fall()
        self.fall()
```

The check_fall method is added to the update method instead as well as the notice method that used to be within the move method. This is because both methods need to be checked every time the player is updated.

```
player.py > Player
4   from entity import Entity
5   from debug import debug
6
7   class Player(Entity):
```

The Entity class is imported into the player file so it can be inherited.

The Player class now inherits Entity instead of pygame.sprite.Sprite since Entity already inherits that and therefore Player will also inherit pygame.sprite.Sprite and it doesn't need to be stated again.

The data for each of the enemies was placed into settings as a nested dictionary. This is so that each enemy's stats can be accessed easily and applied when instantiating enemies.

```
settings.py > ...
32  monster_data = {
33      'raccoon': {'health':4, 'damage':1, 'attack_type':'claw', 'attack_sound':'../audio/attack/claw.wav', 'speed':3, 'resistance':3, 'attack_radius':60, 'notice_radius':350},
34      'cyclops': {'health':4, 'damage':1, 'attack_type':'stomp', 'attack_sound':'../audio/attack/stomp.wav', 'speed':3, 'resistance':3, 'attack_radius':60, 'notice_radius':350},
35      'slime': {'health':3, 'damage':1, 'attack_type':'ice', 'attack_sound':'../audio/attack/ice.wav', 'speed':1, 'resistance':3, 'attack_radius':120, 'notice_radius':400}
36  }
```

```
enemy.py > ...
1   import pygame
2   from settings import *
3   from entity import Entity
4   from support import *
5
6   class Enemy(Entity):
7       def __init__(self,monster_name,pos,groups,obstacle_sprites):
8           super().__init__(groups)
9           self.sprite_type = 'enemy'
10
11          # Graphics
12          self.import_graphics(monster_name)
13          self.status = 'idle'
14          self.image = self.animations[self.status][self.frame_index]
15
16          # Movement
17          self.rect = self.image.get_rect(topleft=pos)
18          self.hitbox = self.rect.inflate(0,-10)
19          self.obstacle_sprites = obstacle_sprites
```

The settings need to be imported so the monster data can be accessed, the Entity class needs to be imported so it can be inherited, and the support file needs to be imported so that the import_folder function can be used.

The Enemy class inherits Entity and has parameters for the monster/enemy name, the position of the enemy, and obstacle_sprites so the collision method can run for the enemies. The groups attributes and methods are inherited and set using the super init. The sprite_type is set to 'enemy' so it's easier to create the enemies in the level since the sprites can be differentiated.

The graphics of the enemy is imported into the attributes using the import_graphics method within the Enemy class. The status is set to 'idle' as a default and the enemy's image is set using the status and the frame index of the enemy. This is similar to how the player's image is altered.

The rect of the enemy is set as an attribute using its image. The hitbox is also set using the enemy's rect. The obstacle_sprites are also set as an attribute so it can be accessed in the collision method.

```
enemy.py > ...
6   class Enemy(Entity):
7       def __init__(self,monster_name,pos,groups,obstacle_sprites):
21          # Stats
22          self.monster_name = monster_name
23          monster_info = monster_data[self.monster_name]
24          self.health = monster_info['health']
25          self.speed = monster_info['speed']
26          self.attack_damage = monster_info['damage']
27          self.resistance = monster_info['resistance']
28          self.attack_radius = monster_info['attack_radius']
29          self.notice_radius = monster_info['notice_radius']
30          self.attack_type = monster_info['attack_type']
31
32      def import_graphics(self,name):
33          self.animations = {'idle':[],'attack':[],
34              'move_up':[], 'move_down':[], 'move_right':[], 'move_left':[]}
35          main_path = f'../graphics/monsters/{name}/'
36          for animation in self.animations.keys():
37              self.animations[animation] = import_folder(main_path + animation)
```

The monster_name is set using the argument and the monster_info variable holds the dictionary of the correct monster, taken from the monster_data within settings. The rest of the enemy's attributes are set using the data from the correct enemy dictionary.

The import_graphics method is like the Player class' import_player_assets method and has the same purpose. The enemy's animation image frames are all placed within the animations dictionary so they can be accessed when the animation changes. The animation frame paths for each animation are placed in the array of the corresponding key and value pair, using the key and the path of the animation frames file passed as arguments for the import_folder function. This then places all the folder's contents within the array.

```
enemy.py > ...
6   class Enemy(Entity):
61      def update(self):
62          self.move(self.speed)
```

The update method is also added to the Enemy class so that all the enemies can be updated while the game runs. The move method is called within update but currently the direction of the enemies would only be set to (0,0) so they won't move.

```
level.py > ...
9   from enemy import Enemy
```

The Enemy class is imported into the level file so that it can be accessed, and enemies can be instantiated.

```
level.py > ...
12  class Level:
37      def create_map(self):
43          map_layouts = {
44              'mainmap': {'boundary': import_csv_layout('../map/mainmap/mainmap_Wallblocks.csv'),
45                          'interactable': import_csv_layout('../map/mainmap/mainmap_Interactable.csv'),
46                          'detail': import_csv_layout('../map/mainmap/mainmap_BigDetails.csv'),
47                          'object': import_csv_layout('../map/mainmap/mainmap_Objects.csv'),
48                          'entities': import_csv_layout('../map/mainmap/mainmap_Entities.csv')},
49              'level1': {'boundary': import_csv_layout('../map/level1/level1_Wallblocks.csv'),
50                          'detail': import_csv_layout('../map/level1/level1_BigDetails.csv'),
51                          'object': import_csv_layout('../map/level1/level1_Objects.csv'),
52                          'fallblock': import_csv_layout('../map/level1/level1_Fallblocks.csv'),
53                          'entities': import_csv_layout('../map/level1/level1_Entities.csv')}
54          }
```

The entities layer of the mainmap and level1 are added to the layouts. The entities layer only consists of the player and enemies. Originally, the NPCs were also within them, but I moved them to the object csv file instead which makes it easier to code but also means that they now appear and are interactable since the code was already set for that to occur.

```
level.py > ...
12  class Level:
37      def create_map(self):
75                          if style == 'object':
76                              surf = graphics['objects'][int(col)]
77                              Tile((x,y),[self.visible_sprites,self.obstacle_sprites],'object',col,surf)
78                          if style == 'entities':
79                              if col == '42':
80                                  self.player = Player(
81                                  (x,y),
82                                  [self.visible_sprites],
83                                  self.obstacle_sprites,
84                                  self.interactable_sprites,
85                                  self.fall_sprites,
86                                  self.create_attack,
87                                  self.destroy_attack,
88                                  self.notice)
89                              else:
90                                  if col == '39': monster_name = 'slime'
91                                  elif col == '40': monster_name = 'cyclops'
92                                  else: monster_name = 'raccoon'
93                                  Enemy(monster_name,(x,y),[self.visible_sprites],self.obstacle_sprites)
```

The style is checked when placing the tiles from the file/matrix like it was done before. The tile that has value '42' represents the player and is where the player should be placed and instantiated. Nothing has changed in the arguments for the player attribute except the position is now just the x and y coordinates of the tile instead.

If the tile isn't a player, then the tile must be an enemy since the entities csv file only contains the player and the enemies. The monster_name is defined using the corresponding value of the tile and is passed as an argument when instantiating the enemy, so the correct enemy is displayed. The position of the enemy is using the x and y coordinates of the tile – like the player does. Currently, the two other arguments are so the enemies can appear (self.visible_sprites) and so the enemy's move method can work by passing the obstacle_sprites.

I encountered an error when instantiating the enemy, but this was simply because when I was checking the col variable in the if statement, I had accidentally put an integer instead of a string (e.g. 39 instead of '39'). This error was solved when I changed the integers into strings.

```
  File "c:\Users\gursi\OneDrive - The Holt School\Year 12\Computing\Coursework\Game\code\level.py", line 93, in create_map
    Enemy(name,(x,y),[self.visible_sprites])
          ^^^^
UnboundLocalError: cannot access local variable 'name' where it is not associated with a value
```

To test that all the enemies were appearing I placed them all within the first level, but they will be removed once they have all been checked and tested. The enemies are now visible in their correct spaces and are the correct type.

Acceptance Test 14: Check that enemies appear properly.

Inputs	Outputs	Validity
Raccoon enemy tile	The raccoon enemy is placed in the correct position with the correct image.	True
Cyclops enemy tile	The cyclops enemy is placed in the correct position with the correct image.	True
Slime enemy tile	The slime enemy is placed in the correct position with the correct image.	True

Now, I had to allow the enemies to be able to move and follow the player. This can be done by checking the distance between the player and the enemies and moving the enemy towards the player once they are in range and attacking the player when they are in range.

enemy.py > Enemy

```
6   class Enemy(Entity):
41      def get_player_distance_direction(self,player):
42          enemy_vec = pygame.math.Vector2(self.hitbox.center)
43          player_vec = pygame.math.Vector2(player.hitbox.center)
44          distance = (player_vec - enemy_vec).magnitude()
45
46          if distance > 0:
47              direction = (player_vec - enemy_vec).normalize()
48          else:
49              direction = pygame.math.Vector2()  # Gives vector of (0,0)
50
51          return (distance,direction)
```

The get_player_distance_direction method was added to the Enemy class to get the distance and direction vector between the player and each enemy. This is why it takes the player as a parameter as well. The enemy_vec variable is the position vector for the enemy and the player_vec variable is the position vector of the player. The distance variable is the size of the difference of the two vectors.

The distance is checked to ensure it isn't zero before the direction variable is set since the direction can't be normalised if it is zero. The normalisation means that the direction stays within -1 and 1, making it easier to apply the speed to the enemy movement. If the distance is zero, this means that the enemy is on top of the player and so the vector should be (0,0). Both the distance and direction are returned.

enemy.py > Enemy

```
6   class Enemy(Entity):
96      def enemy_update(self,player):
97          self.get_status(player)
98          self.actions(player)
```

To get the player's information, the method enemy_update is used that will update the enemy's status and actions while the game is running.

level.py > ...

```
136 class YSortCameraGroup(pygame.sprite.Group):
164     def enemy_update(self,player):
165         enemy_sprites = [sprite for sprite in self.sprites() if hasattr(sprite,'sprite_type') and sprite.sprite_type == 'enemy']
166         for enemy in enemy_sprites:
167             enemy.enemy_update(player)
```

An enemy_update method is added to the YSortCameraGroup class so the enemies can all update themselves while the level is running. The enemy_sprites variable is simply a list of all the enemies and then the enemy_update method is run for each enemy.

level.py > ...

```
12  class Level:
128     def run(self):
129         # Update + draw game
130         self.visible_sprites.custom_draw(self.player)
131         self.visible_sprites.update()
132         self.visible_sprites.enemy_update(self.player)
133         self.ui.display(self.player)
134         self.reload_map()
```

The enemy_update method is run in the run method of the Level class which means it will update the data each main loop.

As I was developing the next part of the enemy movement, I realised that I needed to alter the import_graphics method in the Enemy class since it would make it easier to animate the enemies' movement. This also meant that I had to alter the layout of the folders for the enemies, so it still matches up.

```
enemy.py > Enemy
 6  class Enemy(Entity):
33      def import_graphics(self,name):
34          self.animations = {'idle_up':[],'idle_down':[],'idle_right':[],'idle_left':[],
35              'attack_up':[],'attack_down':[],'attack_right':[],'attack_left':[],
36              'move_up':[], 'move_down':[], 'move_right':[], 'move_left':[]}
37          main_path = f'../graphics/monsters/{name}/'
38          for animation in self.animations.keys():
39              self.animations[animation] = import_folder(main_path + animation)
```

Like the player's animations attribute, the different directions were separated for each action. This makes it easier to animate the enemy.

```
enemy.py > Enemy
 6  class Enemy(Entity):
 7      def __init__(self,monster_name,pos,groups,obstacle_sprites):
13          self.status = 'idle_down'
14          self.status_direction = 'down'
```

This meant that I had to alter the status to contain the direction, and I also created a separate attribute for the status direction.

```
enemy.py > Enemy
 6  class Enemy(Entity):
53      def get_status(self,player):
54          distance = self.get_player_distance_direction(player)[0]
55
56          if self.direction[0] != 0 and self.direction[1] != 0:
57              if abs(self.direction[0]) < abs(self.direction[1]):
58                  if self.direction[1] > 0:
59                      self.status_direction = 'down'
60                  else:
61                      self.status_direction = 'up'
62              else:
63                  if self.direction[0] > 0:
64                      self.status_direction = 'right'
65                  else:
66                      self.status_direction = 'left'
67
68          if distance <= self.attack_radius:
69              self.status = f'attack_{self.status_direction}'
70          elif distance <= self.notice_radius:
71              self.status = f'move_{self.status_direction}'
72          else:
73              self.status = f'idle_{self.status_direction}'
```

The get_status method is for changing the enemy's status while the game runs. The distance between the enemy and player is obtained using the get_player_distance_direction method.

The direction of the enemy is checked to make sure it isn't zero first. Otherwise, when it is zero, it would default to having a status_direction of 'left' instead of keeping what it previously was. The absolute values of the x and y values of the direction are compared to each other. This is because if the x value is less than the y value, it means there's a bigger difference in the vertical distance between the enemy and player, so the enemy should be moving vertically. The y value is then checked if it is positive which would mean the enemy would have to move down. Otherwise, the enemy is moving up since only the vertical movement is being checked in this part.

This checks the horizontal movement since the enemy can only move horizontally and vertically. The same logic is applied but the x value of the direction is checked instead.

This changes the status of the enemy based off whether the player is within the attack radius first and then the notice radius. It's checked in this order since otherwise, the status would never change to attack and would take the move branch instead. If the player isn't in both radii, then the enemy should be idle.

```
enemy.py > Enemy
 6  class Enemy(Entity):
75      def actions(self,player):
76          if self.status[:6] == 'attack':
77              print('attack')
78          elif self.status[:4] == 'move':
79              self.direction = self.get_player_distance_direction(player)[1]
80          else:
81              self.direction = pygame.math.Vector2()
```

The action method is where the enemy can make movements according to the status of the enemy. Currently, the attack status only has a placeholder output, but the move status changes the direction of the enemy toward the player which moves the enemy. If the enemy isn't moving or attacking, the enemy is idle, and the direction is changed back to (0,0) – stopping the enemy from moving.

```
enemy.py > Enemy
 6  class Enemy(Entity):
83      def animate(self):
84          animation = self.animations[self.status]
85          self.frame_index += self.animation_speed
86          if self.frame_index >= len(animation):
87              self.frame_index = 0
88
89          self.image = animation[int(self.frame_index)]
90          self.rect = self.image.get_rect(center=self.hitbox.center)
```

The animate method in the Enemy class is the same as the animate method for the Player class, but it can't be placed within the Entity class because the animations attribute will be different for the enemy and player. This means that it can cause an error if I did move the animate method into Entity instead.

I had also changed the monster's data slightly: I changed the cyclops' speed to 2 from 3.

Now you can see the enemies moving towards the player and attacking when required. Although, the enemies aren't exactly attacking as they should just yet.

GIF (9) Enemy movement

Acceptance Test 15: Check that enemies move properly.

Inputs	Outputs	Validity
Player outside of enemy notice radius	Enemy should be idle and not move while displaying the correct idle image.	True
Player within enemy notice radius	Enemy should start moving towards the player with the correct walking animation and a constant speed until they reach the attack radius.	True
Player within enemy attack radius	Enemy should display the placeholder attack animation while still moving towards the player. The string 'attack' should also be displayed in the Python console once the player and enemy overlap.	True
Enemy collides with objects	Enemy should correctly collide with any objects while they are moving.	True

Next, the enemies need to have their separate attacks and characteristics. I first started with implementing the raccoon's attacks first because they are the enemy introduced within the first level. To implement this, I had to first implement basic attacking then basic player and enemy interaction and finally implement the raccoon's attack. This also required me to create another Python file for the particles called 'particles.py' which includes the AnimationPlayer and ParticleEffect classes.

```
enemy.py > Enemy
 6  class Enemy(Entity):
 7      def __init__(self,monster_name,pos,groups,obstacle_sprites,damage_player):
33          # Player Interaction
34          self.can_attack = True
35          self.attack_time = None
36          self.attack_cooldown = 400
37          self.damage_player = damage_player
38
39          # Invincibilty Timer
40          self.vulnerable = True
41          self.hit_time = None
42          self.invincibility_duration = 300
```

The parameter damage_player is added to the Enemy class' init method so the damage_player method within the Level class can be accessed by the enemy objects.

The can_attack, attack_time and attack_cooldown attributes will be used to help stop the enemy from attacking the enemy infinitely. They are set as their default values. The damage_player attribute is for changing the player's health while the level is running.

These attributes are used when the enemy is being attacked. The vulnerable attribute is a flag, and all the attributes are set to their default values.

```
enemy.py > Enemy
 6  class Enemy(Entity):
64      def get_status(self,player):
79          if distance <= self.attack_radius and self.can_attack:
80              if self.status[:6] != 'attack':
81                  self.frame_index = 0
82              self.status = f'attack_{self.status_direction}'
83          elif distance <= self.notice_radius:
84              self.status = f'move_{self.status_direction}'
85          else:
86              self.status = f'idle_{self.status_direction}'
```

The can_attack flag is added to the get_status method since the enemy should only be able to attack when it is True. The frame_index is set to 0 if the beginning of the status wasn't 'attack' so that the attack animation is started from the beginning and fully completed.

```
enemy.py > Enemy
 6  class Enemy(Entity):
88      def actions(self,player):
89          if self.status[:6] == 'attack':
90              self.attack_time = pygame.time.get_ticks()
91              self.damage_player(self.attack_type)
92          elif self.status[:4] == 'move':
93              self.direction = self.get_player_distance_direction(player)[1]
94          else:
95              self.direction = pygame.math.Vector2()
```

The print statement was replaced by setting the attack_time and the damage_player method. This is so it's possible to apply the cooldown to the enemy's attacks.

```
enemy.py > Enemy
 6  class Enemy(Entity):
97      def animate(self):
98          animation = self.animations[self.status]
99
100         self.frame_index += self.animation_speed
101         if self.frame_index >= len(animation):
102             if self.status[:6] == 'attack':
103                 self.can_attack = False
104             self.frame_index = 0
105
106         self.image = animation[int(self.frame_index)]
107         self.rect = self.image.get_rect(center=self.hitbox.center)
108
109         if not self.vulnerable:
110             alpha = self.wave_value()
111             self.image.set_alpha(alpha)
112         else:
113             self.image.set_alpha(255)  # Full opacity
```

The status is checked to see if the enemy is currently attacking and then sets can_attack to False if it is. This stops the enemy from continuously attacking the player.

This checks if the enemy is vulnerable, meaning that the enemy has been hit by the player, and changes the opacity of the player's image using the wave_value method from the Entity class. This makes the enemy flicker when they are hit. If the enemy isn't vulnerable, their opacity is set back to the max (255).

enemy.py > Enemy

```
  6   class Enemy(Entity):
115       def cooldowns(self):
116           current_time = pygame.time.get_ticks()
117           if not self.can_attack:
118               if (current_time - self.attack_time) >= self.attack_cooldown:
119                   self.can_attack = True
120
121           if not self.vulnerable:
122               if current_time - self.hit_time >= self.invincibility_duration:
123                   self.vulnerable = True
```

The cooldowns method handles all the enemy's cooldowns for both attacking and vulnerability. The current time is obtained using the time that the game has been running for. The can_attack is checked if it's False and then checks if the current time subtracted by the attack time (how long it has been since the enemy has attacked) is greater than or equal to the attack cooldown time. If it is then the can_attack flag is reverted to True, allowing the enemy to attack again.

The enemy is checked to see if it isn't vulnerable and then the time elapsed from when the enemy turned invulnerable (when they were attacked) is checked against the invincibility_duration to see if it's greater or equal to. If it is, the vulnerable flag is set back to True.

enemy.py > Enemy

```
  6   class Enemy(Entity):
125       def get_damage(self,player):
126           if self.vulnerable:
127               self.direction = self.get_player_distance_direction(player)[1]
128               self.health -= weapon_data[player.weapon]['damage']
129               self.hit_time = pygame.time.get_ticks()
130               self.vulnerable = False
```

The get_damage method checks if the vulnerable attribute is True (meaning the enemy got attacked). If it is, the direction of the player is updated, and the enemy's health is decreased by the weapon's damage (which is just 1). The hit_time is also set for the cooldown of the vulnerability, and the vulnerable attribute is set to False to stop the player from infinitely attacking the enemy.

enemy.py > Enemy

```
  6   class Enemy(Entity):
132       def check_death(self):
133           if self.health <= 0:
134               self.kill()
135
136       def hit_reaction(self):
137           if not self.vulnerable:
138               self.direction *= -self.resistance
139
140       def update(self):
141           self.hit_reaction()
142           self.move(self.speed)
143           self.animate()
144           self.cooldowns()
145           self.check_death()
```

The check_death method simply checks if the enemy's health is less or equal to 0 and destroys the enemy object if it is.

The hit_reaction method just checks if the enemy isn't vulnerable (it has been attacked) and moves the enemy back by its resistance. This makes it look like the enemy is being pushed back when the player attacks it and makes it slightly more difficult for the player to attack the enemies.

These methods are added to the update method because they need to be checked each time the enemy is updated.

entity.py > ...

```
  4   class Entity(pygame.sprite.Sprite):
 38       def wave_value(self):
 39           value = sin(pygame.time.get_ticks())
 40           if value >= 0:
 41               return 255
 42           else:
 43               return 0
```

The wave_value method is placed in the Entity class since both the Enemy class and Player class need to use it and it can be shared. The value is based on the sin value of the time which means that it will change according to the shape of the sin curve. This required sin to be imported from the math library. While the value is positive, 255 is returned, otherwise 0 is returned. When applied to the opacity of images, it allows a flickering effect to be achieved since the opacity changes between 255 and 0.

player.py > Player

```
  7   class Player(Entity):
  8       def __init__(self,pos,groups,obstacle_sprites,interactable_sprites,fall_sprites,create_attack,destroy_attack,notice):
 59           # Damage Timer
 60           self.vulnerable = True
 61           self.hurt_time = None
 62           self.invulnerability_duration = 500
```

These attributes are for the vulnerability of the player and function the same as the enemy's attributes for its vulnerability.

player.py > Player

```
  7   class Player(Entity):
186       def cooldowns(self):
187           current_time = pygame.time.get_ticks()
188
189           if self.attacking:
190               if (current_time-self.attack_time) >= self.attack_cooldown + weapon_data[self.weapon]['cooldown']:
191                   self.attacking = False
192                   self.destroy_attack()
193
194           if not self.vulnerable:
195               if (current_time - self.hurt_time) >= self.invulnerability_duration:
196                   self.vulnerable = True
```

The vulnerability cooldown is added to the player's cooldowns method. It functions the exact same as the vulnerability cooldown for the enemy.

player.py > Player

```
7   class Player(Entity):
251     def animate(self):
252         animation = self.animations[self.status]
253
254         # Looping over frame index
255         self.frame_index += self.animation_speed
256         if self.frame_index >= len(animation):
257             self.frame_index = 0
258
259         self.image = animation[int(self.frame_index)]
260         self.rect = self.image.get_rect(center = self.hitbox.center)
261
262         # Flicker
263         if not self.vulnerable:
264             alpha = self.wave_value()
265             self.image.set_alpha(alpha)
266         else:
267             self.image.set_alpha(255)
```

The flicker animation is also added to the player's animation, and it works the same as the enemy's flicker animation.

level.py > Level

```
10  from particles import AnimationPlayer
```

The AnimationPlayer had to be imported to the level file so it could be used.

level.py > Level

```
13  class Level:
14      def __init__(self):
25          # Attack Sprites
26          self.current_attack = None
27          self.attack_sprites = pygame.sprite.Group()
28          self.attackable_sprites = pygame.sprite.Group()
```

The attack_sprites and attackable_sprites attributes were added so they could store the sprites that could be attacked and the sprites that can attack/cause damage (the player's weapon).

level.py > Level

```
13  class Level:
14      def __init__(self):
40          # Particles
41          self.animation_player = AnimationPlayer()
```

The animation_player attribute contains the AnimationPlayer class so it can be used easily within the Level class.

level.py > Level

```
13  class Level:
129     def player_attack_logic(self):
130         if self.attack_sprites:
131             for attack_sprite in self.attack_sprites:
132                 collision_sprites = pygame.sprite.spritecollide(attack_sprite,self.attackable_sprites,False)
133                 if collision_sprites:
134                     for target_sprite in collision_sprites:
135                         target_sprite.get_damage(self.player)
136
137     def damage_player(self,attack_type):
138         if self.player.vulnerable:
139             self.player.change_health('lose')
140             self.player.vulnerable = False
141             self.player.hurt_time = pygame.time.get_ticks()
142             self.animation_player.create_particles(attack_type,self.player.rect.center,[self.visible_sprites])
```

The player_attack_logic method deals with the player attacking the enemies. It first checks if there are attack_sprites (the player's weapon) and checks if it has collided with an attackable sprite. If it has, the attackable sprite should take damage. This may be slightly altered later in development.

The damage_player method deals with when the player is harmed. It first checks to make sure the enemy can only damage the player while they are vulnerable. The player's health is decreased using the change_health method, the player becomes invulnerable and the time when they were hit is stored as the hurt_time attribute. The correct particle is also displayed based on the enemy's attack type, indicating the player being damaged.

level.py > Level

```
13  class Level:
154     def run(self):
155         # Update + draw game
156         self.visible_sprites.custom_draw(self.player)
157         self.visible_sprites.update()
158         self.visible_sprites.enemy_update(self.player)
159         self.player_attack_logic()
160         self.ui.display(self.player)
161         self.reload_map()
```

The player_attack_logic method is added to the run method since it needs to be checked every main loop of the game continuously.

particles.py > ...

```python
import pygame
from support import import_folder

class AnimationPlayer:
    def __init__(self):
        self.frames = {
            # Attacks
            'slash': import_folder('../graphics/particles/slash'),
            'claw': import_folder('../graphics/particles/claw'),
            'ice': import_folder('../graphics/particles/ice'),
            'stomp': import_folder('../graphics/particles/stomp')
        }

    def create_particles(self,animation_type,pos,groups):
        animation_frames = self.frames[animation_type]
        ParticleEffect(pos,animation_frames,groups)
```

Pygame is imported like usual, and the import_folder function is also imported since the folders for each animation needs to be imported.

The AnimationPlayer class holds the different animations and can create the effects. This is stored separate to the ParticleEffect class since all the data can be imported once instead rather than every time the particles are created. The different animations are stored in the frames attribute which is a dictionary for each particle effect and its images.

The create_particles method is used to display the particle effects. It uses the animation_frames for the current animation type/action. A ParticleEffect object is then created using the data.

particles.py > ...

```python
class ParticleEffect(pygame.sprite.Sprite):
    def __init__(self,pos,animation_frames,groups):
        super().__init__(groups)
        self.frame_index = 0
        self.animation_speed = 0.15
        self.frames = animation_frames
        self.image = self.frames[self.frame_index]
        self.rect = self.image.get_rect(center=pos)

    def animate(self):
        self.frame_index += self.animation_speed
        if self.frame_index >= len(self.frames):
            self.kill()
        else:
            self.image = self.frames[int(self.frame_index)]

    def update(self):
        self.animate()
```

The ParticleEffect class is where the actual particle sprite is created. The attributes for the class are all just for animating and displaying the effect.

The animate method goes through all the animation frames within the effect and once it reaches the end, the particle sprite is killed.

The update method makes sure that the particle animates itself while the game runs.

While developing, I also reduced the raccoon's attack radius from 60 to 40 since I thought it was too large, and the raccoon could easily attack the player.

Now the raccoon enemy can attack properly, although it doesn't take the correct amount of hits to kill the raccoon. However, this will be changed when I develop the player and enemy interactions. Furthermore, the attack code will have to be altered when creating the attack for the other enemies since they aren't the same and the code only works for the raccoon currently.

GIF (10) Raccoon attack

Acceptance Test 16: Check that the raccoon enemy's attacks are working properly.

Inputs	Outputs	Validity
Vulnerable player within raccoon attack radius	Raccoon attack is displayed with the correct effect and attack animation, the player's health depletes and their image flickers while they become invulnerable.	True
Invulnerable player within raccoon attack radius	Raccoon doesn't attack the player since it can't until the player becomes vulnerable again.	True
Claw particle effect	Particle effect is correctly displayed on the player.	True
Attack animation	Correct attack animation for the raccoon is displayed.	True
Player within raccoon attack radius while raccoon's can_attack attribute is False	Raccoon can't attack the player until their attack cooldown ends.	True

Acceptance Test 19: Check that the enemies can die properly.

Inputs	Outputs	Validity
Enemy health above 0	Enemy should function as normal.	True
Enemy health below 0	Enemy sprite should be deleted.	True

Level 1 Feedback

Due to time constraints, I can't continue developing Hunted Horizon and so I'm doing the first level's feedback now. Originally, I would've gotten the clients' feedback after I completed the code for each section of the game (level 1, level 2, level 3, arcade mode, boss battle, overall) and then implemented the feedback to improve the game while I'm developing other sections.

My main client gave me the feedback for the first level as it currently is:

He struggled when jumping between platforms in the first level and suggested implementing a **shadow under the player when they jump**. This would mean that the user has a better understanding of how and where the player's jump will end and make it easier to control the player – improving the gameplay.

He also suggested to **change the layout of the map or add detail** to it because it looked desolate. This would make the game more appealing to look at and therefore play.

He suggested adding a **visual output when enemies died** like 'death particles' to make the game seem even more professional and visually appealing since, otherwise, it looks slightly odd when the enemy just disappears.

These are all things I could implement though the jump shadow would take some time to implement due to the logic required in comparison to the other two improvements. Any other comments made by my main client were things that I already intended to implement.

Another problem that I had discovered that caused my client to struggle when completing the level was an **issue with the jumping and falling**. Sometimes, whenever the player would jump and die and then respawn, you could try jumping again but it would always take you to a specific spot instead of properly jumping. The only way to stop that from happening was moving the player up and down before trying to jump again. This is something that I needed to fix as it would reduce the playability of the game and the player would keep dying by accident.

Testing

Overall Acceptance Test

Test No.	Test Description	Inputs (Examples)	Expected Outcome	Actual Outcome
1	Check basic WASD movement and the diagonal movement.	**Valid**: 'W' key moves player up, 'S' key moves player down, 'A' key moves player left, 'D' key moves player right. **Invalid**: Any other key on the keyboard.	**Valid**: Player moves according to the valid input at a constant speed. **Invalid**: The other key pressed doesn't do anything.	As expected. Prototype Evidence (00:00-03:13)(whole video)
2	Check player camera.	**Valid**: player object **Invalid**: display_surface size	**Valid**: The camera (what the user can view) follows the player so the player is always in the middle of the screen. The player is also given depth, so it looks like the player is walking in front of or behind objects. **Invalid**: A syntax error is generated, and the game can't be run.	As expected. Prototype Evidence(whole video)

3	Check the main map and the objects within the map.	**Valid**: main map assets **Invalid**: player assets	**Valid**: The main map correctly appears when the game is run, and the player can correctly collide with objects. **Invalid**: A syntax error is generated, and the game can't be run.	As expected. Prototype Evidence(00:00-03:13)
4	Check the main map's barriers.	**Valid**: barrier tiles **Invalid**: grass tiles	**Valid**: There are invisible barriers around the map that the player can't walk past. **Invalid**: The main map's edges visibly look like grass and the player can walk past them.	As expected. Prototype Evidence (00:08)(00:40)(01:30-01:43)(02:20-02:37)(03:00)
5	Check player jumping movement.	**Valid**: 'Space' key **Invalid**: Any other key on the keyboard	**Valid**: The player jumps upwards, can move midair and fall back down imitating gravity. **Invalid**: The other key pressed doesn't do anything if it doesn't already have a use.	Mainly as expected although with few glitches that needed fixing. Prototype Evidence(03:48-04:09)(04:24)(04:58-06:50)(07:18-08:11)(08:18-08:38)(08:49)
6	Check that player can attack, block and interact using placeholders.	**Valid**: 'J' key for attacking, 'K' key for blocking and 'L' key for interacting **Invalid**: Any other key on the keyboard	**Valid**: A placeholder output is run when the corresponding key is pressed. **Invalid**: The other key pressed doesn't do anything if it doesn't already have a use.	As expected. Prototype Evidence(whole video)
7	Check player animations are working for all current movement.	**Valid**: 'up_0.png' **Invalid**: ''	**Valid**: The player's movements are all accompanied with the correct animation. **Invalid**: A syntax error is generated because it can't find '' and the game can't be run.	As expected. Prototype Evidence(whole video)
8	Check that the weapon is working properly.	**Valid**: 'sword' **Invalid**: ''	**Valid**: The player's weapon correctly appears when the user attacks. **Invalid**: A syntax error is generated because	As expected. Prototype Evidence(03:30-03:45)(03:49-04:00)(04:19-

			it can't find '' and the game can't be run.	04:50)(06:50)(07:19-08:15)
9	Check that blocking is working properly.	**Valid**: 'K' key **Invalid**: Any other key on the keyboard	**Valid**: The player is shown blocking which stops them from being damaged when attacked but they can't move while blocking. **Invalid**: The other key pressed doesn't do anything if it doesn't already have a use.	Not implemented.
10	Check that interacting and interactable objects are working properly.	**Valid**: 'L' key **Invalid**: Any other key on the keyboard	**Valid**: If the player is near an interactable object, the object is interacted with. If they aren't near an interactable object, then nothing happens. **Invalid**: The other key pressed doesn't do anything if it doesn't already have a use.	As expected. Prototype Evidence(00:00-03:17)
11	Check that the HUD is working properly.	**Valid**: levelHUD **Invalid**: HUD	**Valid**: The level or main map HUD should display. The HUD should be updated using data such as the player's health when there are changes. **Invalid**: A syntax error is generated since the HUD variable isn't defined/doesn't exist and the game can't be run.	As expected. Prototype Evidence(03:15-09:01)
12	Check that the first level map loads properly.	**Valid**: level_map1 **Invalid**: map	**Valid**: The first level's map and its objects should load onto the user's screen after deleting the main map and its objects to reduce the game from slowing down. The player should also be placed onto the map properly. **Invalid**: A syntax error is generated since the map variable isn't	As expected. Prototype Evidence(03:10-09:01)

			defined/doesn't exist and the game can't be run.	
13	Check that falling works properly.	**Valid**: gravity **Invalid**: fall	**Valid**: The player should be able to fall off the map appropriately with the correct animation and logic if the user moves the player off the walkable areas of the map. After falling, the player should also be repositioned properly with less health than they did prior to falling. **Invalid**: A syntax error is generated since the fall variable isn't defined/doesn't exist and the game can't be run.	Mainly as expected although with few glitches that needed fixing. Prototype Evidence(03:48-04:09)(04:24)(04:58-06:50)(07:18-08:11)(08:18-08:38)
14	Check that enemies appear properly.	**Valid**: image **Invalid**: object	**Valid**: The correct type of enemy should appear in the level at their specific position when the level starts. **Invalid**: A syntax error is generated since the object variable isn't defined/doesn't exist and the game can't be run.	As expected. Prototype Evidence(03:10-09:01)
15	Check that enemies move properly.	**Valid**: status **Invalid**: attack_type	**Valid**: The enemy should be still until the player is near, then the enemy should follow the player at a constant speed. **Invalid**: This would be a logical error, and the enemies may not move properly since their attack_type isn't required within their movement.	As expected. Prototype Evidence(03:30-08:15)
16	Check that the raccoon enemy's attacks are	**Valid**: attack_type **Invalid**: enemy_eat	**Valid**: The raccoon enemy should be able to do any of its attacks to the player with the	As expected. Prototype Evidence(03:30-03:45)(03:49-

	working properly.		correct cooldowns and type of attack. The attacks should also be deleted once it has happened. **Invalid**: A syntax error is generated since the enemy_eat variable isn't defined/doesn't exist and the game can't be run.	04:00)(04:19-04:50)(06:50)(07:19-08:15)
17	Check that the cyclops enemy's attacks are working properly.	**Valid**: attack_time **Invalid**: enemy_exp	**Valid**: The cyclops enemy should be able to do any of its attacks to the player with the correct cooldowns and type of attack. The attacks should also be deleted once it has happened. **Invalid**: A syntax error is generated since the enemy_exp variable isn't defined/doesn't exist and the game can't be run.	Not implemented.
18	Check that the slime enemy's attacks are working properly.	**Valid**: attack_cooldown **Invalid**: enemy.hitbox	**Valid**: The slime enemy should be able to do any of its attacks to the player with the correct cooldowns and type of attack. The attacks should also be deleted once it has happened. **Invalid**: A logical error may occur by using the enemy's hitbox instead of the player's hitbox and may mean that the enemy can hurt the player just by attacking randomly.	Not implemented.
19	Check that the enemies can die properly.	**Valid**: health **Invalid**: status	**Valid**: Once an enemy's health reaches zero, they must be deleted. **Invalid**: A syntax error is generated since a string input wouldn't	As expected. Prototype Evidence(03:30-03:45)(03:49-04:00)(04:19-04:50)(06:50)(07:19-08:15)

			be required to check this.	
20	Check that the enemies can fall properly.	**Valid**: status **Invalid**: damage_player	**Valid**: The enemy should fall off the map using the correct logic if the enemy moves off the walkable areas of the map. **Invalid**: A syntax error may be generated because the damage_player variable may not fit any of the inputs' data types within checking this.	Not implemented.
21	Check that the enemy animations are all working properly.	**Valid**: animations **Invalid**: resistance	**Valid**: All the enemies' movements should be accompanied with the corresponding animation. **Invalid**: This may cause a syntax or logical error depending on where the resistance variable is used. Overall, this would either make the game crash or hard to understand since the movements aren't properly conveyed.	Somewhat as expected but not yet fully implemented. Prototype Evidence(03:30-03:45)(03:49-04:00)(04:19-04:50)(06:50)(07:19-08:15)
22	Check that enemy-player interactions are working properly.	**Valid**: can_attack **Invalid**: display_surface	**Valid**: All the interactions between the enemy and player should work as intended: attacking, blocking, resistance, killing, etc. **Invalid**: A syntax error will be generated since there isn't a need for a variable of data type Surface in this section of code.	Somewhat as expected but not yet fully implemented. Prototype Evidence(03:30-03:45)(03:49-04:00)(04:19-04:50)(06:50)(07:19-08:15)
23	Check that the death screen works properly.	**Valid**: display_surface **Invalid**: 3	**Valid**: Once the player dies, a death screen should be displayed which allows the player to return to the main map.	Not implemented.

			Invalid: A syntax error would be generated, and the game wouldn't be able to be run.	
24	Check that coins appear and work properly.	**Valid**: hitbox **Invalid**: weapon	**Valid**: The player should be able to see coins within levels that disappear once the player touches them because the player has taken the coin. The coin should be deleted once it has been found and the player's coin total should increase. **Invalid**: A syntax error would be generated because there wouldn't be a string input in this part of the code, and the game wouldn't run.	Not implemented.
25	Check that the user can open a menu and interact with it.	**Valid**: 'M' key **Invalid**: Any other key on the keyboard	**Valid**: When the user presses the 'M' key, the menu should open. While the menu is open, the only keys that do something when pressed are the WASD, M and L keys (A and D only when selecting audio). They can use W and S to switch between options on the menu and L to select an option. The currently selected option must be highlighted in a different colour. To escape the menu, the user can press M again. **Invalid**: The other key pressed doesn't do anything if it doesn't already have a use.	Not implemented.
26	Check that the menu pauses	**Valid**: game_paused **Invalid**: hitbox	**Valid**: While the menu is open, the game should be paused in	Not implemented.

	the game properly.		the background so the user can access the menu. When the user closes the menu, the game should be un-paused from where it left off from. **Invalid**: A syntax error would be generated since the menu can't have a hitbox as it's not a sprite and the game wouldn't run.	
27	Check that the controls part of the menu works properly.	**Valid**: 'controls.png' **Invalid**: ''	**Valid**: When the player selects the controls through the menu, there should be an image of the controls displayed as well as a back button so the user can return to the main menu. **Invalid**: A syntax error is generated because it can't find '' and the game can't be run.	Not implemented.
28	Check that the first level can end properly.	**Valid**: complete **Invalid**: display_surface	**Valid**: Once the player reaches the end of the level, the level should end and indicate it has ended. **Invalid**: A syntax error will be generated since there isn't a need for a variable of data type Surface in this section of code.	Not implemented.
29	Check that the timer works properly.	**Valid**: start_time **Invalid**: hours	**Valid**: When the player starts a level, the time should be note and when they reach the end of the level, the time should be noted again. The end time is subtracted from the start time and that is the duration. The duration is then converted into minutes, seconds and milliseconds and made	Not implemented.

			into a string which is returned. The time should also not go above an hour, if it does, the time should stop at an hour and return that as the time. **Invalid**: A syntax error is generated since the hours variable isn't defined/doesn't exist and the game can't be run.	
30	Check that reaching the end of a level has a working completion screen.	**Valid**: time **Invalid**: enemies_killed	**Valid**: The completion screen should show the time taken, coins gained from the level and a placeholder for any trophies gained. It should also have an option to return to the main map and a placeholder for restarting the level. This screen should appear once the player reaches the end of a level. **Invalid**: A syntax error is generated since the enemies_killed variable isn't defined/doesn't exist and the game can't be run.	Not implemented.
31	Check that the trophies for the first level work properly.	**Valid**: earnable_trophies **Invalid**: coins_earned	**Valid**: If the user has completed a specific trophy's task that is achievable within the first level, the user should gain that trophy. **Invalid**: A syntax error is generated since there isn't a need for an integer variable in this part of the code and the game wouldn't run.	Not implemented.

32	Check that the arcade mode map loads properly.	**Valid**: arcade_map **Invalid**: level_map1	**Valid**: The arcade map and its objects should load onto the user's screen after deleting the main map and its objects to reduce the game from slowing down. The player should also be placed onto the map properly. **Invalid**: A logical error would occur because of the mismatch of the maps which means that the game would still run but not as intended.	Not implemented.
33	Check that enemies can be summoned in waves in the arcade mode.	**Valid**: wave **Invalid**: coins_earned	**Valid**: The waves should start from one and increment when all the enemies from the previous wave have been killed. When the wave increments, another set of random enemies should spawn. This should repeat until the player dies. **Invalid**: This variable wouldn't be used yet and may cause a logical error if it was input into this section of the code.	Not implemented.
34	Check that the player's death in arcade mode results in a completion screen instead.	**Valid**: coins_earned **Invalid**: duration	**Valid**: When the player dies within arcade mode, instead of the death screen displayed in the levels, a completion screen showing the number of waves completed, any trophies gained and how many coins the user has earned from arcade mode. The screen will also have a return to main	Not implemented.

			map option and a placeholder to replay the arcade mode. **Invalid**: This would cause a logical error because the arcade mode wouldn't have a timer active while the user plays it.	
35	Check that the arcade mode HUD works properly.	**Valid**: arcadeHUD **Invalid**: HUD	**Valid**: The arcade HUD should display. The HUD should be updated using data such as the player's health and the wave number when there are changes. **Invalid**: A syntax error is generated since the HUD variable isn't defined/doesn't exist and the game can't be run.	Not implemented.
36	Check that the boss level map loads properly.	**Valid**: boss_map1 **Invalid**: arcade_map	**Valid**: The boss map and its objects should load onto the user's screen after deleting the main map and its objects to reduce the game from slowing down. The player should also be placed onto the map properly. **Invalid**: A logical error would occur because of the mismatch of the maps which means that the game would still run but not as intended.	Not implemented.
37	Check that the boss' attacks are working properly.	**Valid**: attack_type **Invalid**: ''	**Valid**: The boss' attack should be done periodically and across the specific area on the map. **Invalid**: A syntax error is generated because '' doesn't exist.	Not implemented.
38	Check that the boss can	**Valid**: attack_type **Invalid**: can_summon	**Valid**: The boss should summon enemies	Not implemented.

	summon other enemies properly.		randomly while attacking within the boss area. **Invalid**: A syntax error is generated since the can_summon variable isn't defined/doesn't exist and the game can't be run.	
39	Check that the buttons work properly in the boss battle.	**Valid**: 'L' key **Invalid**: 'M' key	**Valid**: The buttons on the boss map should be able to be interacted with, one by one, by the player after a certain time has passed. When pressed, the buttons harm the boss and can't be pressed again. **Invalid**: The menu would open for the player instead of interacting with any of the buttons.	Not implemented.
40	Check that the boss' animations work properly.	**Valid**: animations **Invalid**: attack_cooldown	**Valid**: All the enemies' movements should be accompanied with the corresponding animation. **Invalid**: This may cause a syntax or logical error depending on where attack_cooldown is used. Overall, this would either make the game crash or hard to understand since the movements aren't properly conveyed.	Not implemented.
41	Check that the boss battle HUD works properly.	**Valid**: bossHUD **Invalid**: HUD	**Valid**: The boss battle HUD should display. The HUD should be updated using data such as the player's and boss' health when there are changes. **Invalid**: A syntax error is generated since the HUD variable isn't	Not implemented.

			defined/doesn't exist and the game can't be run.	
42	Check that defeating the boss ends the level and shows a completion screen.	**Valid**: health (boss) **Invalid**: enemies_killed	**Valid**: Once the boss' health is depleted and the boss' death animation finishes, the completion screen should appear. The completion screen should be like the level completion screen except the title would be different and the placeholder option would be a restart boss battle option. **Invalid**: A syntax error is generated since the enemies_killed variable isn't defined/doesn't exist and the game can't be run.	Not implemented.
43	Check that the trophy area on the main map works properly.	**Valid**: trophies_gained **Invalid**: trophy_desc	**Valid**: At the trophy area, the player can interact to see how many trophies they've gained and how many they haven't earned. **Invalid**: A syntax error is generated since the trophy_desc variable isn't defined/doesn't exist and the game can't be run.	Not implemented.
44	Check that the second level loads and works properly.	**Valid**: level_map2 **Invalid**: boss_map1	**Valid**: The second level map and its objects should load onto the user's screen after deleting the main map and its objects to reduce the game from slowing down. The player should also be placed onto the map properly. **Invalid**: A logical error would occur because	Not implemented.

			of the mismatch of the maps which means that the game would still run but not as intended.	
45	Check that the third level loads and works properly.	**Valid**: level_map3 **Invalid**: level_map2	**Valid**: The third level map and its objects should load onto the user's screen after deleting the main map and its objects to reduce the game from slowing down. The player should also be placed onto the map properly. **Invalid**: A logical error would occur because of the mismatch of the maps which means that the game would still run but not as intended.	Not implemented.
46	Check that the NPCs load properly.	**Valid**: NPC_name **Invalid**: lines	**Valid**: The NPCs should appear on the main map and the player shouldn't be able to walk over them. **Invalid**: A syntax error is generated since the lines variable isn't used in this part of the code and it doesn't match with any of the data types used in this section of code.	Not implemented.
47	Check that NPC interaction works properly.	**Valid**: lines **Invalid**: display_surface	**Valid**: The player should be able to interact with the NPC when they are near. The NPC should say a few lines of dialogue at the bottom of the screen and the player can press interact again to get to the next line. The dialogue only appears for a while and is then deleted. If the user	Not implemented.

			continues the dialogue while another line is running, then the previous line is deleted and replaced with the new one. **Invalid**: A syntax error will be generated since there isn't a need for a variable of data type Surface in this section of code.	
48	Check that player progress is saved properly.	**Valid**: trophies_gained **Invalid**: enemies_killed	**Valid**: After the user has completed a level/the arcade mode/a boss level, that data is saved for when the player returns to the game. **Invalid**: A syntax error is generated since the enemies_killed variable isn't defined/doesn't exist and the game can't be run.	Not implemented.
49	Check that players can return to the main map through the menu properly.	**Valid**: 'L' key **Invalid**: Any other key on the keyboard when trying to select return to main map on the menu	**Valid**: The player can return to the main map through the menu but if they are in a level/arcade mode, they lose their progress. **Invalid**: The other key pressed doesn't do anything if it doesn't already have a use.	Not implemented.
50	Check that players can restart levels/arcade mode/boss battles.	**Valid**: 'L' key **Invalid**: 'M' key	**Valid**: All the restart placeholders on the completion screens should now work. Restarting from a completion screen doesn't mean you lose your previous data. **Invalid**: The 'M' key shouldn't do anything while the completion screen is active.	Not implemented.

51	Check that the start screen works properly.	**Valid**: 'start.png' **Invalid**: ''	**Valid**: When the game is loaded, the first thing the user must see is a start screen from which the user can then start playing by pressing the spacebar. **Invalid**: A syntax error is generated because it can't find '' and the game can't be run.	Not implemented.
52	Check that the loading screens work properly.	**Valid**: 'loading.png' **Invalid**: ''	**Valid**: In between different modes there should be a loading screen displayed while the game deletes the previous map and loads the new one. **Invalid**: A syntax error is generated because it can't find '' and the game can't be run.	Not implemented.
53	Check that trophies work properly.	**Valid**: earnable_trophies **Invalid**: weapon	**Valid**: Trophies can be gained by the player once they complete the task for that trophy. Once they have gained that trophy, they can't gain it again. **Invalid**: A logical error may occur that may confuse users on the trophies.	Not implemented.
54	Check that completion screens display information properly.	**Valid**: coins **Invalid**: enemies_killed	**Valid**: The completion screen should correctly display the information for the specific game mode's completion screen. If there is no data then it should display none instead. **Invalid**: A syntax error is generated since the enemies_killed variable isn't defined/doesn't exist and the game can't be run.	Not implemented.

55	Check that levels can be appropriately blocked.	**Valid**: complete **Invalid**: level_map2	**Valid**: The levels should be played in linear, and they only unlock once other levels have been completed. This is also true for the boss battle. If the player hasn't met the requirements, then the entrance to the level is blocked. **Invalid**: A syntax error is generated since the data type wouldn't match any of the code in this section.	Not implemented.
56	Check that the main map can update to show available levels.	**Valid**: main_map **Invalid**: level_map1	**Valid**: The main map should alter slightly to show available levels and non-available levels for the user. **Invalid**: A logic error would occur since the alteration of the map may not work as intended.	Not implemented.
57	Check that player audio works properly.	**Valid**: weapon_attack_sound **Invalid**: ''	**Valid**: Most actions made by the player should be accompanied by a matching sound. **Invalid**: A syntax error is generated because it can't find '' and the game can't be run.	Not implemented.
58	Check that enemy audio works properly.	**Valid**: death_sound **Invalid**: weapon_attack_sound	**Valid**: Most actions made by an enemy should be accompanied by a matching sound. **Invalid**: The game will still run but this would be a logic error since the sounds wouldn't match up.	Not implemented.
59	Check that system audio works properly.	**Valid**: coin_sound **Invalid**: 'coin.png'	**Valid**: Any other actions must be accompanied by a matching sound.	Not implemented.

			Invalid: A syntax error is caused since the data type wouldn't match up in this section of code.	
60	Check that game music works properly.	**Valid**: main_sound **Invalid**: death_sound	**Valid**: There should be music for each of the levels in the game that loops while the user plays. **Invalid**: The game will still run but this would be a logic error since the audio wouldn't match up.	Not implemented.
61	Check that the audio settings menu works properly.	**Valid**: WASD keys **Invalid**: Any other key on the keyboard	**Valid**: The audio levels are preset for the user however they can change according to what the user has set. **Invalid**: The other key pressed doesn't do anything if it doesn't already have a use.	Not implemented.
62	Check that player progress can be loaded properly.	**Valid**: player_progress **Invalid**: display_surface	**Valid**: The game should check if the user has saved progress on the game already and load it if that is the case. **Invalid**: A syntax error will be generated since there isn't a need for a variable of data type Surface in this section of code.	Not implemented.

The overall post development testing is within the Prototype Evidence video.

Usability Testing

Because only two of the main usability features was implemented, I will test and evaluate them.

HUD:

> I tested the HUD for the first level by altering the player's health and there are no issues, and the HUD updates as expected. Because the player's health can't go below 0, this also means that there aren't any File Not Found errors since the health will remain in the range of the different HUD images. The HUD also correctly appears when the first level is loaded. This is all shown in the video evidence below.

GIF (11) Usability testing HUD

Notice Bubble:

I tested the notice bubble by ensuring that it works near every interactable object. The only issue was with the NPCs since the notice bubble could go behind them instead of always being on top. This would've been something I'd fix when implementing the NPCs.

GIF (12) Usability testing notice bubble

Evaluation

The video 'Prototype Evidence' shows what a playthrough of the prototype looks like.

Success Criteria Targets

Requirement	Justification	Met?
At least three playable levels	This is so there can be a variety in the level designs and the situations within the levels which makes the game more interesting and is what the stakeholders want.	Partially met (first level nearly properly playable).
One mini boss fight	This adds difficulty to the game as well as variety in gameplay which is what the stakeholders want.	Not met.
Arcade mode	This allows players to try a different game mode and provides variety to the gameplay. Variety is what the stakeholders all wanted as they voted that they prefer different game modes, and the client wanted to play something new.	Not met.
Main map	This makes the game flow more continuously rather than being more disconnected since it makes the game feel like it has its own world rather than the game modes being separate. This also allows for world-building story which makes the game more story-driven and is what the client prefers in games. The questionnaire participants also preferred having a story.	Met.
At least three working enemies	Having at least three enemies makes the gameplay more varied since they can attack the player in different ways, and it would require the player to react differently to each of the enemies. This would improve gameplay which is an aspect that the stakeholders found very important.	Partially met (raccoon enemy works).
Player WASD movement and attack/block	This is to ensure that the player movement is working correctly so that the player can play the game without any worries. This also affects the gameplay a lot so it is a vital requirement since otherwise the player can't play the game properly.	Partially met (blocking wasn't implemented).
Player health	This is to create difficulty for the player otherwise if the player had infinite health, there wouldn't be much of a challenge to complete levels. Most of the stakeholders expressed that the game should be challenging if it's a platformer game.	Met.
Collectibles	This is to make the game more fun for the player as they have incentive to play the game more to gain more collectibles. This also ties into having trophies in the game which is something that many stakeholders preferred.	Not met.
NPCs and NPC dialogue	This adds a story to the game through dialogue which is something that all stakeholders preferred to have.	Partially met (NPCs appear on main map).
Trophy tracking	Having trophies allows players to challenge themselves to obtain trophies but it only challenges players that want to obtain the trophies. It also gives	Not met.

	the players a sense of achievement which may make the game feel more fun and is supported by the questionnaire.	
Working menu	This is to allow the player to pause the game, understand the controls, change settings as well as leave what they are currently playing to go to the main map. A working menu is usually in every game and is something that players would most likely expect.	Not met.
Save mechanism	This is so players can keep track of their progress of the game and their trophies. This means that the player can come back to the game at any time if they save before they quit which is in line with the original audience who aren't able to play games enough.	Not met.
Title screen	The screen would show the name of the game and tell the player to click a button to start the game. This is so the player can have a small menu before they play the game to improve the gameplay by allowing the player to settle into the game easier and because many games have this feature.	Not met.
Death screen	This is so that the player knows they have used all their lives and will exit them from what they were doing (a level or arcade mode) to the main map. The stakeholders wished for a challenging game so this will also add challenge and stop the player from continuing where they left off.	Not met.
Loading screens	This helps the player understand that the game is loading and that they've properly interacted with whatever they wanted to interact with (e.g. starting a level, starting the arcade mode). Therefore, this avoids the player from clicking too much and provides a clearer understanding of what is going on in the game, also improving the gameplay since players are less confused.	Not met.

Overall, many of the success criteria weren't met due to time constraints and would naturally be met if I continued the development of the game (in the general order of the acceptance tests). In addition, any feedback from my clients would also be implemented alongside the development. This ensures that I would meet all the success criteria and solve the problem according to my clients' wishes.

If I was to implement the success criteria that wasn't met, I might implement it in the following ways:

At least three playable levels

> Once I had completed the first level, because the game's levels and code are modular, I wouldn't be required to implement much code for the other two levels. All I would have to do is create the maps on Tiled and then convert them into csv files so it can be imported into the game. Then I would have to check that everything is in the right positions and allow the entrance of the next level's map to load the map.

The parts that were remaining for the first level were the ability to return to the main map, coins, a timer, gainable trophies, and the completion screens. These all would've been modular and could be shared between the levels.

One mini boss fight

This would've required me to create another map and potentially another class for the mini boss that would inherit the Enemy class. This is because the boss would have different actions compared to the normal enemies and would be defeated differently.

My initial ideas for the mini boss fight would be to have the mini boss on top of a platform while attacking the player through long range attacks and spawning normal enemies near the player. The player would be able to defeat the boss by pressing the buttons that would appear and cause the boss damage until the boss' health reached zero.

This would mean that I would also have to create a way for the boss to be able to spawn the enemies in free positions and I would have to create interactable objects that would cause the boss harm when they are interacted with. This also means I would need extra particle animations to indicate the boss being harmed.

Arcade mode

This would require me to add methods to the Level class so that waves of enemies can appear for the player to fight and so the difficulty increases as the player completes more waves. In addition to creating another map in Tiled and importing it into the game using csv files.

At least three working enemies

For this, I would have to create more methods for the enemies and differentiate the Enemy class' attacks based on their monster_name. This is because all the enemies have very varied attack methods.

The slime enemy with the long-range attack would have to be altered to increase their attack range and to stop following the player once they are within the attack range. This is because the enemy wouldn't need to get closer to the player once they are in attack range logically. I would also have to create a way for the long-range attack to animate correctly and hit the player and cause damage.

The cyclops enemy is meant to have two different attacks so I would have to create logic for the enemy to be able to choose which attack type is more suitable for the current situation. The particle animation also needs to be on the enemy instead of the player when they stomp.

I also needed to ensure that the enemies are killed according to the amount of health they have (3 health means that the enemy should be killed in three hits from the player).

Player WASD movement and attack/block

The blocking would require any attack on the player to be nulled and instead of taking damage, the player must be pushed back like how enemies are once they are attacked. This would be easy to implement and may only require an extra block method with the Player class, and a few more attributes. The player being pushed back gives them incentive to not hold block forever since they could get pushed off the map and lose health.

Collectibles

This requires me to create another sprite class for the coins so they can be collected and collisions with the player can be checked. This is in addition to adding an overall coins attribute to the player and attributes within the level to show how many coins were gained in the current level. I would also have to find a way to cancel the coins earned by the player in a level if they end up failing the level but this can be overcome by only adding the coins gained in the level to the player's overall coins once they've reached the end.

NPCs and NPC dialogue

Since they already appear, I would have to create dialogue for each of the NPCs which would simply be displayed on the screen using a text box at the bottom of the screen if they are interacted with. The user should be able to interact with their dialogue and press interact again to get the NPC to say their next line. This might require the interact method to be further developed to allow this.

Trophy tracking

This would require me to create trophies that were feasible for the player to earn. I would also have to create a separate method within the Level class to check if the player has earned any trophies by completing a specific task. This would be based on the current level so it should only check the trophies for that level unless there were overall trophies that the player had completed. The trophies section then would have to be updated as well as the player's attributes for trophies they have earned and the earnable trophies.

Working menu

I would first need to find a way to pause the game in the background while the menu is open and then allow the player to select things within the menu using the WASD keys. The 'M' key should also be placed as an input for the Player class within the input method for opening and closing the menu. This means that I might have to create another flag attribute to check if the menu was open. The menu also needs to be able to change things within the settings (such as the sound) which may mean that I would have to create another class for the sound so its attributes can be changed by the player.

Save mechanism

This would require an extra file to be updated when the user saves their data so it can be updated and reloaded (e.g. within a notepad file). This also means that I would have to change the player's data so that it gathers some of its attributes' data from that file, instead of resetting the game whenever the user closes and reopens the game – making them lose their progress.

Title screen

This wouldn't take too much time and code to implement, though I would have to redesign the screen, so it suits my client's preferences. This is because it would just require the title screen image to be blitted to the screen and for the space key on the keyboard inputs to get rid of the title screen image and allow the player to play. This might require another flag to change the space key back to its original use for jumping.

Death screen

This would act similarly to the menu since the user should be able to select the options on the screen like the menu. The only difference is that it is bigger than the menu. The different options would have to be linked to the proper methods and the death screen should appear for the correct

context (level death screen or boss battle death screen?) and only when the player runs out of health.

Loading screens

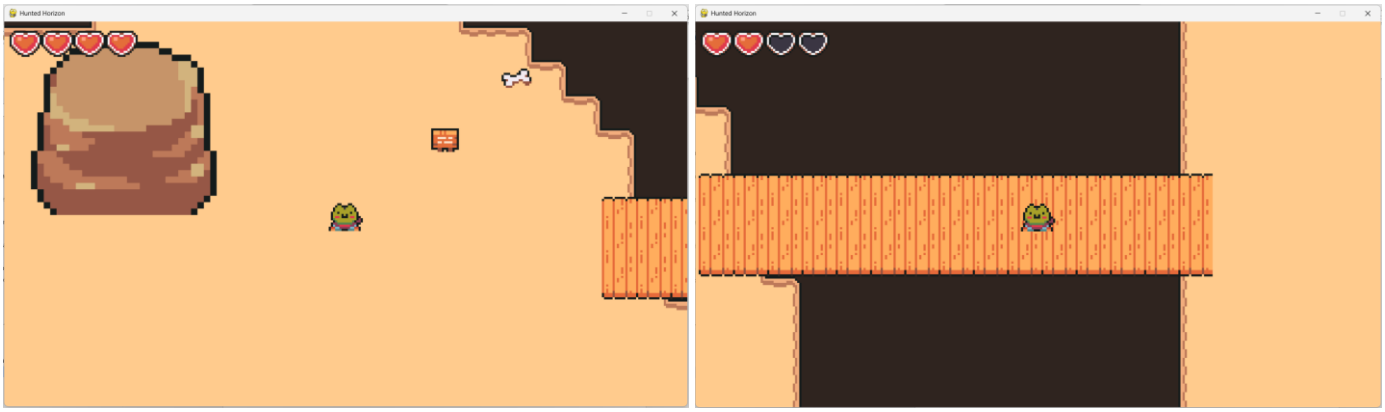
The loading screen would be coded similarly to the title screen and wouldn't take too long to implement. Though it may take more time if I fulfil my client's wish of having an animated loading screen with the player sprite walking. The loading screen would have to be blitted whenever the player interacts with an entrance, so the loading of maps seems more professional.

Usability Features

HUD

The HUD appears when you enter the first level and correctly updates according to the player's health. As shown in the original design, the hearts are displayed in the top left corner and are very visible for the player. I ensured that they weren't too big however, since it could make it harder to play the game since the user can see less of the map.

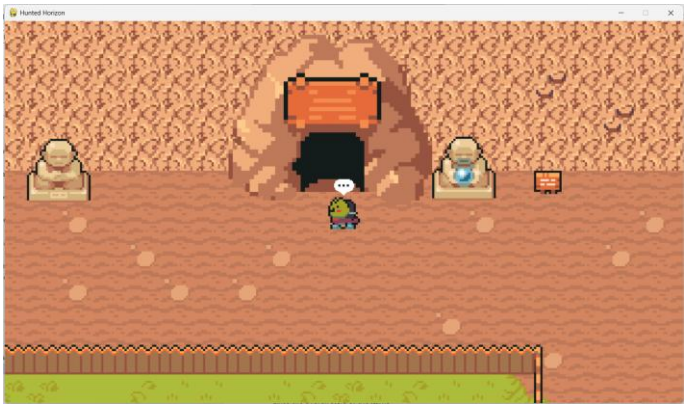
The HUD for the main map could've also been developed to show the player their stats like how many coins or trophies they have collected. In addition to the menu button reminder that my main client suggested.



Notice Bubble

The notice bubble appears above the player's head when there are things the player can interact with. Although this is different to the original design since the bubble is always above the player instead of being above the NPC or interactable object. However, this still conveys the same usability and would be even easier for the user to spot. In addition, it was easier to implement it this way and I was able to keep the bubble animation.

Because there isn't a clear visual indication that the player has interacted with the interactable items, I could've also made the notice bubble become highlighted once the player interacts with the object.



Any usability features that I couldn't implement would be added and altered according to the client's suggestions if I continued development. Most of the unimplemented usability features are apart of the success criteria targets so I would approach them as mentioned in the evaluation of the success criteria targets.

If I was to change the usability features according to my client's feedback, this is what it may look like. These models were again created within **PowerPoint**.

Main map HUD

User is now informed of how to open the menu using the 'M' key on the keyboard on the main map. Placed on the top right in small text so it isn't distracting from the main map.

This is a new HUD that would be added to inform the player of how to open the menu so they can view the other controls. However, I would still be assuming that the user has knowledge of other games and will realise they can use the WASD keys to navigate the menu.

Level Completion Screen

Collectibles have been changed to coins since that's what I had later decided on having the collectibles as. The number has also been altered to be more realistic.

The number of deaths has been added to the level completion screen as requested by my client. The colour of the number is the same colour as the background of the death screen.

My client also asked what it may look like if the player has earned multiple trophies so I have also created a design of what it may look like:

The name of the other trophy is set below, and the trophy symbol has a multiplication next to it, indicating how many trophies they've earned. If the player earns more than 2 trophies, the number in the multiplication will reflect that and the names of all the trophies will be displayed next to it.

This would be the same concept for any of the other completion screens as well if the player has earned multiple trophies.

Arcade Mode Completion Screen

Unlike how the client requested, I decided it would still benefit the user to have the coins displayed so they can know how many they have earned. Therefore, I kept the coins on the screen.

The time taken and enemies defeated have been added as requested. The enemies defeated number is in a dark colour to stand out from the background.

Trophy Screen

The trophies would be able to be selected to check their information. The currently selected trophy will highlight its name in white, making it easy for the user to navigate the screen.

Trophy Information Screen

Both screens would appear after the user has selected to see a trophy's information. The different screens appear depending on whether the user had earned the trophy or not yet.

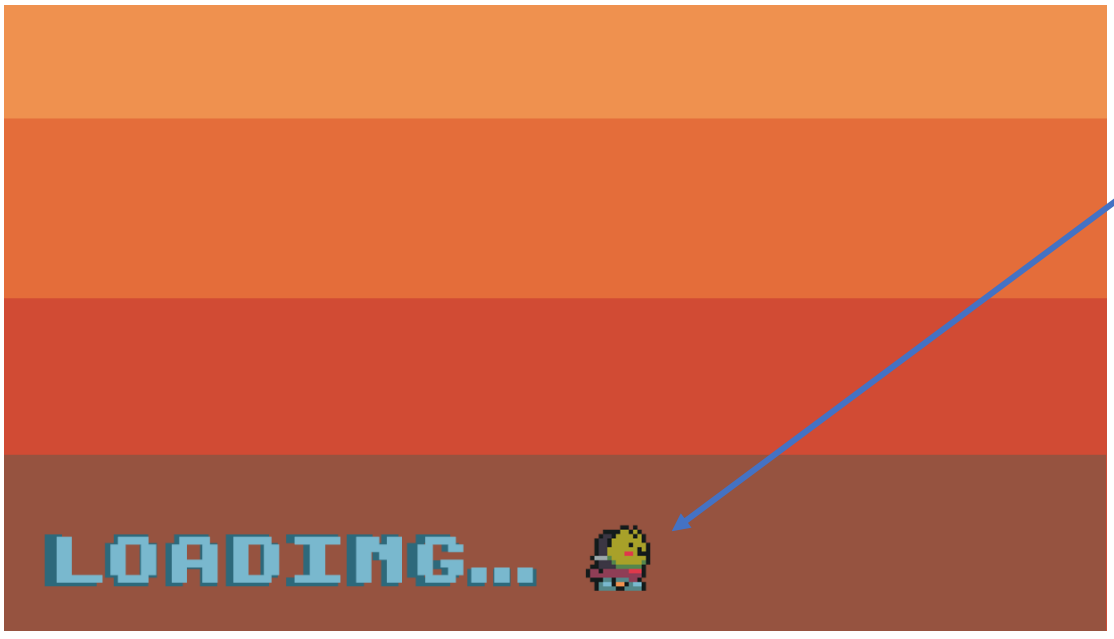
Visual indication of whether trophy has been earned which is clear for the user.

The user can only back out of this screen since it only needs to display information and nothing else needs to be selected.

The title and description of the trophy is displayed so the user knows which trophy they have selected and how to earn it. The status also shows whether the user has earned it or not.

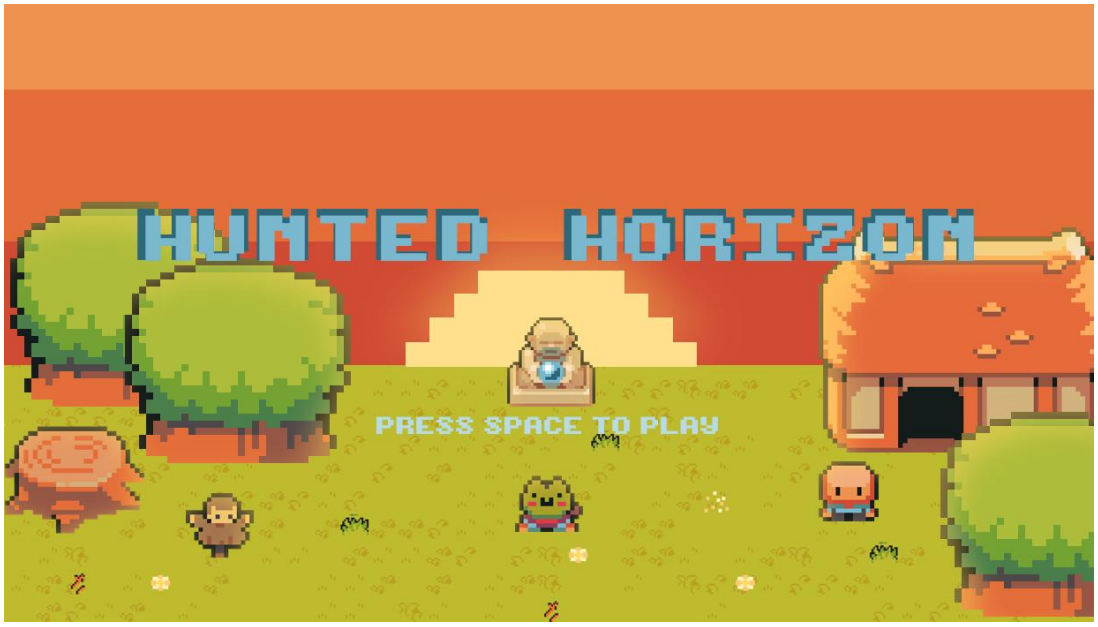
The trophy is grey to indicate that the user has yet to earn the trophy. This changes after they have earned it.

Loading Screen



Player can be seen walking on the loading screen, as requested by my client.

Title Screen



This is what the title screen may look like, but this may change according to my client's preferences if I was still developing the game.

Prototype Client Feedback

My main client provided his thoughts on the whole game (excluding the first level since that was given in the level 1 feedback).

He suggested adding rocks or obstacles to make the invisible game barriers more realistic and adding easter eggs like special collectables.

Overall, he said that he found the game fun to play, and he liked the style of the game. He also liked the concepts and ideas that were developed into the game.

I also agree with all my client's suggestions for improvement and think that the first level's invisible wall barriers can be altered so the walkable map area is slightly bigger. This is because it looked slightly weird when the player was blocked by nothing even though they could see a walkable platform. This would be in addition to adding the rocks and obstacles like my client mentioned.

The special collectables would also make it easy for me to add more trophies for the user since they could get a trophy for finding all the special collectables.

Maintenance

If the game was fully developed, it would simply require to be tested to ensure there are no errors or bugs. This is because the game is not online and wouldn't require a server to be played, it would just need to be downloaded and ran.

However, if there was a bug that I didn't manage to find (which is common when developing a game), I would have to see the error and try to figure out why it occurs in my code. Since my solution would be for multiple people, I may upload the game on a store (e.g. Steam or itch.io) so people can download the game. This would allow users to raise issues if there was an error and I would be able to reupload the fixed game – making maintenance easy.

Limitations

One limitation that my users may have is **storage issues** but this could be avoided by utilising compression techniques to avoid this. I could also create a separate version of the game for users with less storage so that they can still play the game.

One of the main limitations of my prototype is that it is coded using **Pygame**. Although it is possible to complete my solution using Pygame, I realised while developing that many of the features were difficult to implement and were time consuming. If I had used a game engine such as Godot, it may have been easier to create my solution, and I could've implemented more features. One example would be the entire 2D mimicking 3D aspect of my game - it wouldn't need to be done since Godot supports the creation of both 2D and 3D games and I could make the game 3D instead but make it look 2D. Godot also has its own scripting language that is called GDScript that is very similar to Python so I wouldn't have to learn too much if I was to use Godot.

Another limitation of my prototype is that I relied on the use of **asset packs** which, although helped me to save time, it also limited what I could create in my game. This is because if there wasn't certain images and animations, I had to create them myself which took time or forgo my ideas.

One other limitation was **my knowledge** of Pygame and game creation. This meant that, because I wasn't fully familiar with Pygame and creating games with it, I had to spend a lot of time trying to figure out how to code certain parts of the game which slowed down development. This was especially the case when I was trying to work out the jumping mechanism and is why it's still not fully without glitches. However, I did also try to learn Pygame prior to starting development which did help slightly since I understood the concepts that I had learnt.

Code Listings

main.py > ...

```python
import pygame, sys
from settings import *
from level import Level

class Game:
    def __init__(self):

        # General setup
        self.screen = pygame.display.set_mode((WIDTH,HEIGHT))
        pygame.display.set_caption('Hunted Horizon')
        self.clock = pygame.time.Clock()

        self.level = Level()

    def run(self):
        while True:
            for event in pygame.event.get():

                # Allows user to close program
                if event.type == pygame.QUIT:
                    pygame.quit()
                    sys.exit()

            # Screen
            self.screen.fill('black')
            self.level.run()
            pygame.display.update()
            self.clock.tick(FPS)

if __name__ == '__main__':
    game = Game()
    game.run()
```

particles.py > ...

```python
import pygame
from support import import_folder

class AnimationPlayer:
    def __init__(self):
        self.frames = {
            # Attacks
            'slash': import_folder('../graphics/particles/slash'),
            'claw': import_folder('../graphics/particles/claw'),
            'ice': import_folder('../graphics/particles/ice'),
            'stomp': import_folder('../graphics/particles/stomp')
        }

    def create_particles(self,animation_type,pos,groups):
        animation_frames = self.frames[animation_type]
        ParticleEffect(pos,animation_frames,groups)
```

particles.py > ...

```python
class ParticleEffect(pygame.sprite.Sprite):
    def __init__(self,pos,animation_frames,groups):
        super().__init__(groups)
        self.frame_index = 0
        self.animation_speed = 0.15
        self.frames = animation_frames
        self.image = self.frames[self.frame_index]
        self.rect = self.image.get_rect(center=pos)

    def animate(self):
        self.frame_index += self.animation_speed
        if self.frame_index >= len(self.frames):
            self.kill()
        else:
            self.image = self.frames[int(self.frame_index)]

    def update(self):
        self.animate()
```

level.py > Level

```python
import pygame
from settings import *
from tile import Tile
from player import Player
from support import *
from weapon import Weapon
from interact import Notice_Bubble
from ui import UI
from enemy import Enemy
from particles import AnimationPlayer
from debug import debug

class Level:
    def __init__(self):

        # Get display surface
        self.display_surface = pygame.display.get_surface()

        # Sprite group setup
        self.visible_sprites = YSortCameraGroup()
        self.obstacle_sprites = pygame.sprite.Group()
        self.interactable_sprites = pygame.sprite.Group()
        self.fall_sprites = pygame.sprite.Group()

        # Attack Sprites
        self.current_attack = None
        self.attack_sprites = pygame.sprite.Group()
        self.attackable_sprites = pygame.sprite.Group()

        # Notice Sprite
        self.current_notice = None
```

level.py > Level

```
13  class Level:
14      def __init__(self):
33          # Sprite setup
34          self.current_mode = game_mode.mode
35          self.create_map()
36
37          # User Interface
38          self.ui = UI()
39
40          # Particles
41          self.animation_player = AnimationPlayer()
42
43      def create_map(self):
44          graphics = {
45              'details': import_folder('../graphics/details'),
46              'objects': import_folder('../graphics/objects')
47          }
48
49          map_layouts = {
50              'mainmap': {'boundary': import_csv_layout('../map/mainmap/mainmap_Wallblocks.csv'),
51                          'interactable': import_csv_layout('../map/mainmap/mainmap_Interactable.csv'),
52                          'detail': import_csv_layout('../map/mainmap/mainmap_BigDetails.csv'),
53                          'object': import_csv_layout('../map/mainmap/mainmap_Objects.csv'),
54                          'entities': import_csv_layout('../map/mainmap/mainmap_Entities.csv')},
55              'level1': {'boundary': import_csv_layout('../map/level1/level1_Wallblocks.csv'),
56                          'detail': import_csv_layout('../map/level1/level1_BigDetails.csv'),
57                          'object': import_csv_layout('../map/level1/level1_Objects.csv'),
58                          'fallblock': import_csv_layout('../map/level1/level1_Fallblocks.csv'),
59                          'entities': import_csv_layout('../map/level1/level1_Entities.csv')}
60          }
```

level.py > Level

```
13  class Level:
43      def create_map(self):
62          for map,layouts in map_layouts.items():
63              if map == game_mode.mode:
64                  current_layouts = layouts
65
66          for style,layout in current_layouts.items():  # Items function retrieves the key and value pair
67              for row_index,row in enumerate(layout):  # Enumerate returns row index number for each row as row_index
68                  for col_index,col in enumerate(row):
69                      if col != '-1':
70                          x = col_index * TILESIZE
71                          y = row_index * TILESIZE
72                          if style == 'boundary':
73                              Tile((x,y),[self.obstacle_sprites],'invisible',col)  # col is the actual value of the tile
74                          if style == 'fallblock':
75                              Tile((x,y),[self.fall_sprites],'fallblock',col)
76                          if style == 'interactable':
77                              Tile((x,y),[self.interactable_sprites],'interactable',col)
78                          if style == 'detail':
79                              surf = graphics['details'][int(col)]
80                              Tile((x,y),[self.visible_sprites,self.obstacle_sprites],'detail',col,surf)
81                          if style == 'object':
82                              surf = graphics['objects'][int(col)]
83                              Tile((x,y),[self.visible_sprites,self.obstacle_sprites],'object',col,surf)
```

level.py > Level

```
13  class Level:
43      def create_map(self):
84                              if style == 'entities':
85                                  if col == '42':
86                                      self.player = Player(
87                                      (x,y),
88                                      [self.visible_sprites],
89                                      self.obstacle_sprites,
90                                      self.interactable_sprites,
91                                      self.fall_sprites,
92                                      self.create_attack,
93                                      self.destroy_attack,
94                                      self.notice)
95                                  else:
96                                      if col == '39': monster_name = 'slime'
97                                      elif col == '40': monster_name = 'cyclops'
98                                      else: monster_name = 'raccoon'
99                                      Enemy(
100                                         monster_name,
101                                         (x,y),
102                                         [self.visible_sprites,self.attackable_sprites],
103                                         self.obstacle_sprites,
104                                         self.damage_player)
```

level.py > Level

```
13  class Level:
106     def reload_map(self):
107         if self.current_mode != game_mode.mode:
108             for sprite in self.visible_sprites:
109                 sprite.kill()
110             for sprite in self.obstacle_sprites:
111                 sprite.kill()
112             for sprite in self.interactable_sprites:
113                 sprite.kill()
114             for sprite in self.fall_sprites:
115                 sprite.kill()
116
117             self.current_mode = game_mode.mode
118             self.visible_sprites = YSortCameraGroup()
119             self.create_map()
120
121     def create_attack(self):
122         self.current_attack = Weapon(self.player,[self.visible_sprites,self.attack_sprites])
123
124     def destroy_attack(self):
125         if self.current_attack:
126             self.current_attack.kill()
127         self.current_attack = None
128
129     def player_attack_logic(self):
130         if self.attack_sprites:
131             for attack_sprite in self.attack_sprites:
132                 collision_sprites = pygame.sprite.spritecollide(attack_sprite,self.attackable_sprites,False)
133                 if collision_sprites:
134                     for target_sprite in collision_sprites:
135                         target_sprite.get_damage(self.player)
```

level.py > Level

```
13  class Level:
137     def damage_player(self,attack_type):
138         if self.player.vulnerable:
139             self.player.change_health('lose')
140             self.player.vulnerable = False
141             self.player.hurt_time = pygame.time.get_ticks()
142             self.animation_player.create_particles(attack_type,self.player.rect.center,[self.visible_sprites])
143
144     def notice(self):
145         if self.player.noticing and self.current_notice == None:
146            self.current_notice = Notice_Bubble(self.player,[self.visible_sprites])
147         elif self.current_notice and not self.player.noticing:
148             self.current_notice.kill()
149             self.current_notice = None
150
151         if self.current_notice:
152             self.current_notice.update()
153
154     def run(self):
155         # Update + draw game
156         self.visible_sprites.custom_draw(self.player)
157         self.visible_sprites.update()
158         self.visible_sprites.enemy_update(self.player)
159         self.player_attack_logic()
160         self.ui.display(self.player)
161         self.reload_map()
```

level.py > ...

```
163 class YSortCameraGroup(pygame.sprite.Group):
164     def __init__(self):
165
166         # General setup
167         super().__init__()
168         self.display_surface = pygame.display.get_surface()
169         self.half_width = self.display_surface.get_size()[0] // 2
170         self.half_height = self.display_surface.get_size()[1] // 2
171         self.offset = pygame.math.Vector2()
172
173         # Floor Creation
174         self.floor_surf = pygame.image.load(f'../graphics/tilemap/{game_mode.mode}_ground.png').convert()
175         self.floor_rect = self.floor_surf.get_rect(topleft=(0,0))
176
177     def custom_draw(self,player):
178             # Getting offset
179             self.offset.x = player.rect.centerx - self.half_width
180             self.offset.y = player.rect.centery - self.half_height
181
182             # Drawing the Floor
183             floor_offset_pos = self.floor_rect.topleft - self.offset
184             self.display_surface.blit(self.floor_surf,floor_offset_pos)
185
186             # Drawing the Sprites
187             for sprite in sorted(self.sprites(),key=lambda sprite: sprite.rect.centery):
188                 offset_pos = sprite.rect.topleft - self.offset
189                 self.display_surface.blit(sprite.image,offset_pos)
190
191     def enemy_update(self,player):
192         enemy_sprites = [sprite for sprite in self.sprites() if hasattr(sprite,'sprite_type') and sprite.sprite_type == 'enemy']
193         for enemy in enemy_sprites:
194             enemy.enemy_update(player)
195         #debug(enemy_sprites[14].can_attack)
```

player.py > Player

```python
import pygame
from settings import *
from support import import_folder
from entity import Entity
from debug import debug

class Player(Entity):
    def __init__(self,pos,groups,obstacle_sprites,interactable_sprites,fall_sprites,create_attack,destroy_attack,notice):
        super().__init__(groups)
        self.image = pygame.image.load('../graphics/test/player.png').convert_alpha()
        self.rect = self.image.get_rect(topleft = pos)
        self.hitbox = self.rect.inflate(0,-26)
        self.fall_hitbox = self.rect.inflate(-50,-62)
        self.fall_hitbox.bottom = self.hitbox.bottom

        self.width = 64
        self.height = 64

        # Graphic Setup
        self.import_player_assets()
        self.status = 'down'

        # Player movement
        self.speed = 5

        self.current_ground = self.hitbox.y
        self.gravity = 0.8
        self.jump_force = -15
        self.vertical_speed = 0
        self.jumping = False
        self.down = False
        self.jump_released = True
        self.jump_pos = None
        self.falling = False
```

player.py > Player

```python
    class Player(Entity):
        def __init__(self,pos,groups,obstacle_sprites,interactable_sprites,fall_sprites,create_attack,destroy_attack,notice):
            self.attacking = False
            self.attack_cooldown = 400
            self.attack_time = None

            # Weapon
            self.create_attack = create_attack
            self.destroy_attack = destroy_attack
            self.weapon = list(weapon_data.keys())[0]

            # Notice + Interacting
            self.notice = notice
            self.noticing = False
            self.notice_id = None
            self.notice_response = None

            self.obstacle_sprites = obstacle_sprites
            self.interactable_sprites = interactable_sprites
            self.fall_sprites = fall_sprites

            # Stats
            self.health = 4
            self.coins = 123

            # Damage Timer
            self.vulnerable = True
            self.hurt_time = None
            self.invulnerability_duration = 500

            # Repositioning
            self.spawn_pos = self.hitbox.center
            self.repos = None
```

player.py > Player

```
7   class Player(Entity):
68      def import_player_assets(self):
69          character_path = '../graphics/player/'
70          self.animations = {'up':[], 'down':[], 'left':[], 'right':[],
71              'up_idle':[], 'down_idle':[], 'left_idle':[], 'right_idle':[],
72              'up_attack':[], 'down_attack':[], 'left_attack':[], 'right_attack':[],
73              'up_jump':[], 'down_jump':[], 'left_jump':[], 'right_jump':[]}
74
75          for animation in self.animations.keys():
76              full_path = character_path + animation
77              self.animations[animation] = import_folder(full_path)
78
79      def input(self):
80          keys = pygame.key.get_pressed()
81
82          if not self.attacking and not self.falling:
83              if keys[pygame.K_w]:
84                  self.direction.y = -1
85                  self.current_ground = self.hitbox.y
86                  self.status = 'up'
87              elif keys[pygame.K_s]:
88                  self.direction.y = 1
89                  self.current_ground = self.hitbox.y
90                  self.down = True
91                  self.status = 'down'
92              else:
93                  self.direction.y = 0
94                  self.down = False
```

player.py > Player

```
 7   class Player(Entity):
79       def input(self):
96               if keys[pygame.K_a]:
97                   self.direction.x = -1
98                   self.status = 'left'
99               elif keys[pygame.K_d]:
100                  self.direction.x = 1
101                  self.status = 'right'
102              else:
103                  self.direction.x = 0
104
105              if keys[pygame.K_SPACE] and self.jump_released:
106                  self.jump()
107                  self.jump_released = False
108
109              if not keys[pygame.K_SPACE]:
110                  self.jump_released = True
111
112              if keys[pygame.K_j] and not self.jumping:
113                  self.attacking = True
114                  self.attack_time = pygame.time.get_ticks()
115                  self.create_attack()
116
117              if keys[pygame.K_k]:
118                  print('block')
119
120              if keys[pygame.K_l]:
121                  self.interact()
```

player.py > Player

```
7   class Player(Entity):
123     def get_status(self):
124         # Idle status
125         if self.direction.x == 0 and self.direction.y == 0:
126             if not 'idle' in self.status and not 'attack' in self.status and not 'jump' in self.status:
127                 self.status = self.status + '_idle'
128 
129         if self.attacking:
130             self.direction.x = 0   # Stops player from moving while attacking
131             self.direction.y = 0   # ^
132             if not 'attack' in self.status:
133                 if 'idle' in self.status:
134                     self.status = self.status.replace('_idle','_attack')
135                 elif 'jump' in self.status:
136                     self.status = self.status.replace('_jump','_attack')
137                 else:
138                     self.status = self.status + '_attack'
139         else:
140             if 'attack' in self.status:
141                 self.status = self.status.replace('_attack','')
142 
143         if self.jumping:
144             if not 'jump' in self.status:
145                 if 'idle' in self.status:
146                     self.status = self.status.replace('_idle','_jump')
147                 elif 'attack' in self.status:
148                     self.status = self.status.replace('_attack','_jump')
149                 else:
150                     self.status = self.status + '_jump'
151         else:
152             if 'jump' in self.status:
153                 self.status = self.status.replace('_jump','')
```

player.py > Player

```
7   class Player(Entity):
```

```
155     def check_fall(self):
156         self.fall_hitbox.midbottom = self.hitbox.midbottom
157
158         if self.fall_hitbox.collidelist(list(self.fall_sprites)) != -1 and not self.jumping:
159             self.falling = True
160             self.direction.x = 0
161             self.direction.y = 0
162
163     def check_notice(self):
164         self.noticing = False
165         self.notice_id = None
166         self.notice_response = None
167         for sprite in self.interactable_sprites:
168             if sprite.hitbox.colliderect(self.hitbox):
169                 self.noticing = True
170                 self.notice_id = sprite.id
171                 self.notice_response = sprite.response
172
173         for object in self.obstacle_sprites:
174             if object.notice_radius:
175                 if object.notice_radius.colliderect(self.hitbox):
176                     self.noticing = True
177                     self.notice_id = object.id
178                     self.notice_response = object.response
179
180     def interact(self):
181         if self.noticing and self.notice_id:
182             print(self.notice_response)
183             if int(self.notice_id) == 19:
184                 game_mode.mode = 'level1'
```

player.py > Player

```
7   class Player(Entity):
```

```
186     def cooldowns(self):
187         current_time = pygame.time.get_ticks()
188
189         if self.attacking:
190             if (current_time-self.attack_time) >= self.attack_cooldown + weapon_data[self.weapon]['cooldown']:
191                 self.attacking = False
192                 self.destroy_attack()
193
194         if not self.vulnerable:
195             if (current_time - self.hurt_time) >= self.invulnerability_duration:
196                 self.vulnerable = True
197
198     def jump(self):
199         if not self.jumping:
200             self.jumping = True
201             self.vertical_speed = self.jump_force
202             self.jump_pos = self.hitbox.center
203
204     def apply_gravity(self):
205         if self.jumping and self.down:
206             self.vertical_speed += self.gravity
207             self.hitbox.y -= self.vertical_speed
208
209             if self.hitbox.y <= self.current_ground:
210                 self.hitbox.y = self.current_ground
211                 self.jumping = False
212                 self.vertical_speed = 0
```

player.py > Player

```
  7   class Player(Entity):
204       def apply_gravity(self):
214           elif self.jumping:
215               self.vertical_speed += self.gravity
216               self.hitbox.y += self.vertical_speed
217
218               if self.hitbox.y >= self.current_ground:
219                   self.hitbox.y = self.current_ground
220                   self.jumping = False
221                   self.vertical_speed = 0
222
223       def fall(self):
224           if self.falling:
225               pos = self.rect.topleft
226               if self.width > 0 and self.height > 0:
227                   self.image = pygame.transform.scale(self.image,(int(self.width),int(self.height)))
228                   self.rect = self.image.get_rect(topleft = pos)
229                   self.width -= 1.5
230                   self.height -= 1.5
231                   self.hitbox.y += 3
232               else:
233                   self.falling = False
234                   self.change_health('lose')
235                   if self.jump_pos:
236                       self.hitbox.center = self.jump_pos
237                   else:
238                       self.hitbox.center = self.spawn_pos
239                   self.width = 64
240                   self.height = 64
```

player.py > Player

```
7   class Player(Entity):
242     def change_health(self,change):
243         if change == 'lose':
244             if self.health > 0:
245                 self.health -= 1
246                 if self.health == 0:
247                     print('Game Over')
248         if change == 'gain':
249             self.health = 4
250
251     def animate(self):
252         animation = self.animations[self.status]
253
254         # Looping over frame index
255         self.frame_index += self.animation_speed
256         if self.frame_index >= len(animation):
257             self.frame_index = 0
258
259         self.image = animation[int(self.frame_index)]
260         self.rect = self.image.get_rect(center = self.hitbox.center)
261
262         # Flicker
263         if not self.vulnerable:
264             alpha = self.wave_value()
265             self.image.set_alpha(alpha)
266         else:
267             self.image.set_alpha(255)
```

player.py > Player

```
7   class Player(Entity):
270     def update(self):
271         self.input()
272         self.cooldowns()
273         self.get_status()
274         self.animate()
275         self.move(self.speed)
276         self.apply_gravity()
277         self.check_notice()
278         self.notice()
279         self.check_fall()
280         self.fall()
```

enemy.py > Enemy

```python
import pygame
from settings import *
from entity import Entity
from support import *

class Enemy(Entity):
    def __init__(self,monster_name,pos,groups,obstacle_sprites,damage_player):
        super().__init__(groups)
        self.sprite_type = 'enemy'

        # Graphics
        self.import_graphics(monster_name)
        self.status = 'idle_down'
        self.status_direction = 'down'
        self.image = self.animations[self.status][self.frame_index]

        # Movement
        self.rect = self.image.get_rect(topleft=pos)
        self.hitbox = self.rect.inflate(0,-10)
        self.obstacle_sprites = obstacle_sprites

        # Stats
        self.monster_name = monster_name
        monster_info = monster_data[self.monster_name]
        self.health = monster_info['health']
        self.speed = monster_info['speed']
        self.attack_damage = monster_info['damage']
        self.resistance = monster_info['resistance']
        self.attack_radius = monster_info['attack_radius']
        self.notice_radius = monster_info['notice_radius']
        self.attack_type = monster_info['attack_type']
```

enemy.py > Enemy

```python
class Enemy(Entity):
    def __init__(self,monster_name,pos,groups,obstacle_sprites,damage_player):
        # Player Interaction
        self.can_attack = True
        self.attack_time = None
        self.attack_cooldown = 400
        self.damage_player = damage_player

        # Invincibilty Timer
        self.vulnerable = True
        self.hit_time = None
        self.invincibility_duration = 300

    def import_graphics(self,name):
        self.animations = {'idle_up':[],'idle_down':[],'idle_right':[],'idle_left':[],
            'attack_up':[],'attack_down':[],'attack_right':[],'attack_left':[],
            'move_up':[], 'move_down':[], 'move_right':[], 'move_left':[]}
        main_path = f'../graphics/monsters/{name}/'
        for animation in self.animations.keys():
            self.animations[animation] = import_folder(main_path + animation)

    def get_player_distance_direction(self,player):
        enemy_vec = pygame.math.Vector2(self.hitbox.center)
        player_vec = pygame.math.Vector2(player.hitbox.center)
        distance = (player_vec - enemy_vec).magnitude()

        if distance > 0:
            direction = (player_vec - enemy_vec).normalize()
        else:
            direction = pygame.math.Vector2()  # Gives vector of (0,0)

        return (distance,direction)
```

enemy.py > Enemy

```
6   class Enemy(Entity):
64      def get_status(self,player):
65          distance = self.get_player_distance_direction(player)[0]
66
67          if self.direction[0] != 0 and self.direction[1] != 0:
68              if abs(self.direction[0]) < abs(self.direction[1]):
69                  if self.direction[1] > 0:
70                      self.status_direction = 'down'
71                  else:
72                      self.status_direction = 'up'
73              else:
74                  if self.direction[0] > 0:
75                      self.status_direction = 'right'
76                  else:
77                      self.status_direction = 'left'
78
79          if distance <= self.attack_radius and self.can_attack:
80              if self.status[:6] != 'attack':
81                  self.frame_index = 0
82              self.status = f'attack_{self.status_direction}'
83          elif distance <= self.notice_radius:
84              self.status = f'move_{self.status_direction}'
85          else:
86              self.status = f'idle_{self.status_direction}'
87
88      def actions(self,player):
89          if self.status[:6] == 'attack':
90              self.attack_time = pygame.time.get_ticks()
91              self.damage_player(self.attack_type)
92          elif self.status[:4] == 'move':
93              self.direction = self.get_player_distance_direction(player)[1]
94          else:
95              self.direction = pygame.math.Vector2()
```

enemy.py > Enemy

```
6   class Enemy(Entity):
97      def animate(self):
98          animation = self.animations[self.status]
99
100         self.frame_index += self.animation_speed
101         if self.frame_index >= len(animation):
102             if self.status[:6] == 'attack':
103                 self.can_attack = False
104             self.frame_index = 0
105
106         self.image = animation[int(self.frame_index)]
107         self.rect = self.image.get_rect(center=self.hitbox.center)
108
109         if not self.vulnerable:
110             alpha = self.wave_value()
111             self.image.set_alpha(alpha)
112         else:
113             self.image.set_alpha(255)  # Full opacity
114
115     def cooldowns(self):
116         current_time = pygame.time.get_ticks()
117         if not self.can_attack:
118             if (current_time - self.attack_time) >= self.attack_cooldown:
119                 self.can_attack = True
120
121         if not self.vulnerable:
122             if (current_time - self.hit_time) >= self.invincibility_duration:
123                 self.vulnerable = True
124
125     def get_damage(self,player):
126         if self.vulnerable:
127             self.direction = self.get_player_distance_direction(player)[1]
128             self.health -= weapon_data[player.weapon]['damage']
129             self.hit_time = pygame.time.get_ticks()
```

enemy.py > Enemy

```
  6   class Enemy(Entity):
125       def get_damage(self,player):
130               self.vulnerable = False
131
132       def check_death(self):
133           if self.health <= 0:
134               self.kill()
135
136       def hit_reaction(self):
137           if not self.vulnerable:
138               self.direction *= -self.resistance
139
140       def update(self):
141           self.hit_reaction()
142           self.move(self.speed)
143           self.animate()
144           self.cooldowns()
145           self.check_death()
146
147       def enemy_update(self,player):
148           self.get_status(player)
149           self.actions(player)
```

entity.py > ...

```
 1  import pygame
 2  from math import sin
 3
 4  class Entity(pygame.sprite.Sprite):
 5      def __init__(self,groups):
 6          super().__init__(groups)
 7          self.frame_index = 0
 8          self.animation_speed = 0.15
 9          self.direction = pygame.math.Vector2()
10
11      def move(self,speed):
12          if self.direction.magnitude() != 0:
13              self.direction = self.direction.normalize()  # Makes the direction vector always 1
14
15          self.hitbox.x += self.direction.x * speed
16          self.collision('horizontal')
17          self.hitbox.y += self.direction.y * speed
18          self.collision('vertical')
19          self.rect.center = self.hitbox.center
20
21      def collision(self,direction):
22          if direction == 'horizontal':
23              for sprite in self.obstacle_sprites:
24                  if sprite.hitbox.colliderect(self.hitbox):  # Checks if obstacle and player are colliding
25                      if self.direction.x > 0:  # Moving right
26                          self.hitbox.right = sprite.hitbox.left
27                      if self.direction.x < 0:  # Moving left
28                          self.hitbox.left = sprite.hitbox.right
```

entity.py > ...

```
 4   class Entity(pygame.sprite.Sprite):
21       def collision(self,direction):
30           if direction == 'vertical':
31               for sprite in self.obstacle_sprites:
32                   if sprite.hitbox.colliderect(self.hitbox):
33                       if self.direction.y > 0:  # Moving down
34                           self.hitbox.bottom = sprite.hitbox.top
35                       if self.direction.y < 0:  # Moving up
36                           self.hitbox.top = sprite.hitbox.bottom
37
38       def wave_value(self):
39           value = sin(pygame.time.get_ticks())
40           if value >= 0:
41               return 255
42           else:
43               return 0
```

ui.py > ...

```
 1   import pygame
 2   from settings import *
 3
 4   class UI:
 5       def __init__(self):
 6           # General
 7           self.display_surface = pygame.display.get_surface()
 8           self.font = pygame.font.Font(UI_FONT,UI_FONT_SIZE)
 9
10       def show_health(self,current):
11           self.health_img = pygame.image.load(f'../graphics/health/{current}.png').convert_alpha()
12           self.display_surface.blit(self.health_img,(10,10))
13
14       def display(self,player):
15           if game_mode.mode[:5] == 'level':
16               self.show_health(player.health)
```

settings.py > ...

```python
# Game setup
WIDTH = 1280
HEIGHT = 720
FPS = 60
TILESIZE = 64

class MODE:
    def __init__(self):
        self.mode = 'mainmap'

game_mode = MODE()

# UI
UI_FONT = '../graphics/font/NormalFont.ttf'
UI_FONT_SIZE = 18
TEXT_COLOUR = '#EEEEEE'

weapon_data = {
    'sword': {'cooldown':100, 'damage':1, 'graphic':'../graphics/weapons/sword/full.png'}
}

interactable_objects = {
    23: 'Boss',
    32: 'Arcade',
    37: 'Trophies',
    43: 'Inspector',
    44: 'Old man',
    45: 'Villager 1',
    46: 'Villager 2'
}
```

settings.py > ...

```python
monster_data = {
    'raccoon': {'health':4, 'damage':1, 'attack_type':'claw', 'attack_sound':'../audio/attack/claw.wav', 'speed':3, 'resistance':3, 'attack_radius':40, 'notice_radius':350},
    'cyclops': {'health':4, 'damage':1, 'attack_type':'stomp', 'attack_sound':'../audio/attack/stomp.wav', 'speed':2, 'resistance':3, 'attack_radius':60, 'notice_radius':350},
    'slime': {'health':3, 'damage':1, 'attack_type':'ice', 'attack_sound':'../audio/attack/ice.wav', 'speed':1, 'resistance':3, 'attack_radius':120, 'notice_radius':400}
}
```

weapon.py > ...

```python
import pygame

class Weapon(pygame.sprite.Sprite):
    def __init__(self,player,groups):
        super().__init__(groups)
        direction = player.status.split('_')[0]

        # Graphics
        full_path = f'../graphics/weapons/sword/{direction}.png'
        self.image = pygame.image.load(full_path).convert_alpha()

        # Placement
        if direction == 'right':
            self.rect = self.image.get_rect(midleft=(player.rect.midright + pygame.math.Vector2(0,16)))
        elif direction == 'left':
            self.rect = self.image.get_rect(midright=(player.rect.midleft + pygame.math.Vector2(0,16)))
        elif direction == 'down':
            self.rect = self.image.get_rect(midtop=(player.rect.midbottom + pygame.math.Vector2(-10,0)))
        else:
            self.rect = self.image.get_rect(midbottom=(player.rect.midtop + pygame.math.Vector2(-10,0)))
```

interact.py > Notice_Bubble

```python
import pygame

class Notice_Bubble(pygame.sprite.Sprite):
    def __init__(self,player,groups):
        super().__init__(groups)

        self.animation = ['0','1','2','3']
        self.frame_index = 0
        self.animation_speed = 0.025

        self.direction = player.direction
        self.player = player
        self.player_top = player.rect.midtop

        self.image = pygame.image.load('../graphics/notice/0.png').convert_alpha()
        self.rect = self.image.get_rect(midbottom=self.player_top)

    def follow_player(self,player):
        if self.direction != [0,0]:
            self.player_top = player.rect.midtop
            self.rect.midbottom = self.player_top

    def animate(self):
        self.frame_index += self.animation_speed
        if self.frame_index >= len(self.animation):
            self.frame_index = 0
        full_path = f'../graphics/notice/{self.animation[int(self.frame_index)]}.png'
        self.image = pygame.image.load(full_path).convert_alpha()

    def update(self):
        self.follow_player(self.player)
        self.animate()
```

debug.py > ...

```python
# Displays information for debugging on screen
import pygame
pygame.init()
font = pygame.font.Font(None,30)

def debug(info, y=10, x=500):
    display_surface = pygame.display.get_surface()
    debug_surf = font.render(str(info),True,'White')
    debug_rect = debug_surf.get_rect(topleft=(x,y))
    pygame.draw.rect(display_surface,'Black',debug_rect)
    display_surface.blit(debug_surf,debug_rect)
```

support.py > ...

```python
import pygame
from csv import reader
from os import walk

def import_csv_layout(path):
    terrain_map = []
    with open(path) as level_map:
        layout = reader(level_map,delimiter=',')
        for row in layout:
            terrain_map.append(list(row))
        return terrain_map

def import_folder(path):
    surface_list = []
    for _,__,img_files in walk(path):
        for image in img_files:
            full_path = path + '/' + image
            image_surf = pygame.image.load(full_path).convert_alpha()
            surface_list.append(image_surf)
    return surface_list
```

tile.py > ...

```python
import pygame
from settings import *

class Tile(pygame.sprite.Sprite):
    def __init__(self,pos,groups,sprite_type,id,surface=pygame.Surface((TILESIZE,TILESIZE))):
        super().__init__(groups)
        self.sprite_type = sprite_type
        self.image = surface
        self.id = id

        if sprite_type == 'object':
            self.rect = self.image.get_rect(topleft=(pos[0],pos[1]-TILESIZE))
        else:
            self.rect = self.image.get_rect(topleft = pos)

        self.hitbox = self.rect.inflate(0,-10)
        self.notice_radius = None

        if sprite_type == 'interactable':
            if int(self.id) == 17:
                self.response = 'Level 3'
            if int(self.id) == 18:
                self.response = 'Level 2'
            if int(self.id) == 19:
                self.response = 'Level 1'

        if sprite_type == 'object':
            for objectid,objectresponse in interactable_objects.items():
                if int(self.id) == objectid:
                    self.notice_radius = self.hitbox.inflate(50,100)
                    self.response = objectresponse
```

Bibliography

[1] https://en.wikipedia.org/wiki/Sackboy:_A_Big_Adventure

[2] https://www.youtube.com/watch?v=it9gbQl-z3Q

[3] https://en.wikipedia.org/wiki/Death%27s_Door_(video_game)

[4] https://www.youtube.com/watch?v=wqwxFC19OlU

[5] https://pixel-boy.itch.io/ninja-adventure-asset-pack

[6] https://www.geeksforgeeks.org/working-zip-files-python/

[7] https://docs.python.org/3/library/gzip.html

[8] https://docs.python.org/3/library/bz2.html

[9] https://www.youtube.com/@ClearCode

[10] https://www.youtube.com/watch?v=QU1pPzEGrqw

[11] https://www.youtube.com/watch?v=N6xqCwblyiw

[12] https://youtu.be/wJMDh9QGRgs?si=ZWmTbsT0ELLQQCCm

[13] https://www.youtube.com/@ClearAtlas

[14] https://www.youtube.com/watch?v=7H6_fcRZLJY

[15] https://www.pygame.org/docs/

[16] https://stackoverflow.com/questions/3400525/global-variable-from-a-different-file-python